

“Zero” open source project documentation

Generic graphic interface for user interaction with object graphs

Radosław Tereszczuk 2009

This software is not publicly available at this moment. For source code and other issues please contact tereszczuk@gmail.com

Abstract

“In-memory object graph as fully functional software” idea implementation is presented. Line-of-business feature set, model driven user interface generation framework with sound support for industry standard tools, languages and databases is described.

Solution can be characterized by: object model independence, being “object centric”, metadata provider independence, target UI medium independence, generation process full parameterization, Type Schema and Generators implementations plug-in system, ability to use any target UI specific control, 100% run time work and run time user customization abilities.

Table of Contents

1.	Introduction.....	6
1.1.	Information science context.....	6
1.2.	Topic overview	6
1.3.	State of the art	6
1.4.	Aim of this paper	7
1.5.	Solution.....	7
1.6.	Languages, tools and environment used	8
1.7.	Results summary	8
1.8.	Content of this paper	13
2.	The problem: User interface generation.....	14
2.1.	Model-driven architecture.....	14
2.2.	Generating UI from model.....	16
2.3.	State of the art	16
2.3.1.	Microsoft Access.....	16
2.3.2.	NakedObjects	18
2.3.3.	DynamicData	26
2.3.4.	XAML Power Toys.....	30
2.3.5.	State of the art summary	34
2.4.	Model-driven run time graphic interface generation	35
3.	Tools used	36
3.1.	Net Framework	36
3.2.	C# 3.0.....	36
3.3.	ASP.NET	36
3.4.	Wpf	36
5.	The solution: Generic graphic interface.....	38
5.1.	On a way to “in-memory object graph as fully functional software”	38
5.1.1.	Run-time object type structure as “The model”	38
5.1.2.	User interaction with object graph	40
5.1.3.	Model as multi user interfaces base	43
5.2.	Overall solution architecture overview	45
5.3.	The TypeSchema	47
5.3.1.	TypeSchema overview	48

5.3.2.	TypeSchema fields	49
5.3.3.	TypeSchema methods	50
5.3.4.	Metadata - TypeSchema attributes	50
5.3.5.	Working with TypeSchema	54
5.3.6.	ClrTypeSchema	55
5.4.	Generator and VisualizerItem	56
5.5.	The Visualizer	59
5.6.	The Session	61
5.6.1.	ObjectCollection	65
5.6.2.	Session Items	66
5.6.3.	Session Items Graphic Containers	66
5.6.4.	Navigators	67
5.6.5.	Session Item's Relations	68
5.6.5.1.	Master-detail relation	68
5.6.5.2.	Intensional navigation	68
5.6.5.3.	Extensional navigation	69
5.7.	The Session Manager	69
6.	Solution implementation	73
6.1.	Common attributes	73
6.1.1.	Description	73
6.1.2.	Ignore	75
6.1.3.	MethodsFirst	76
6.1.4.	ObjectSourceCollection	78
6.1.5.	TextMultiLine	79
6.1.6.	ValidateMethod	82
6.1.7.	VisualGroupName	83
6.1.8.	VisualName	86
6.2.	ASP.NET specific attributes	87
6.2.1.	UseControl	87
6.2.2.	UseGenerator	88
6.2.3.	UseVisualizerItem	88
6.3.	WPF specific attributes	88
6.3.1.	BulkColors	88

6.3.2.	ControlParameter	89
6.3.3.	GeneratorAttributes.....	90
6.3.4.	LabelParameter	92
6.3.5.	MethodControlParameter.....	94
6.3.6.	MethodUseControl.....	96
6.3.7.	TabName.....	98
6.3.8.	TabParameter	100
6.3.9.	UseControl.....	101
6.3.10.	UseGenerator	103
6.3.11.	UseVisualizerItem.....	103
6.3.12.	VerticalMargin.....	106
6.3.13.	VisualGroupParameter.....	107
6.3.14.	VisualGroupUseControl.....	108
6.3.15.	VisualizerParameter	109
7.	Solution applications, limitations and future work	111
7.1.	Applications	111
7.2.	Limitations	111
7.3.	Future work.....	111
	Summary	112
	Bibliography	113
	Appendix A. PowerPlant model.....	114
	Appendix B. PowerPlant.Employee TypeSchema.....	120

1. Introduction

1.1. Information science context

Computer software engineering is hard. Nowadays software systems become everyday life's ordinary components. They are connected and interdependent. And their complexity is growing. Nowadays computer software engineering is even harder than before.

In context of having not enough resources (time, skilled programmers etc) for developing software, new projects are often cancelled or even not started. Software industry is continuously developing solutions to those problems – some of the solutions work, some of them are more of wishful thinking .

Model driven architecture and user interface generation are one of those solutions. We want to show that those can be successfully used in development of complex software. We go even further and present some new paradigm:

“in-memory object graph as fully functional software”

1.2. Topic overview

In short: model driven user interface generation is a process where domain describing model is used as a base data for generating user interface assets.

Generation process can have many implementation variants. For example – generation may be parameterized by model itself or by other meta information. Another variant space is generation time - user interface generation may happen at design, compile or run time.

In this paper we explore multi user interface medium, metadata parameterized, run time user interface generation framework that allows to realize “in-memory object graph as fully functional software” vision by allowing user to interact with in-memory object graph by auto generated user interface.

1.3. State of the art

User interface generation concept is not new. It is present in academia for long time. And mass software often uses it. Microsoft Access [1] is good example of mass software user interface generating application. Those capabilities were present in Access in version 1.0 !

Some sort of user interface generation has been traditionally available in so called “4G languages” – used mainly as front end to vendor specific database system. Modern visual programming environments often have such capabilities.

What has to be stated is that in mass market software user interface generation is mainly design time process with reduced parameterization abilities and, what is more important it generally lack any object

interaction (method invocation) abilities. As such those popular solutions are not enough for realizing “in-memory object graph as fully functional software” vision.

1.4. Aim of this paper

As “in-memory object graph as fully functional software” vision is not very popular, both in industry and academia there is not a lot of its practical implementations. Existing one (Naked Objects [2]) has several limitations.

There is need for line-of-business feature set, model driven user interface generation framework with sound support for industry standard tools, languages and databases.

1.5. Solution

We developed some conceptual framework for rich feature set user interface generation system. This framework (code name ‘zero’) can be summarized by following properties:

- 100% no requirements for any additional meta attributes. Original, pure domain object model is enough to generate fully functional user interface.
- Object model independence (via internal, own, cross-domain Type Schema model) – ability to work with any object model.
- Being “object centric” – user interaction not only with data but with objects with means ability to invoke methods.
- Metadata provider independence.
- Target UI medium independence.
- Generation process full parameterization.
- Type Schema and Generators implementations plug-in system.
- Ability to use any target UI specific control.
- 100% run time work.
- Run time user customization abilities.

1.6. Languages, tools and environment used

As our goal was “*line-of-business feature set model driven user interface generation framework with sound support for industry standard tools, languages and databases*” we had to carefully chose appropriate tools.

We chose Microsoft .NET [14] as implementation platform. Microsoft .NET is mature software infrastructure with industry proven abilities. As target UI platforms we chose ASP.NET [6] for WWW sessions and Windows Presentation Foundation [7] for windows native client sessions.

All ‘zero’ software was developed in C#3.0 [8] using Visual Studio 2008 [9].

1.7. Results summary

We implemented “zero” conceptual framework in form of target UI medium independent generation framework. Two target UI generations infrastructures were implemented: ASP.NET based for WWW sessions and Windows Present ion Foundation based for windows native client sessions.

Because this paper’s domain is in some aspects novel, we will present short results summary here. In this paper we will use PowerPlant model that describes some abstract power plant. Full PowerPlant model is shown in Appendix A.

We will present part of example model (**Employee** C# class) and corresponding “zero” auto generated UI forms. In basic scenario in “zero” we use “.NET” classes object model (detected in runtime by reflection) as model. This is code-first approach.

```
public class Employee
{
    [VisualName(Name="Employee's Name")]
    [Description(Text="This is employees name")]
    public string Name {get; set;}

    public string Surname {get; set;}

    [TextMultiLine]
    public string Comments { get; set; }

    [ObjectSourceCollection(Name = "Positions")]
    public Position Position {get; set;}

    [ObjectSourceCollection(Name = "WorkDivisions")]
    public WorkDivision WorkDivision {get;set;}

    public int Salary {get; set;}

    public List<Device> ResponsibleForDevices {get;set;}

    public List<PhysicalAddress> PhysicalAddresses { get; set; }
    public List<EmailAdress> EmailAdresses { get; set; }
    public List<PhoneNumber> PhoneNumbers { get; set; }
```

```

public void SendMessage(string message){
}

public void IncreaseSalaryBy10Percent (){
    Salary += Salary / 10;
}

public void SetSalaryToPositionMaxSalary(){
    Salary = Position.MaxSalary;
}

public void SetSalaryToPositionMinSalary(){
    Salary = Position.MinSalary;
}

public void SetSalaryToWorkDivisionAvarage(){
    Salary = (int)PowerPlant._PowerPlant.Employees.Where(e =>
e.WorkDivision == this.WorkDivision).Average(e => e.Salary);
}

}

```

Employee C# class definition is compiled to binary form and is used in run time to UI generation. Some **Employee**'s class fields are decorated with attributes – those are metadata used to parameterize UI generation process. For example **Name** field has two such attributes: *VisualName* and *Description*:

```

[VisualName(Name="Employee's Name")]
[Description(Text="This is employees name")]
public string Name {get; set;}

```

Figure 1 shows Native Windows application medium auto generated UI in *Detail* view. *Detail* view shows only one object at a time.

Typically in line-of-business applications collections of objects are used (for example list of employees). “.NET” natively supports collections of objects by generic collection types. Figure 2 shows list of **Employee** objects in so called *Bulk* view, with presents many objects at one time.

TEST (1)

edit

|<<<

new

delete

update

>>>

>|

Employee's Name

Wiesiek

Surname

Bleniak

This is employees name

Comments

Position

Electro king

WorkDivision

Power management

Salary

525

ResponsibleForDevices

	Model	InspectionStatus	Status	SendsPowerTo
	IQO 25	Passed	Running	BXY 67...
	XTK 74	Not passed - minor failure	Failure	NWM 84...
	LHG 61	Needs check	Failure	BHM 60...
	DHQ 11	Needs check	Failure	SEH 52...
	DPQ 22	Needs check	Emergency alaram	TJL 63...
	NSP 17	Not passed - minor failure	Running	NAG 64...
	NAG 64	Needs check	Stopped	SEH 52...

+new

PhysicalAddresses

	Street	City	PostalCode	Country
	Naczelnika 4/149	Opole	58-137	Poland
	W szczerym polu 1/56	Zakopane	58-418	Poland

+new

EmailAddresses

	Adress
	name@host.net
	name@host.net

+new

PhoneNumbers

	Number	Name
	(10)106568	

+new

SendMessage

message

run

IncreaseSalaryBy10Percent

run

SetSalaryToPositionMaxSalary

run

SetSalaryToPositionMinSalary

run

SetSalaryToWorkDivisionAvarage

run

Figure 1. Native Windows application medium auto generated UI. *Detail* view.

Search	Red X	Name	Surname	Job Title	Department	Number	Code
		Olek	Kurski	Senior Manager	Emergency	406	PVX 53...
		Olek	Dąbrowski	Plant Supervisor	Testing	426	UVX 86...
		Wiesiek	Leń	Junior Engineer	Service	308	PVX 53...
		Olek	Kurski	Electric Master	Service	666	FGT 80...
		Tadek	Maliniak	Junior Manager	Inspection	360	AUF 70...
		Witek	Dąbrowski	Senior Engineer	Inspection	581	DPQ 22...
		Tadek	Dąbrowski	Electrician	Emergency	665	CLM 25...
		Tadek	Leń	Engineer	Inspection	762	WJJ 74...
		Franek	Wirkus	Senior Engineer	Emergency	847	BYV 17
		Olek	Kurski	Senior Engineer	Service	697	KXB 14...
		Jurek	Dąbrowski	Senior Manager	Inspection	568	IES 57...
		Franek	Dąbrowski	Electro king	Power management	596	TMM 05...
		Franek	Leń	Plant Supervisor	Service	580	AUF 70...
		Franek	Bieniak	Junior Engineer	Emergency	587	FGE 78...
		Wiesiek	Cygan	Engineer	Testing	514	MAV 12...
		Tadek	Dąbrowski	Electrician	Power management	580	DDM 14...
		Witek	Leń	Plant Supervisor	Power management	363	ACP 76...
		Wiesiek	Bieniak	Electro king	Power management	525	IQO 25...
		Franek	Malinowski	Electrician	Service	714	AXE 56...
		Juzek	Leń	Senior Engineer	Service	452	OMF 20...
		Tadek	Jakubowski	Electrician	Testing	647	RDU 48...
		Franek	Dąbrowski	Electric Master	Testing	908	PCN 28...
		Franek	Cygan	Senior Engineer	Testing	570	FBC 17...
		Tadek	Jakubowski	Senior Engineer	Inspection	875	PVL 20...
		Tadek	Kurski	Plant Supervisor	Power management	527	GRB 51...
		Franek	Maliniak	Engineer	Service	696	IDC 87...
		Franek	Jakubowski	Electrician	Inspection	594	LLT 86...
		Jurek	Malinowski	Engineer	Testing	700	UVX 86...
		Franek	Bieniak	Engineer	Service	612	DDC 28...
		Jurek	Cygan	Senior Engineer	Emergency	907	CBG 24...
		Tadek	Dąbrowski	Junior Engineer	Service	317	LXP 10...
		Witek	Bieniak	Junior Engineer	Inspection	394	JJF 50...
		Wiesiek	Cygan	Junior Manager	Testing	761	APK 02...
		Jurek	Leń	Junior Engineer	Emergency	598	MXU 67...
		Wiesiek	Dąbrowski	Electro king	Emergency	478	LFR 53...
		Jurek	Bieniak	Junior Engineer	Base services	463	NPX 42...
		Witek	Leń	Senior Manager	Testing	477	YYY 41...
		Witek	Cygan	Engineer	Emergency	509	CLM 25...

Figure 2. Native Windows application medium auto generated UI. *Bulk* view.

As “zero” supports ASP.NET as UI generation medium, we will show similar figures for WWW based scenario.

[\[Employees\]](#)
[\[WorkDivisions\]](#)
[\[DeviceBrands\]](#)
[\[DeviceInspectionStatuses\]](#)
[\[DeviceModels\]](#)
[\[DeviceStatuses\]](#)
[\[Positions\]](#)
[\[StateReportValueClasses\]](#)

[|<<](#)
[<](#)
[Zapisz zmiany](#)
[Usuń ten obiekt](#)
[Stwórz nowy obiekt](#)
[>](#)
[>>|](#)

SendMessage	message wykonaj
IncreaseSalaryBy10Percent	wykonaj
SetSalaryToPositionMaxSalary	wykonaj
SetSalaryToPositionMinSalary	wykonaj
SetSalaryToWorkDivisionAvarage	wykonaj
Employee's Name	Wiesiek
Surname	This is employees name Bieniak
Comments	
Position	Electro king nawiguj
WorkDivision	Power menagement nawiguj
Salary	525
ResponsibleForDevices	IQQ 25 ✕ XTK 74 ✕ LHG 61 ✕ DHQ 11 ✕ DPQ 22 ✕ NSP 17 ✕ NAG 64 ✕ dodaj nowy
PhysicalAddresses	Naczelnika 4/149 58-137 Opole Poland ✕ W szczyrym polu 1/56 58-418 Zakopane Poland ✕ dodaj nowy
EmailAddresses	name@host.net ✕ name@host.net ✕ dodaj nowy
PhoneNumbers	(10)106568 ✕ dodaj nowy

Figure 3. ASP.NET application medium auto generated UI. *Detail* view.

	[Devices]	[Employees]	[WorkDivisions]	[DeviceBrands]	[DeviceInspectionStatuses]	[DeviceModels]	[DeviceStatuses]	[Positions]	[StateReportValueClasses]		
Lp	Employee's Name	Surname	Comments	Position	WorkDivision	Salary	ResponsibleForDevices	PhysicalAddresses	EmailAddresses	PhoneNumbers	
1	Wiesiek	Cygan		Electrician	Service	837	CHP 44...	Wielkowiejska 5/86 22-551 Zakopane Poland...	name@host.net...	(83)387731...	pokaż  
2	Tadek	Leń		Junior Menager	Power managment	842	NIA 26...	W szczerym polu 1/93 57-366 Tłusz Poland...	name@host.net...	503541660...	pokaż  
3	Olek	Kurski		Senior Engineer	Emergency	828	WRM 58...	Gorzka 6/14 87-776 Zakopane Poland	name@host.net...	500517332...	pokaż  
4	Wiesiek	Leń		Engineer	Service	629	FDS 15...	W szczerym polu 6/98 37-578 Opole Poland...	name@host.net...	(27)011368...	pokaż  
5	Olek	Kurski		Senior Menager	Emergency	406	PVX 53...	Naczelnika 1/67 51-115 Opole Poland	name@host.net...	(26)812570...	pokaż  
6	Olek	Dąbrowski		Plant Supervisor	Testing	426	UVX 86...	Leśnej strugi 1/42 84-348 Warszawa Poland...	name@host.net	(26)148661...	pokaż  
7	Wiesiek	Leń		Junior Engineer	Service	308	PVX 53...	Bombardierów 5/88 75-625 Warszawa Poland	name@host.net...	503087685...	pokaż  
8	Olek	Kurski		Electric Master	Service	666	FGT 80...	Bombardierów 7/72 48-617 Warszawa Poland...	name@host.net...	(52)034680...	pokaż  
9	Tadek	Maliniak		Junior Menager	Inspection	599	AUF 70...	Leśnej strugi 6/84 53-535 Tłusz Poland...	name@host.net...	500054212...	pokaż  
10	Witek	Dąbrowski		Senior Engineer	Inspection	581	DPQ 22...	Leśnej strugi 8/59 75-572 Karpacz Poland...	name@host.net...	(65)328575...	pokaż  
11	Tadek	Dąbrowski		Electrician	Emergency	665	CLM 25...	Warszawska 4/93 14-253 Karpacz Poland...	name@host.net...	(52)807321...	pokaż  
12	Tadek	Leń		Engineer	Inspection	762	WJJ 74...	W szczerym polu 5/39 13-752 Karpacz Poland...	name@host.net	(48)882364...	pokaż  
13	Franek	Wirkus		Senior Engineer	Emergency	847	BYV 17	Warszawska 5/46 14-565 Warszawa Poland...	name@host.net	(88)066311...	pokaż  
14	Olek	Kurski		Senior Engineer	Service	697	KXB 14...	W szczerym polu 2/58 66-846 Zakopane Poland	name@host.net...	505050880	pokaż  
15	Jurek	Dąbrowski		Senior Menager	Inspection	568	IES 57...	Zołądkowa 4/121 55-436 Opole Poland...	name@host.net...	505545526...	pokaż  
16	Franek	Dąbrowski		Electro king	Power managment	596	TMM 05...	Gorzka 8/3 28-781 Warszawa Poland...	name@host.net...	(42)042758...	pokaż  
17	Franek	Leń		Plant Supervisor	Service	580	AUF 70...	Gorzka 6/112 68-187 Płońsk Poland...	name@host.net...	506780381	pokaż  
18	Franek	Bieniak		Junior Engineer	Emergency	587	FGE 78...	Leśnej strugi 4/45 14-425 Tłusz Poland...	name@host.net	(54)885420...	pokaż  
19	Wiesiek	Cygan		Engineer	Testing	751	MAV 12...	W szczerym polu 1/50 55-462 Opole Poland	name@host.net...	(17)748110	pokaż

Figure 4. ASP.NET application medium auto generated UI. *Bulk* view.

1.8. Content of this paper

This paper is divided by 6 chapters. Those are as follows:

- Chapter 1 - Introduction.
- Chapter 2 – Problem overview and state of the art presentation.
- Chapter 3 – Tools used.
- Chapter 4 – The solution: ‘zero’ conceptual framework.
- Chapter 5 – The implementation: ASP.NET and Windows Presentation Foundation target UI generation sub systems.
- Chapter 6 - Solution applications, limitations and future work.
- Appendix A – PowerPlant model.

2. The problem: User interface generation

2.1. Model-driven architecture

To talk about *model-driven architecture* we have to define *model* first. We will use Wikipedia's definition [13]:

A model in science is a physical, mathematical, or logical representation of a system of entities, phenomena, or processes. Basically a model is a simplified abstract view of the complex reality. It may focus on particular views, enforcing the "divide and conquer" principle for a compound problem. Formally a model is a formalized interpretation which deals with empirical entities, phenomena, and physical processes in a mathematical, or logical way.

From software engineering perspective models have one important property – in most cases models are expressed in declarative way. Declarative expressing means that one expresses problem in categories of *what* instead of *how*.

Model-driven architecture is base concept for this paper. Let's start with definition taken from Wikipedia [10]:

Model-driven architecture (MDA) is a software design approach for the development of software systems. It provides a set of guidelines for the structuring of specifications, which are expressed as models. Model-driven architecture is a kind of domain engineering, and supports model-driven engineering of software systems. It was launched by the Object Management Group (OMG) in 2001.

Above definition refers to Object Management Group. OMG is standard body made from consortium of more than 800 industry companies. OMG has developed some standards aimed at MDA but we will not refer to them in direct way.

For our needs we propose another definition:

Model-driven architecture is software engineering approach where system model is base asset for development processes.

As software engineering methods could be quite different our definition underlines *model as a base asset* with is core concept for MDA. This approach is show on figure 5.

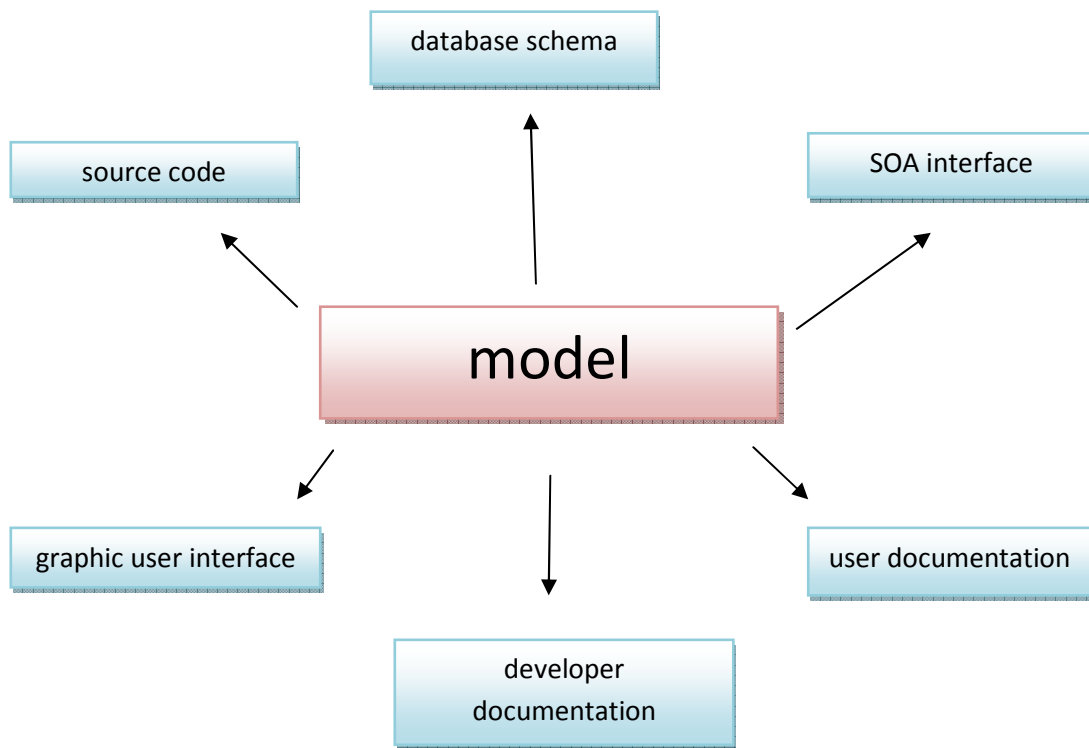


Figure 5. Model-driven architecture

A little comment to figure 5 is needed. One might say that “user documentation” and especially “developer documentation” is not software *per se*, but we have to disagree. Those assets are immanent part of real-life computer software and their development could be model based as well.

Processes shown as arrows on figure 5 may be automated or human based. In both cases it is MDA. What has to be stated is:

If one wants to automate development processes using MDA, model has to be annotated with dedicated, additional meta data.

Practice shows that 100% automated real-life software generation is nothing more than academic utopia. It does not mean that automated software asset generation as such is bad idea. We just have to forget that it will solve in 100% all domain specific software engineering problems. In practice often not all software aspects could be 100% described by domain model. Dedicated domain languages – so called domain specific languages - DSL (see [11]) comes to play here. DSL define domain specific syntax and semantics. Domain specific syntax allows one to express domain related concepts without redundancy, but semantic aspect of DSLs is most important – it brings domain specific mind set to some concrete syntax. DSL in most cases allows to describe domain problem in more details. But in real-life scenarios DSLs always have some limitations in domain describing abilities. Also we have to remember that besides domain related features software have implementation/deployment specific features. Of course there are implementation/deployment DSLs but those have their limitations too.

In this paper we are interested only in object models. We do not deal with any processes models or any DSL expressed models.

2.2. Generating UI from model

UI generation from the model is this papers late motive. As this concept is so important we will illustrate it on dedicated figure.

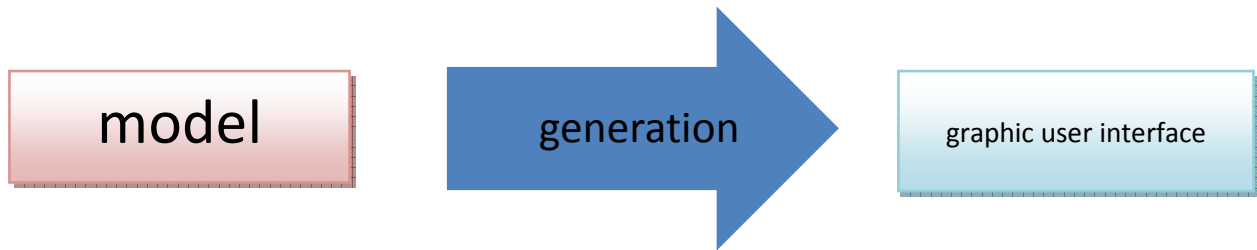


Figure 6. UI generation from the model

In process shown on figure 6, model is typically expressed as some data or object structure. In this paper models used for UI generation are always object models.

2.3. State of the art

Graphic interface elements model based automatic generation is widely know topic. We present four different solution implementations: Microsoft Access [1], NakedObjects [2], DynamicData [3] and XAML Tools [4].

2.3.1. Microsoft Access

Microsoft Access [1] is compact database system aimed at small, user created data bases. Our interests in this paper concentrate at Access form generation tools.

Form generation use case is as follows:

- User designs database table (see figure 7).
- User runs form generation wizard.

- User parameterizes generation process thru interaction with wizard.
- Generated form (see figure 8) is added to Access's database assets.

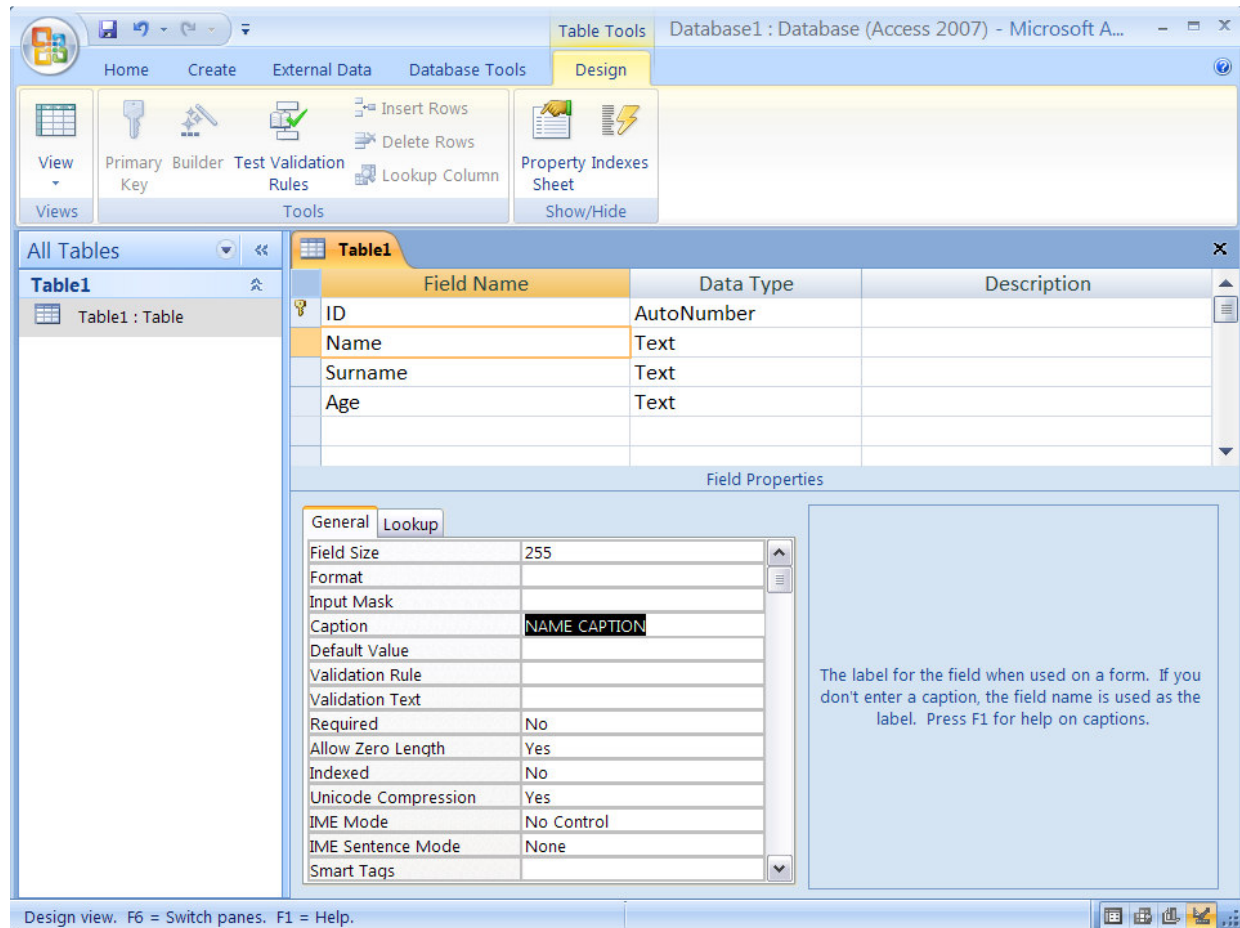


Figure 7. Access database table design

By database table design user can specify text formatting, validation rules and label used for each table's field.

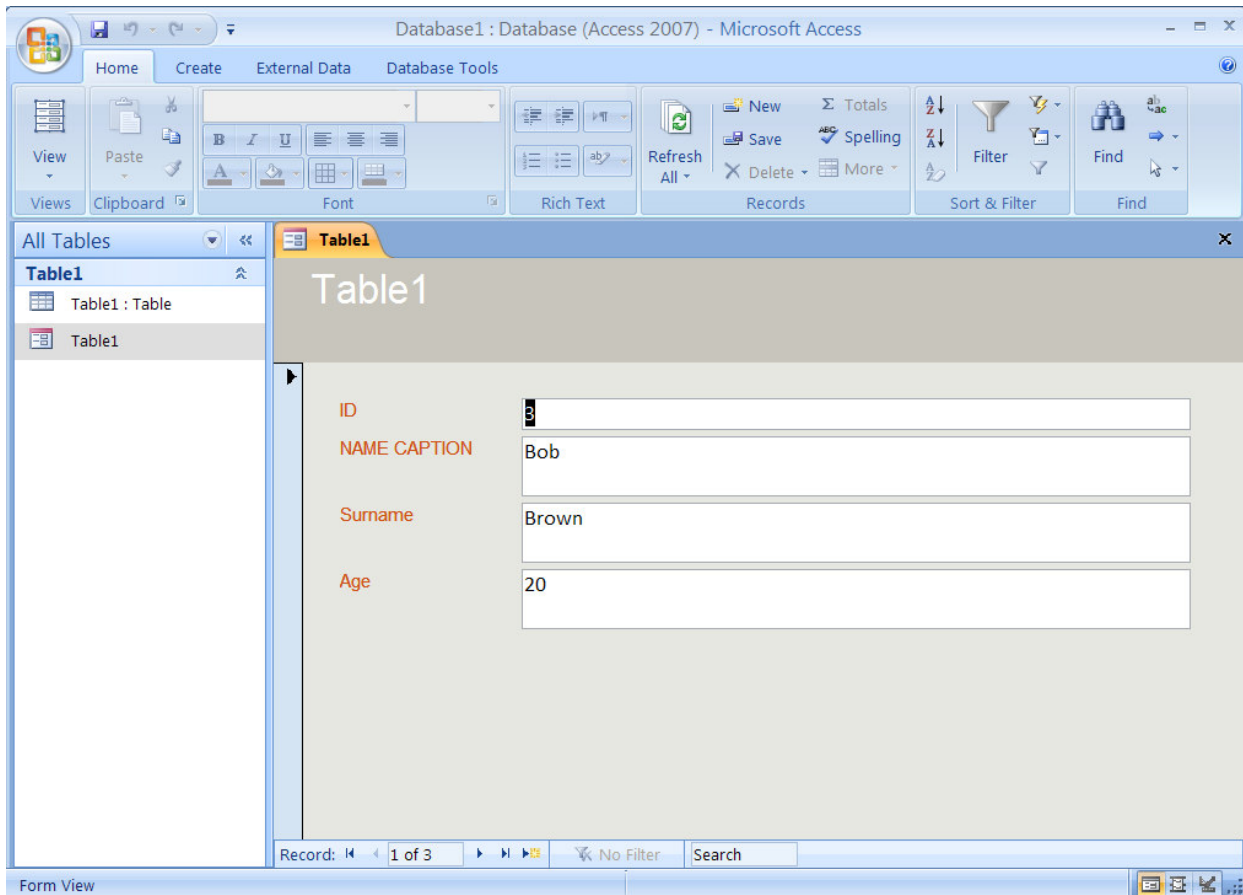


Figure 8. Access generated form

2.3.2. NakedObjects

NakedObjects [2] is solution most similar to ours. Nowadays it is mature, 3.0 version open source product with successful realizes vision similar to ours.

In NakedObjects, Java object model is base asset for run time user interface generation. Currently NakedObjects supports two UI targets: HTML WWW based and native (java application) one. Standard Java language attributes, that decorates classes and fields definitions are used as meta data for UI customization.

Most important feature of NakedObjects is giving user ability to invoke methods on domain objects. Method invocation is very important aspect of user interaction with object graph and if omitted makes whole idea of automated graphic interface generation not suited for true object oriented systems. We will return to this topic in chapter 4.

As NakedObject is only one public know solution similar to our, we will present few screen shots. By this it will be easier to compare those solutions.



Figure 9. NakedObjects native client demo application start screen

NakedObjects native client start screen (figure 9) is quite ascetic one. User have to manually examine icons with right mouse click to proceed further.

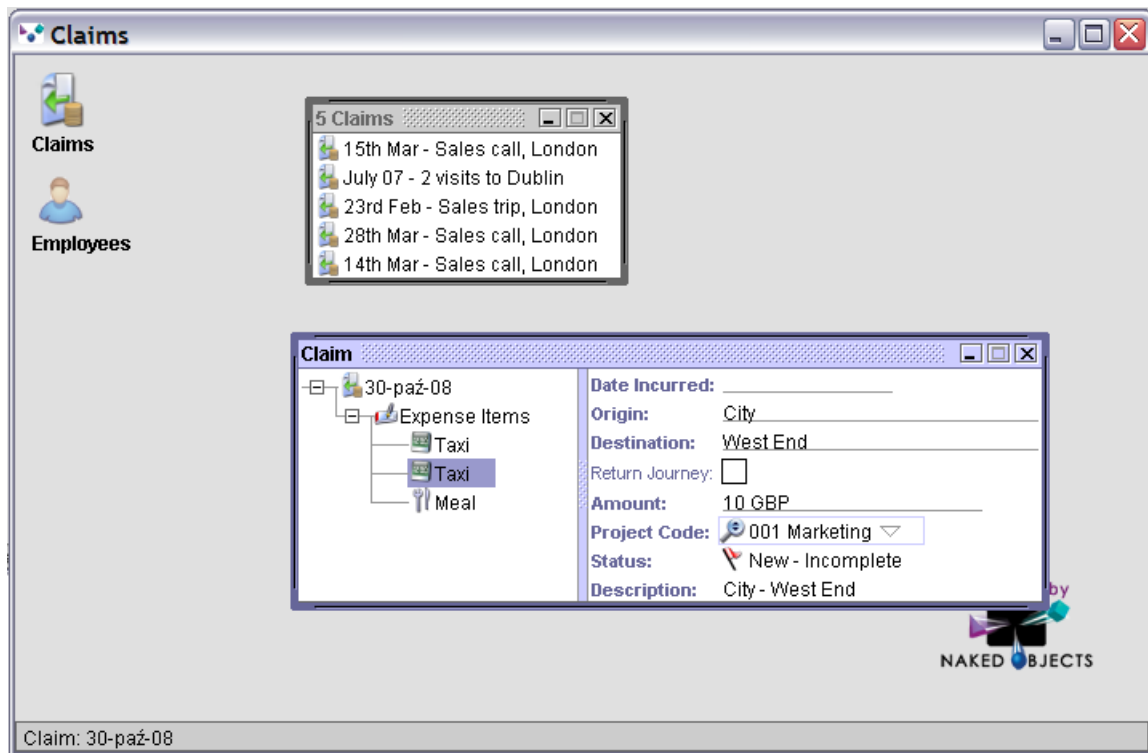


Figure 10. NakedObjects native client demo application sample view

Two basic NakedObject object views are shown in figure 10. In upper part of screenshot we have *Bulk* view with presents a collection of objects. In lower part of screenshot we have *Detail* view with presents particular object. As **Claim** object have sub objects (two **Taxi** ones, and one **Meal**) their relation can be show in treeview (left part of **Claim** window).

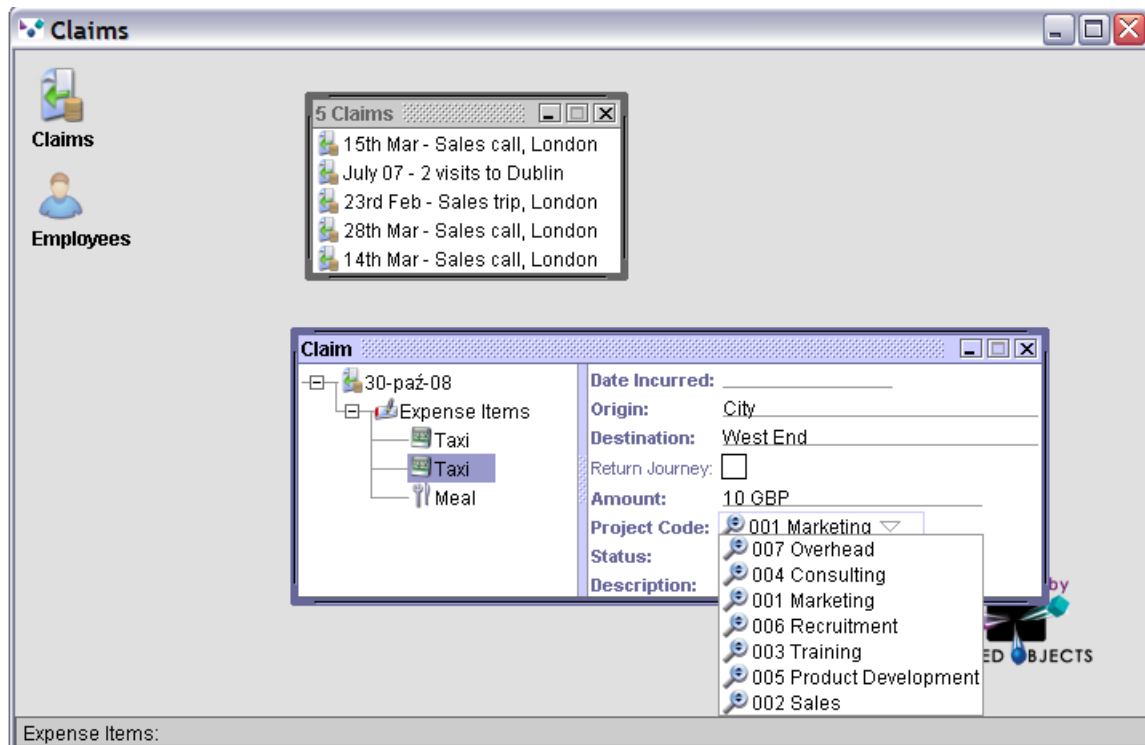


Figure 11. NakedObjects native client demo application 1-n relation UI support

1-n relation are quite common in object models. When auto generating UI those have to be supported with dedicated mechanism that examines “n” relation side objects and present them as some form of graphic list. Such situation is shown on figure 11 – **Project Code** list is presented to user in dropdown box.

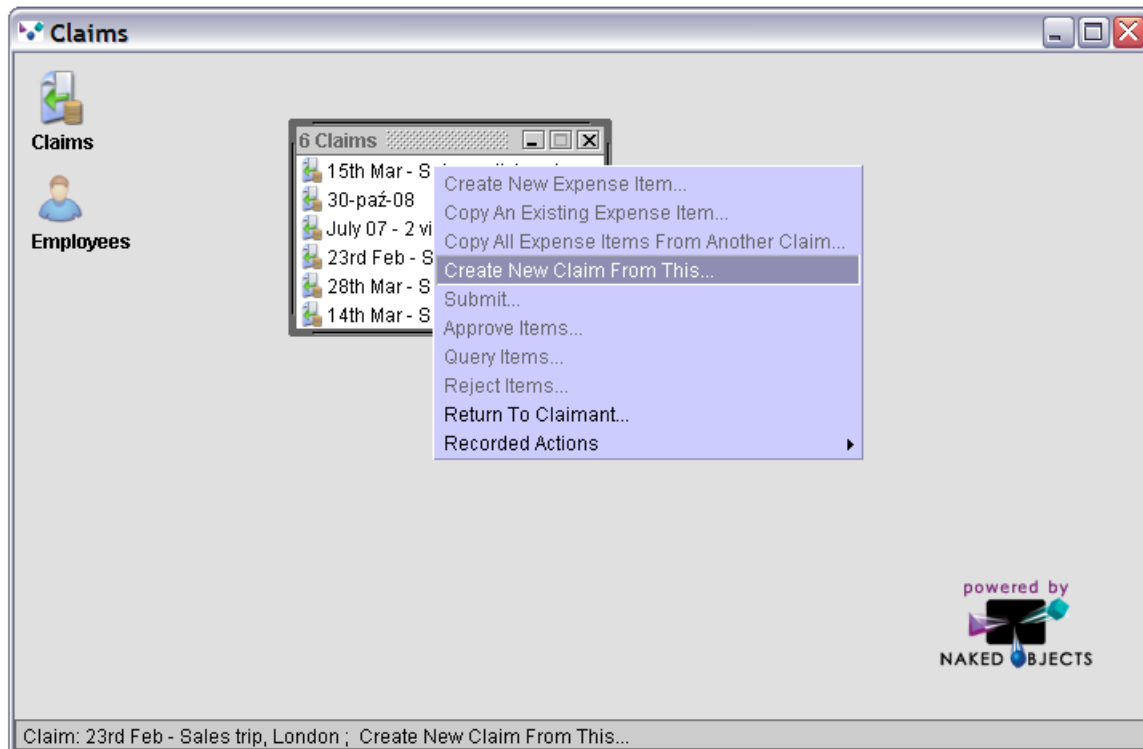


Figure 12. NakedObjects native client demo application method invocation UI support

Figure 12 shows list of methods that can be invoked over selected object (violet window). Original source code method names have been replaced with end-user readable ones.

When method invocation needs parameterization, dedicated sub-window is shown to the user.

Below we present WWW HTML client screenshots. Figures below are corresponding ones to those above.

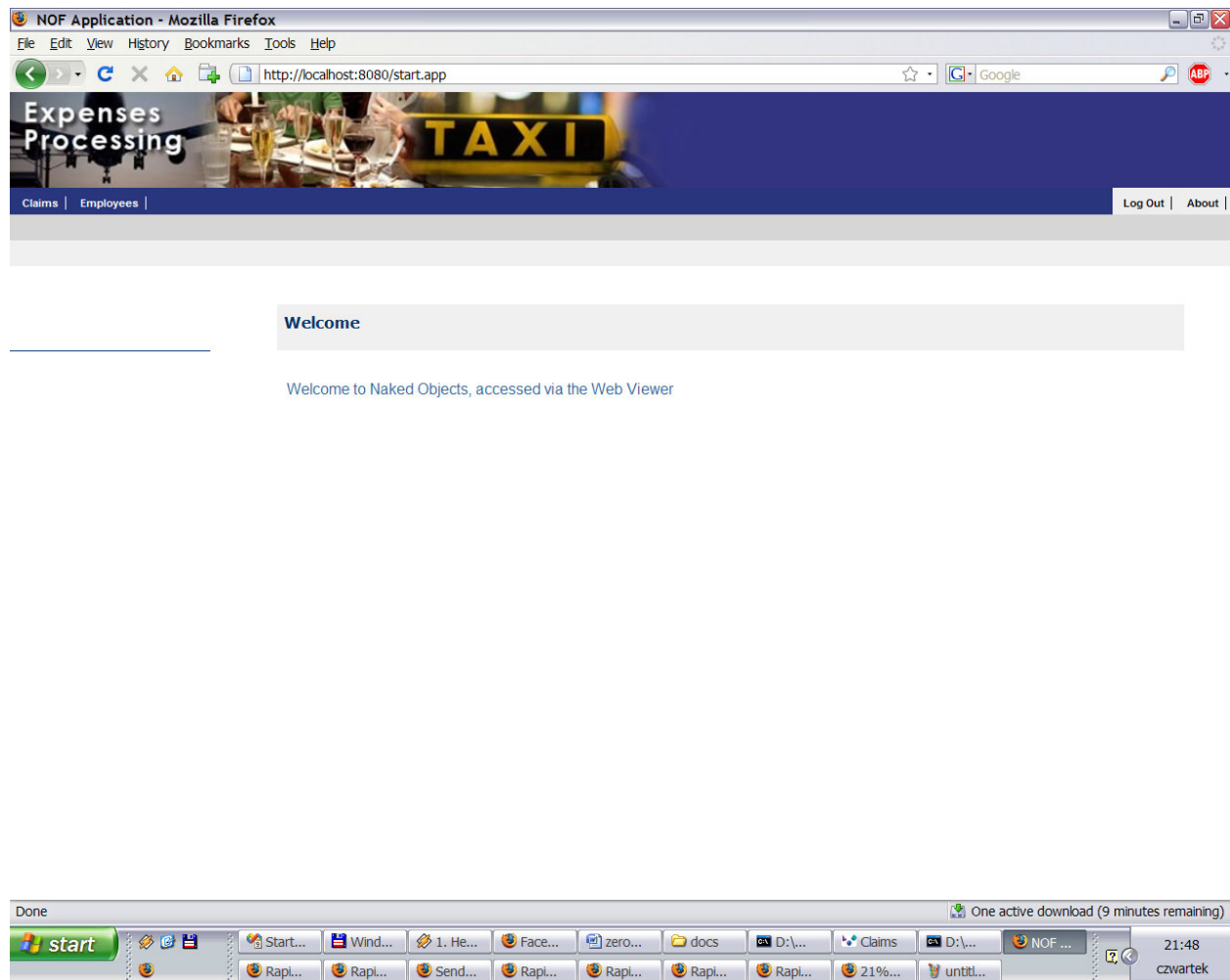


Figure 13. NakedObject web client demo application start screen

5 Claims - Mozilla Firefox

File Edit View History Bookmarks Tools Help

http://localhost:8080/method.app?id=1&action=74

Expenses Processing TAXI

Claims Employees Log Out About

14th Mar - Sales call, London 5 Claims 001 Marketing 15th Mar - Sales call, London

15th Mar - Sales call, London > Edit > 14th Mar - Sales call, London > Edit > Claims > Find My Claims > 5 Claims

	Description	Date Created	Status	Claimant	Approver	Project Code	Total
15th Mar - Sales call, London	15th Mar - Sales call, London	2007-04-03	Submitted For Approval	Sven Bloggs	Dick Barton	001 Marketing	68 GBP
July 07 - 2 visits to Dublin	July 07 - 2 visits to Dublin	2007-07-15	Submitted For Approval	Sven Bloggs	Dick Barton	002 Sales	758 GBP
23rd Feb - Sales trip, London	23rd Feb - Sales trip, London	2007-04-03	Returned To Claimant	Sven Bloggs	Dick Barton	001 Marketing	28 GBP
28th Mar - Sales call, London	28th Mar - Sales call, London	2007-04-03	New	Sven Bloggs	Dick Barton	001 Marketing	51,4 GBP
14th Mar - Sales call, London	14th Mar - Sales call, London	2007-04-03	New	Sven Bloggs		001 Marketing	68 GBP

Done

One active download (8 minutes remaining)

start Start... Wind... 1. He... Face... zero... docs D:\... Claims D:\... 5 Clai... 21:51 czwartek

Rapl... Rapl... Send... Rapl... Rapl... Rapl... Rapl... 39%... untill...

Figure 14. NakedObject web client demo application sample view

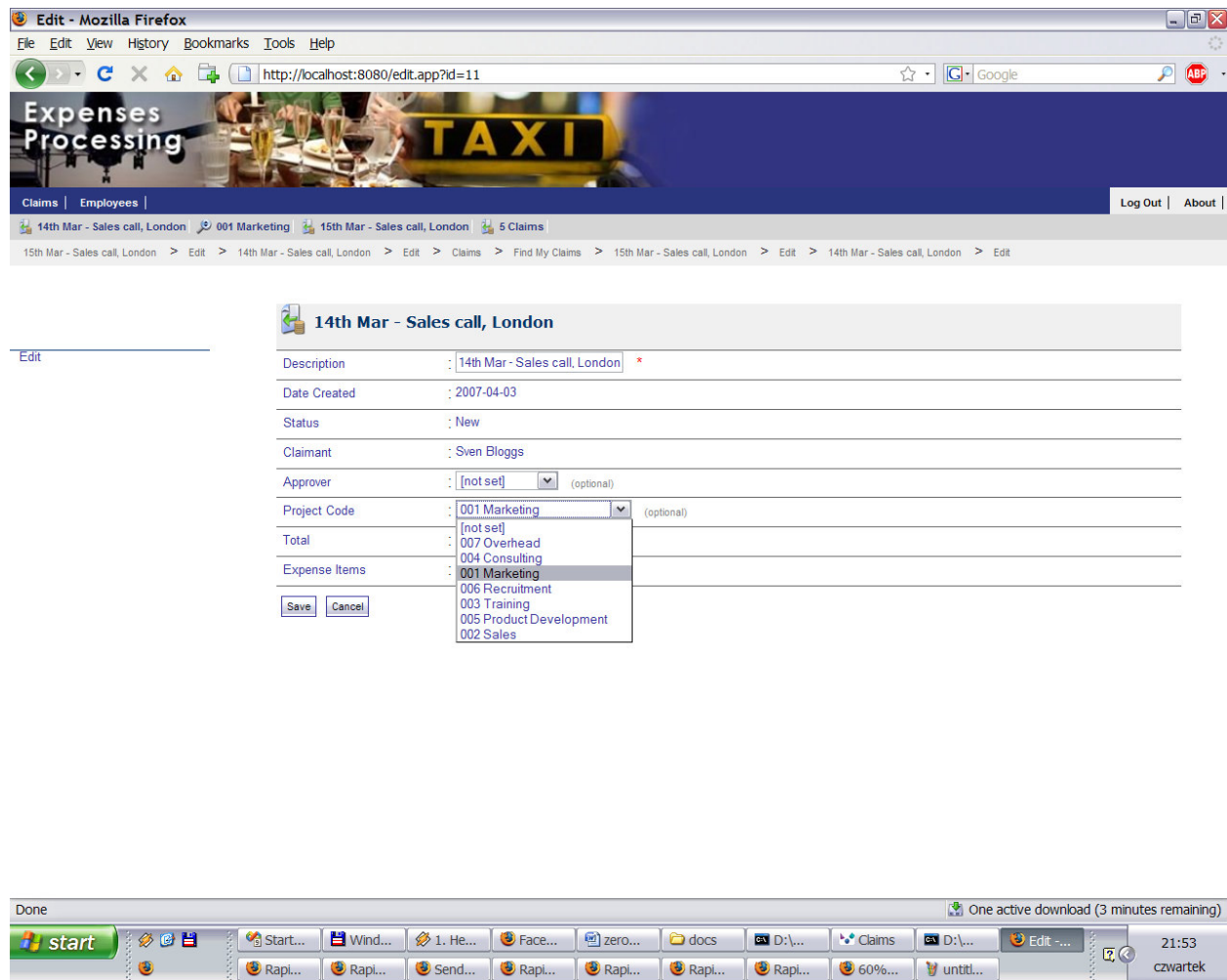


Figure 15. NakedObject web client demo application 1-n relation UI support

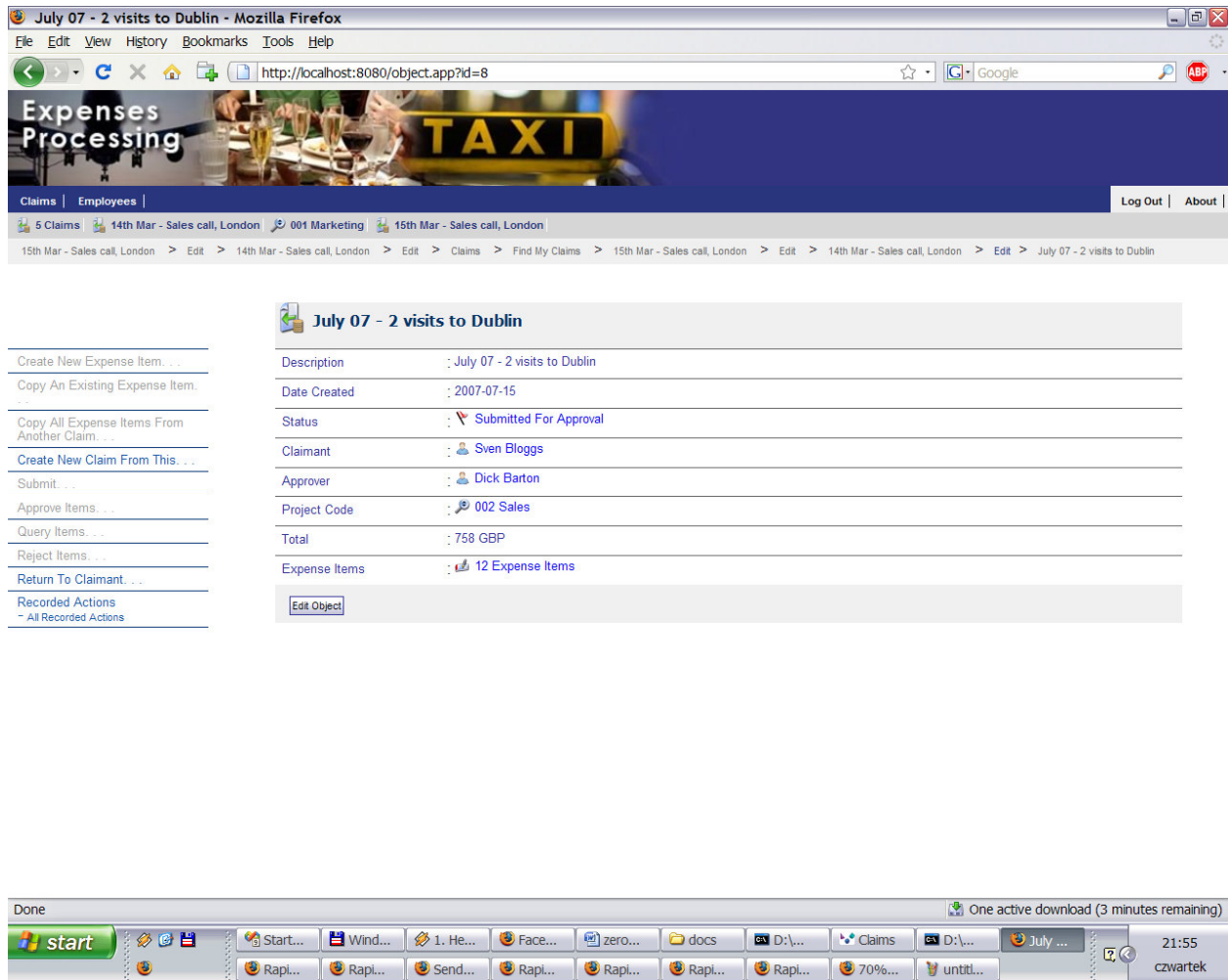


Figure 16. NakedObject web client demo application method invocation UI support

NakedObjects are described in more detail in its author PhD thesis [12].

2.3.3. DynamicData

DynamicData [3] is Microsoft's web UI generation framework that became part of .NET standard library in .NET 3.5 SP1. Same as in NakedObjects, in DynamicData:

- “.NET” platform object data model is used as base asset for UI generation.
- “.NET” language attributes are used for UI generation customization.

DynamicData is production ready framework with reach customization abilities. It is quite similar to NakedObjects/“zero” but there is basic difference – DynamicData allows user to interact only with relational data and not with objects – method invocation is not possible. To better illustrate DynamicData features we will show some basic screenshots.

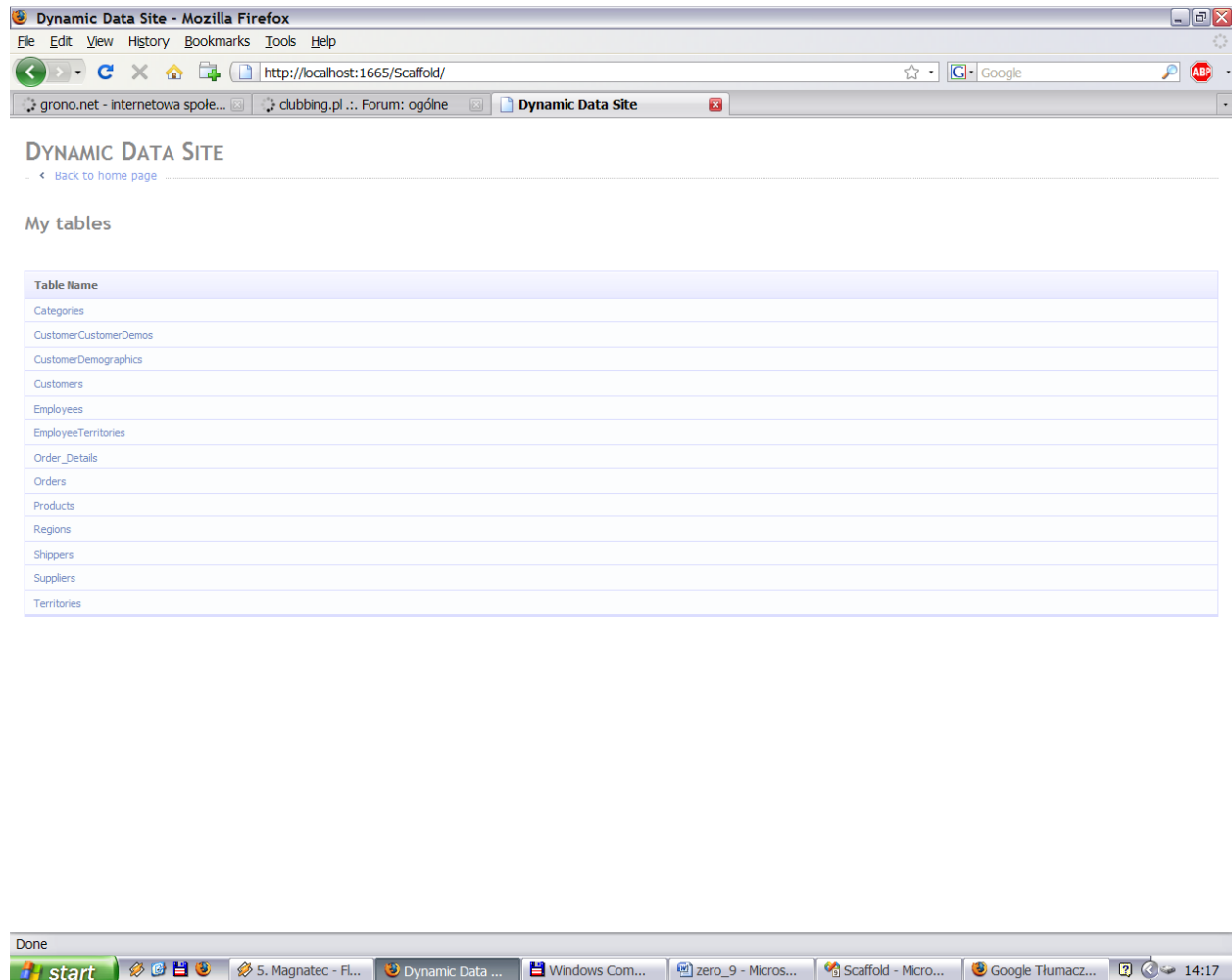


Figure 17. Sample DynamicData site start screen

Figure 17 shows sample DynamicData site start screen. List of all classes (tables) in system is shown to the user.

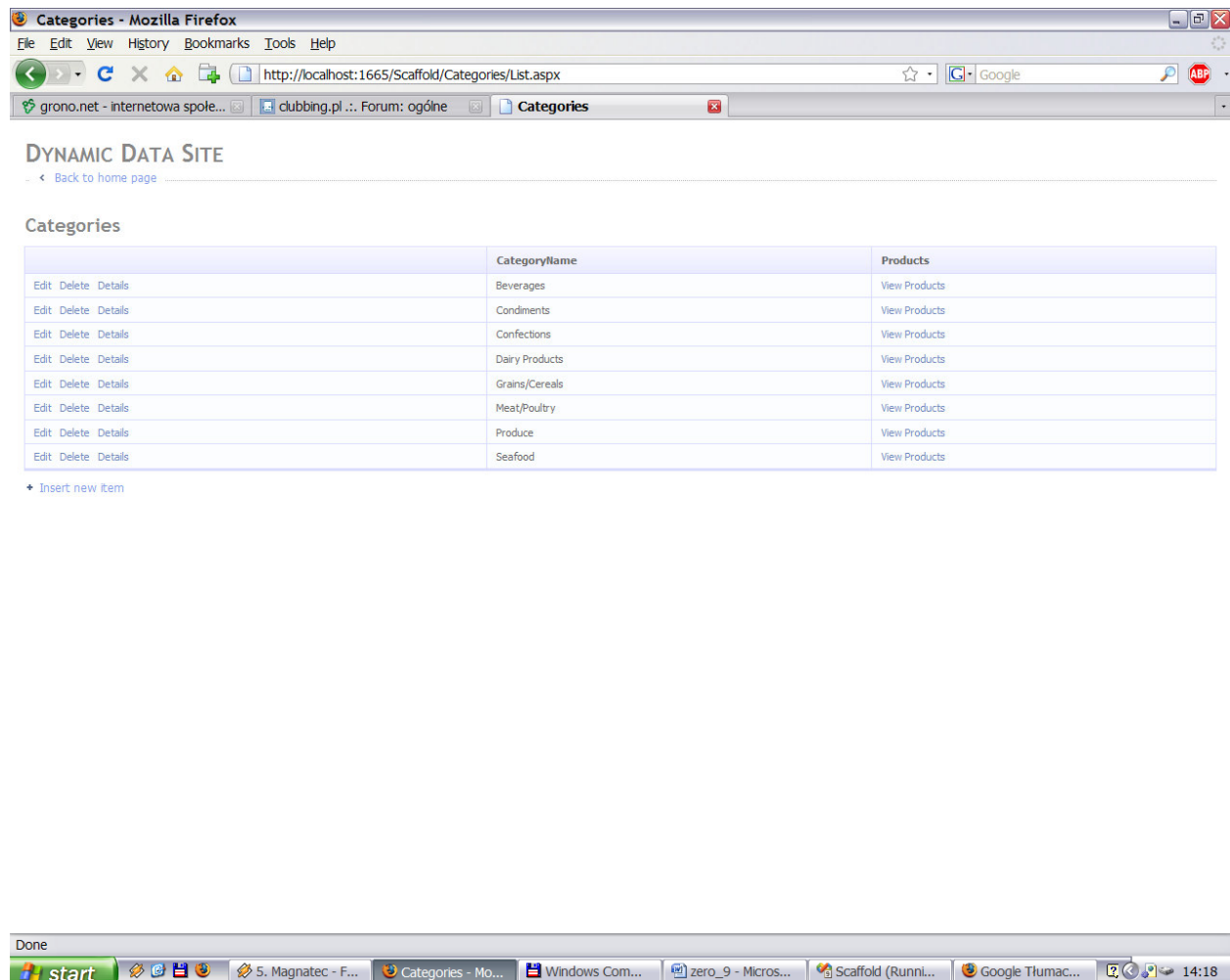


Figure 18. Sample DynamicData site Categories list

Figure 18 shows sample DynamicData site **Categories** list. For each **Category** there are “Edit”, “Delete” and “Details” action links. What is interesting, DynamicData knows about **Category** <> **Product** 1-n relationship and uses this knowledge while generating above form – for each Category “View Products” action link is presented.

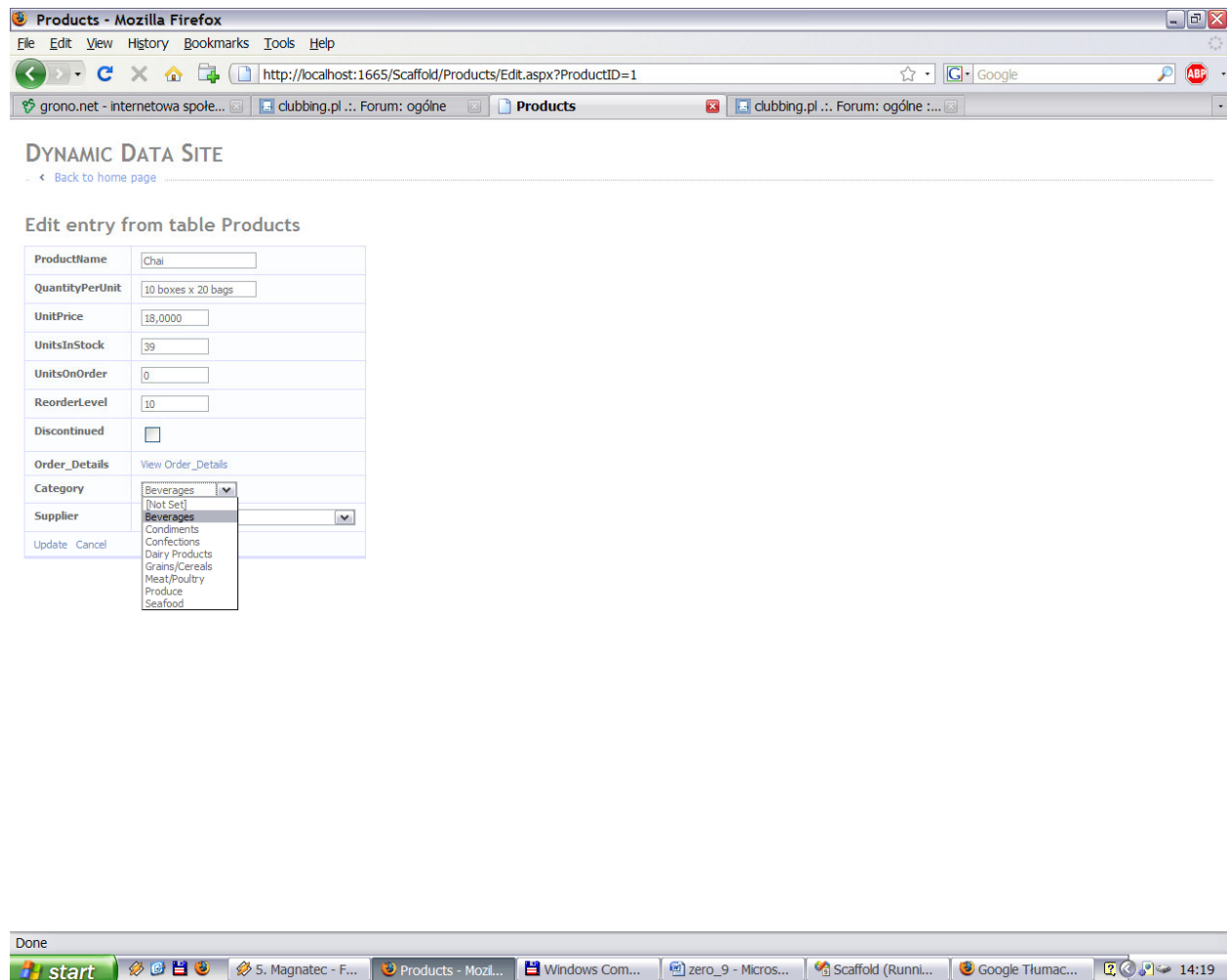


Figure 19. Sample DynamicData site Product detail view

Figure 19 shows **Product Detail** view. **Category** <> **Product** 1-n relationship is known to DynamicData. That's why **Category** field is presented as a dropdown populated with **Categories**.

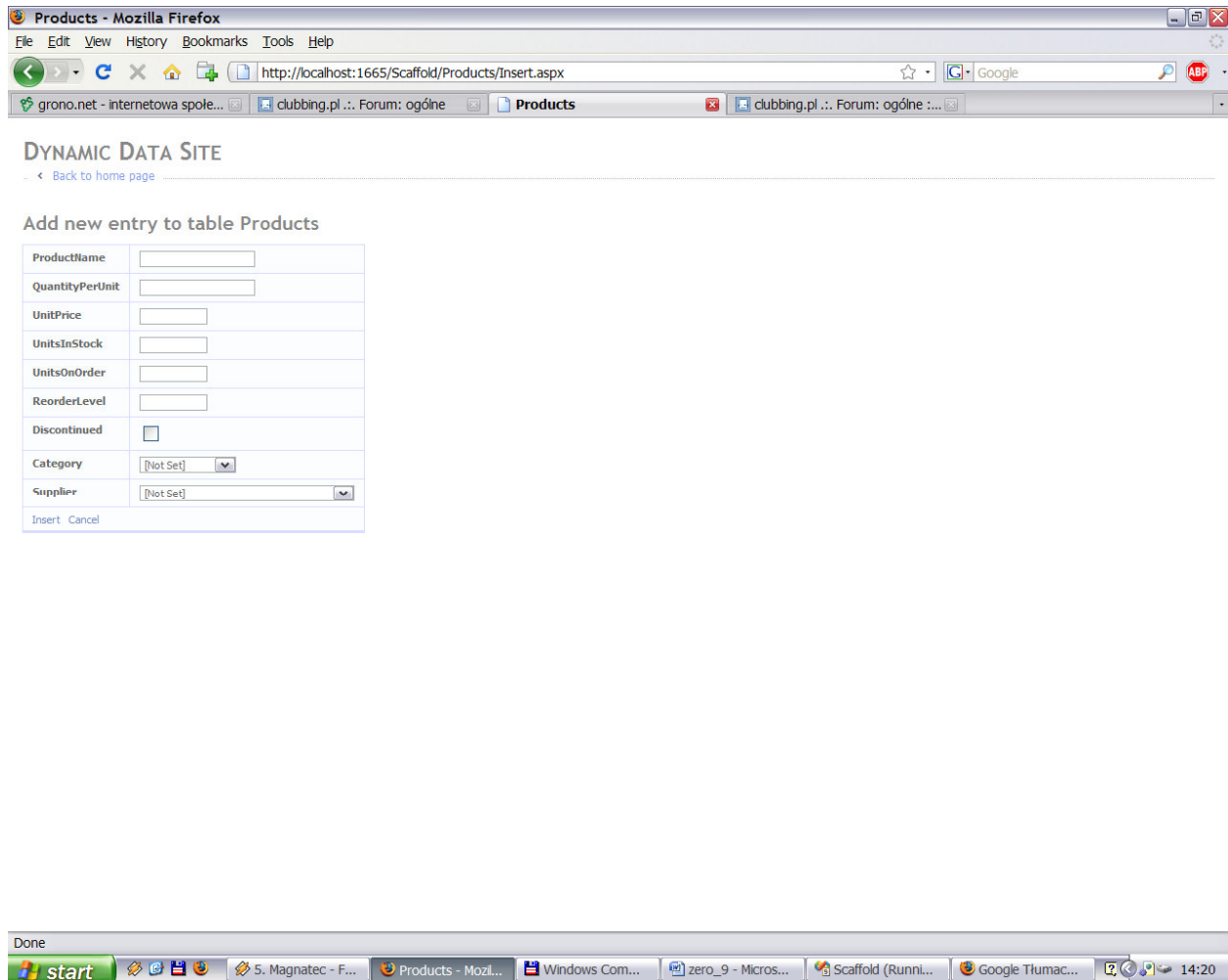


Figure 20. Sample DynamicData site new Product item form

2.3.4. XAML Power Toys

XAML Power Toys [4] is suite of programming environment (Visual Studio) extensions and add-in's. Among many toys there are two user interface generators. Those are list and grid control generators.

Code generation use case is as follows:

- User selects particular class for generation process (see figure 21).
- Generic code generation parameterization form is presented. User may customizes code generation process here (see figure 22).

- XAML code is generated to clipboard.
- User pastes generated code from clipboard to source code window.

For each field of selected class user can customize following properties:

- Control used for user interaction with field.
- Label.
- Text formatting.
- Max text length and control width.
- Data binding mode.

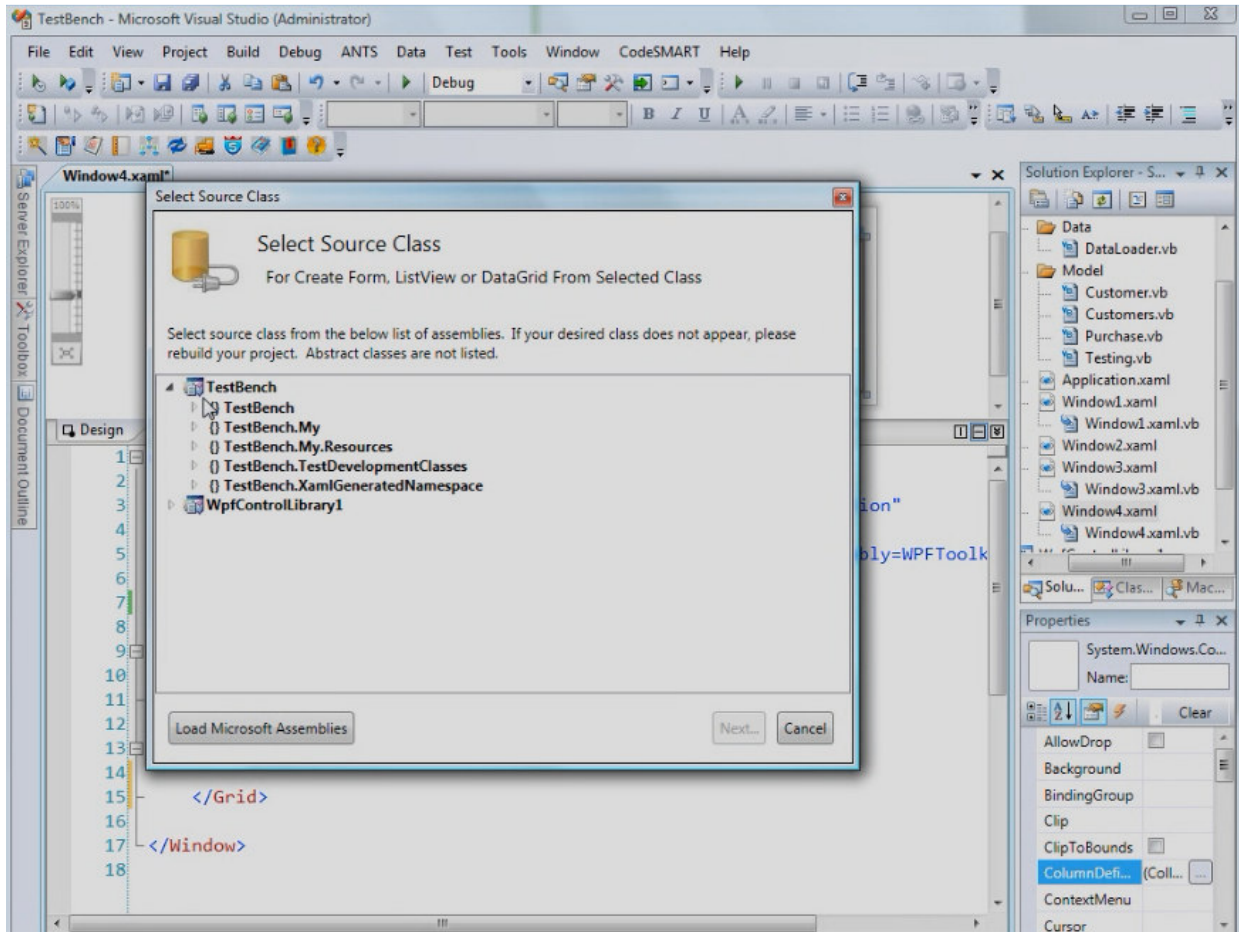


Figure 21. XAML Toys source class selection dialog

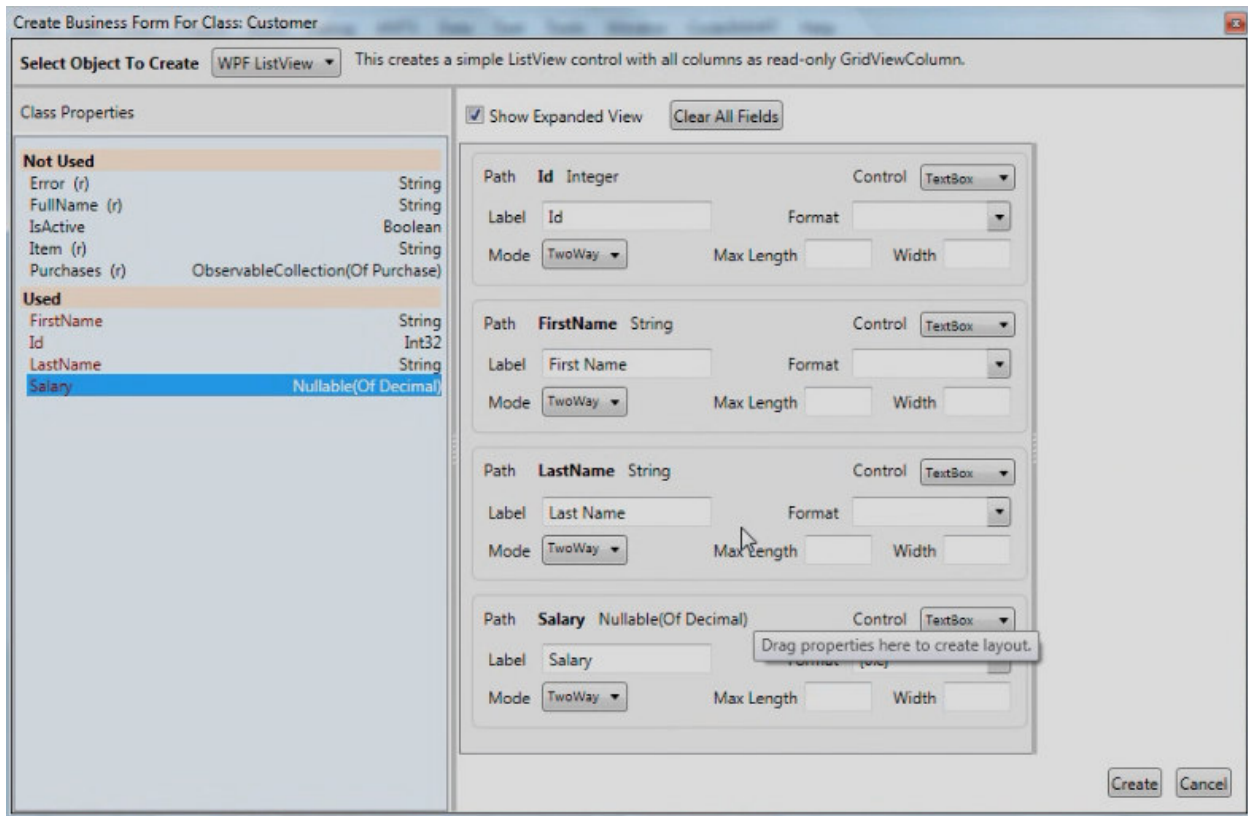


Figure 22. XAML Toys code generation parameterization

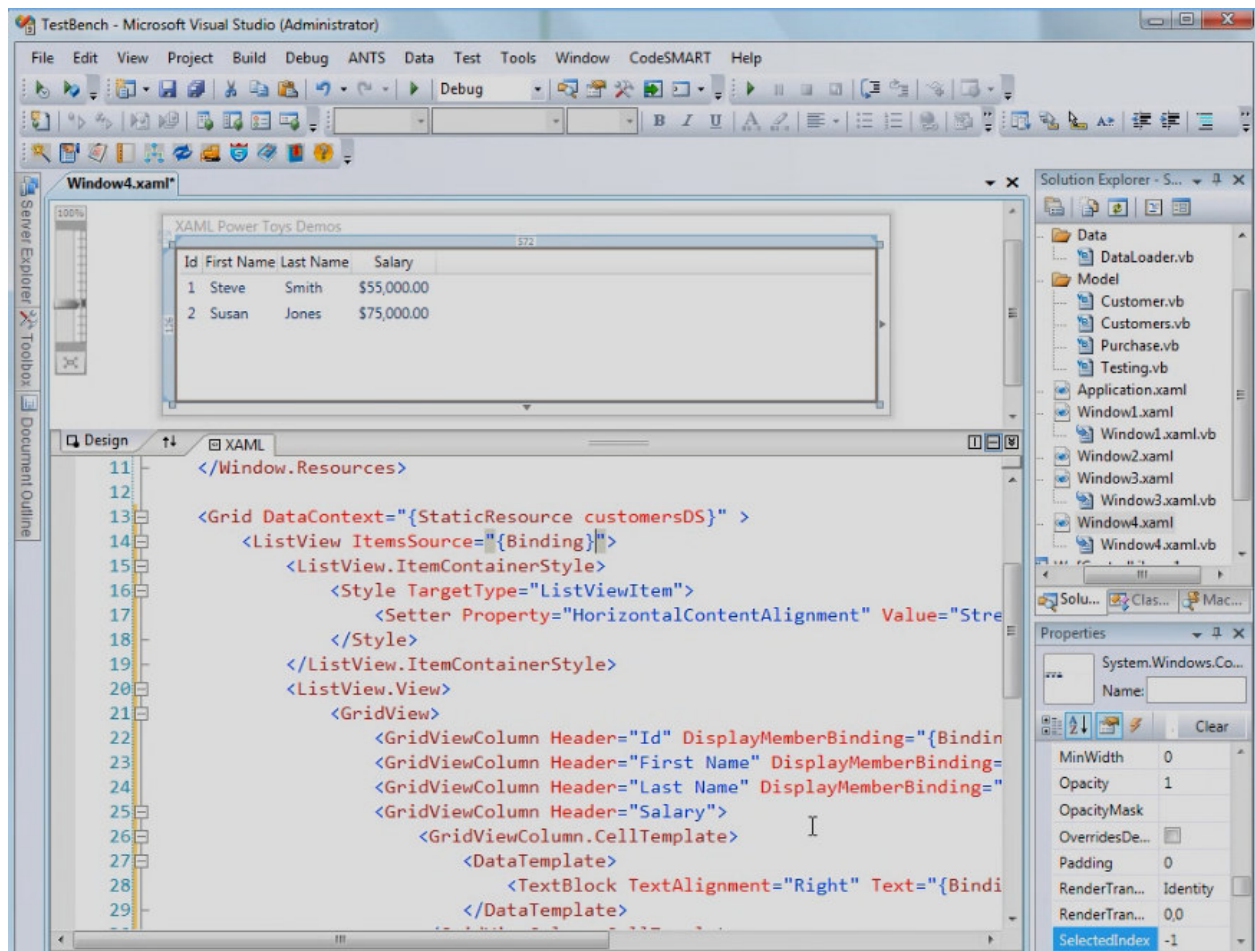


Figure 23. XAML Toys List Control view (at the top) plus generated XAML view (bottom)

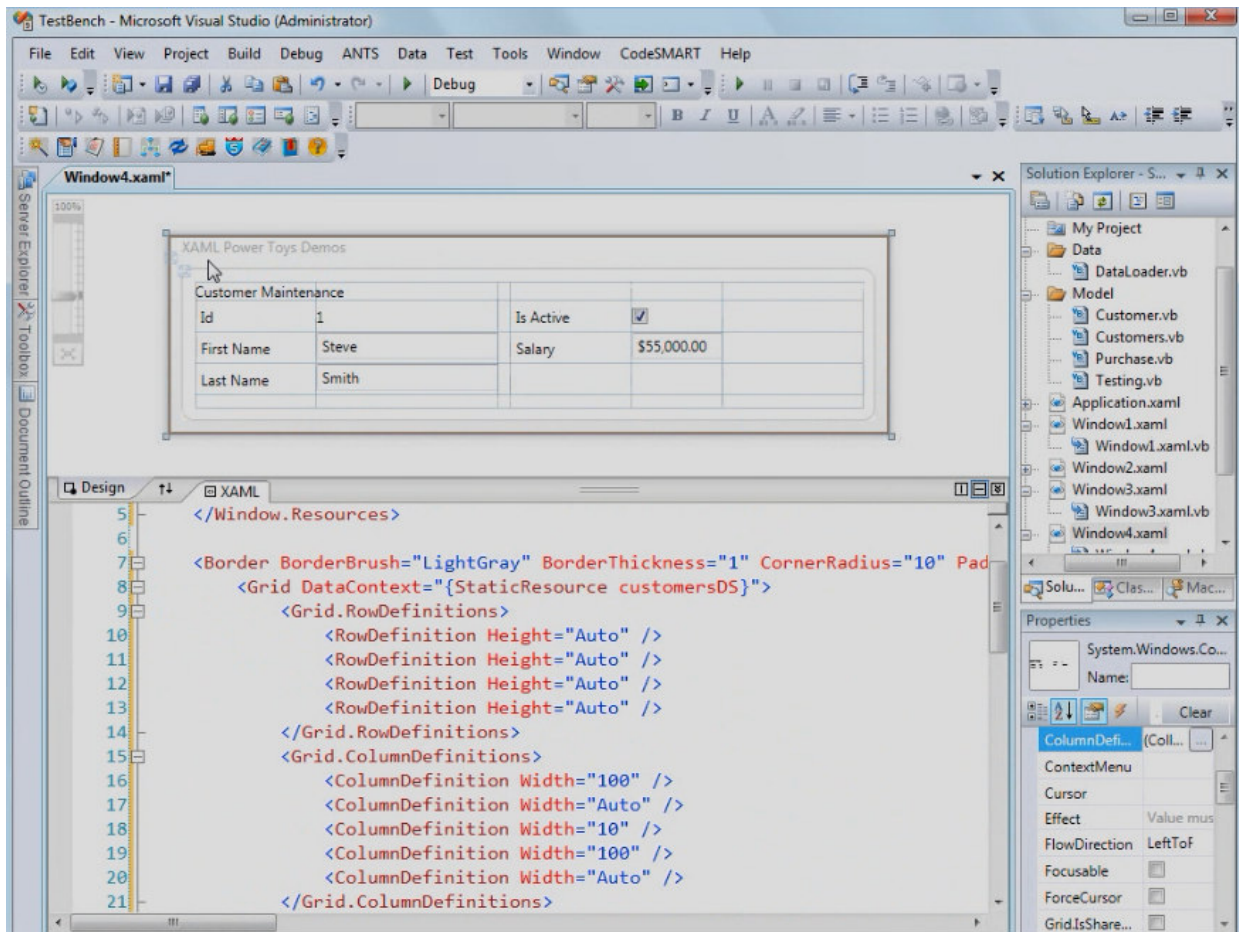


Figure 24. XAML Toys Grid Control view (at the top) plus generated XAML view (bottom)

2.3.5. State of the art summary

We presented four different products aimed at UI generation from model. Microsoft Access presents typical “4 generation language” approach where generation is done at design time. XAML Toys represents similar approach. DynamicData and NakedObjects have generation process being executed in run time. NakedObjects is most similar to “zero” – because of allowing user to fully interact with domain objects by method invocations.

2.4. Model-driven run time graphic interface generation

User interface generation is traditionally seen as design time computer programmer help. Auto generated user interfaces become standard visual programming environment assets that can be customized further by programmer as all other visual assets may be.

There is another possible solution – generation process being run time executed. In this situation all visual appearance customization may be preceded by computer non professional.

3. Tools used

We chose Microsoft .NET [14] as implementation platform. Microsoft .NET is mature software infrastructure with industry proven abilities. As target UI platforms we chose ASP.NET [6] for WWW sessions and Windows Presentation Foundation [7] for windows native client sessions.

All ‘zero’ software was developed in C#3.0 [8] using Visual Studio 2008 [9].

3.1. Net Framework

Microsoft .NET Framework [14] is mature software execution platform with broad standard library and dedicated development environment – Visual Studio. Whole .NET Framework is very broad topic with we are not going to refer here.

“.Net” binary object model is current Microsoft’s standard for component writers. Before “.Net” there was COM standard, with was hard to program (especially in pure C/C++) and has several limitations.

3.2. C# 3.0

C# [8] is flagship .NET Framework’s programming language. In C# 2.0 generic features were introduced to language, thus allowing generic collection classes to be written and incorporated into standard library. In C# 3.0 some further extensions have been introduced among whose lambda’s and extension methods are most important.

3.3. ASP.NET

ASP.NET [6] is .NET Framework’s WWW construction platform. It hides fact that HTTP (HTTP is WWW protocol) as a protocol does not remember any states. ASP.NET creates abstraction layer for a programmer, that allows to treat WWW session in way similar to native windows application – there is state, controls and events as in nowadays window UI frameworks.

3.4. Wpf

Windows Presentation Foundation (WPF) [7] is suggested UI framework for native windows applications in .NET Framework. WPF is vector based, thus allows to be screen / pixel size independent. WPF allows creation of graphic intensive UIs with small programming effort. Those power full capabilities are possible mainly by using declarative programming – dedicated XML based language (XAML) is used.

What is very much important – on modern graphic adapters WPF uses hardware acceleration allowing graphic intensive UIs to run fast.

5. The solution: Generic graphic interface

5.1. On a way to “in-memory object graph as fully functional software”

5.1.1. Run-time object type structure as “The model”

In software engineering process model can be expressed / exists in many forms / stages. Some model forms / stages are derived, and there usually is one basic form / stage from whose all other model forms / stages are derived. In most scenarios, this basic model form / stage is created with direct human interaction.

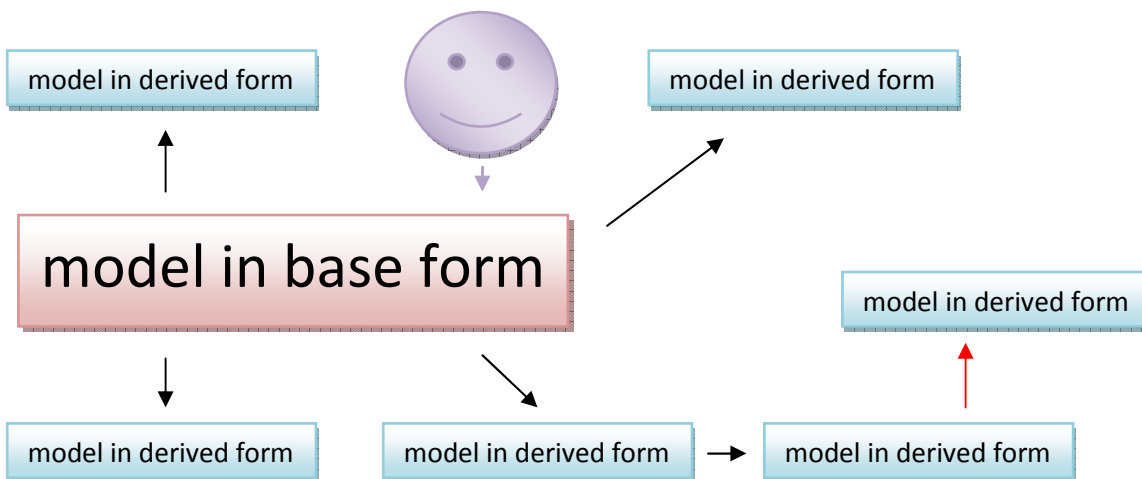


Figure 25. Model in base form and derived ones

Figure 25 shows derivation relations between model instances in different forms / stages. Human is shown as violet smiley face. In this example scenario human creates model in base form and other model forms are created automatically. There are possible other scenarios with more complex relations. For example derivation marked as red arrow could be done by human etc.

Base form of a model used in software creation process is critical factor. For example in typical scenario with object programming language and a relational database system there are possible two basic scenarios: “code first” (shown on figure 26) or “database schema first” (figure 27).

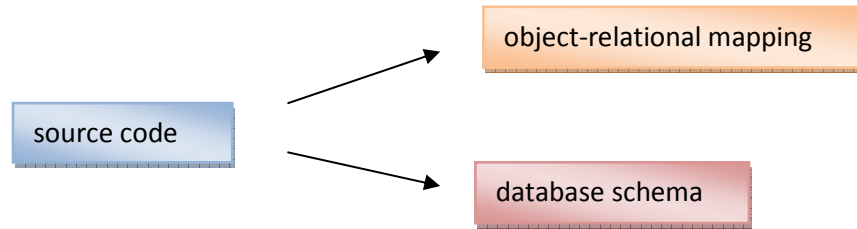


Figure 26. “Code first” scenario

In “code first” scenario, program source code is created and treated as basic asset in software creation process – other model forms are created as derived ones.

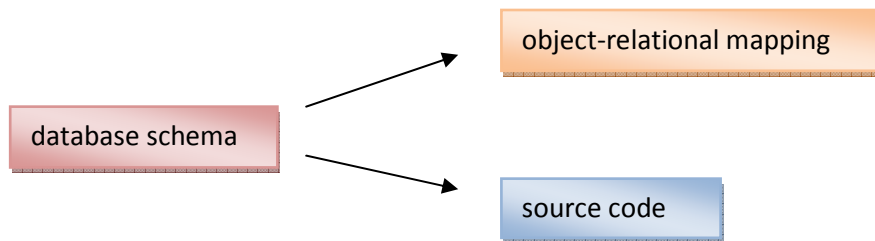


Figure 27. “Database schema first” scenario

In “database schema first” approach, database schema is basic asset and source code and object-relational mapping are generated from database schema definition. Difference between “code first” scenario and “database schema first” scenario may seem as minor one. This is false impression – choosing model base form is one of basic decisions in software engineering process.

As this paper is about realizing “in-memory object graph as fully functional software” vision we choose run-time type structure as base model. Run-time type structure is not exactly same thing as source code type structure. In “.NET” Framework run-time type structure is standardized in form of binary standard – every “.NET” compiler produces standardized binary format output files. If dedicated compiler exists, arbitrary programming language can be used to target “.NET” Framework execution platform. Source code’s constructs are not standardized in terms of whole “.NET” platform – they vary from one language to another. But “.NET” Framework binary type system is common and constant for a whole platform. This situation is shown on figure 28.

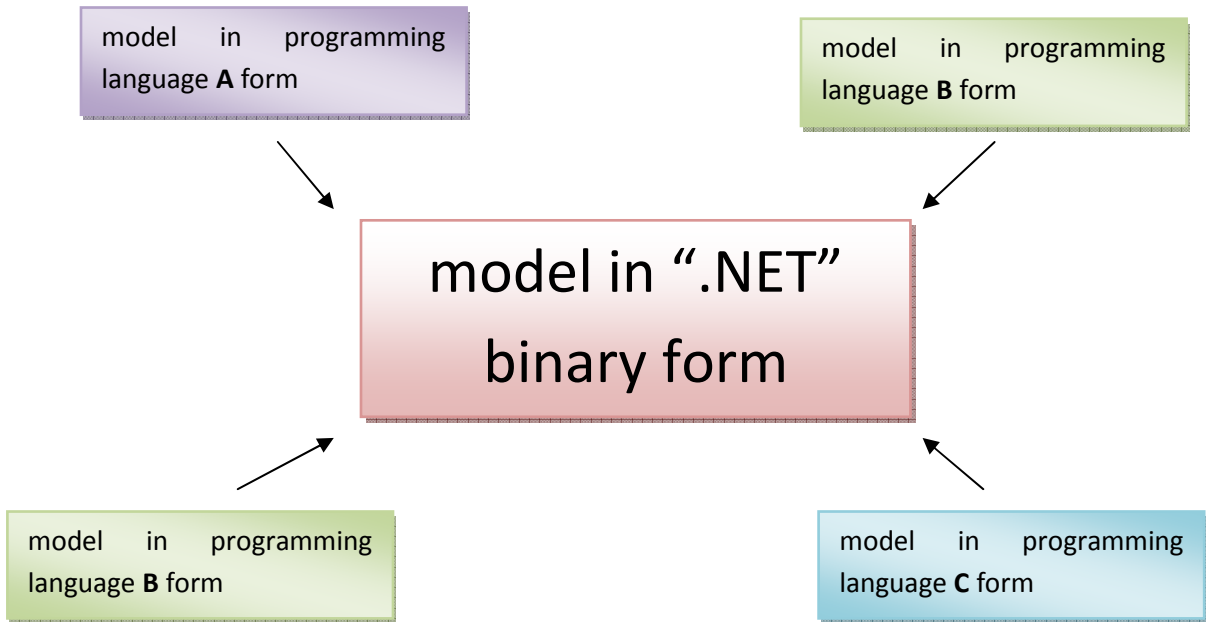
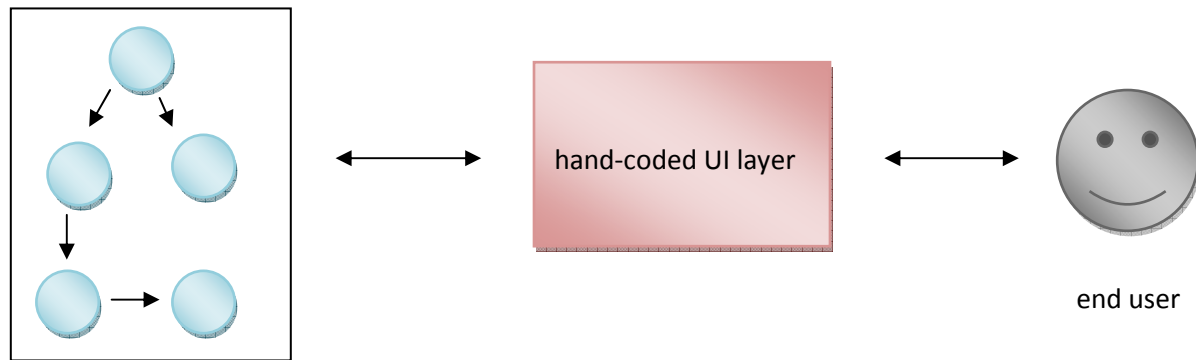


Figure 28. ".NET" binary model form and models expressed in programming languages A, B and C

Choosing run-time binary type model as base model form in software creation process have pros and cons. This is not solution that fits all software engineering processes. As we show later on, we will need some modifications in this area but for now we can assume that run-time binary type model is base form of models in "zero". This solution is code-centric and forces proper types modeling while programming.

5.1.2. User interaction with object graph

In this subsection we will present state of mind that is critical for understanding "in-memory object graph as fully functional software" idea. But first we will show conventional approach that is used widely in object software engineering.



in memory domain objects graph

Figure 29. Hand-coded UI layer

Figure 29 shows fragment of typical line-of-business application having domain modeled by object-oriented methodology. Domain representing objects reside in memory as object graph structure. If domain representing classes are modeled in object oriented mind set, then they will contain domain specific logic represented as methods.

User has no direct access to objects – he can not invoke methods directly. End user is accessing domain objects in not direct way via UI layer. UI layer typically creates additional “abstraction layer” over domain objects graph. This abstraction layer is something that can not be omitted – it is result of software engineering process. Usually such abstraction layer is used to make end user’s experience more easy / productive / etc. Of course such layer is not always bad thing but one must remember that UI layer code does not come for free – time / money / programmers resources are needed for its creation.

So why not allow end user to directly interact with in memory domain object graph ?

For example in-memory object graph inspection tools are available in Visual Studio’s debugger. This feature is shown on figure 30. As we see this tool’s view is quite complicated and is not intended to be used by engineered software end-user.

There is need for some form of automatically generated UI that allows in-memory object graph intuitive interaction for the end-user. This situation is shown on figure 31.

Such automatically generated UI could allow for user’s direct access to in-memory object graph – for example user may have ability not only to view and edit domain objects, but he could have direct access to object methods via method invoke ability. Such situation makes “in-memory object graph as fully functional software” vision more real.

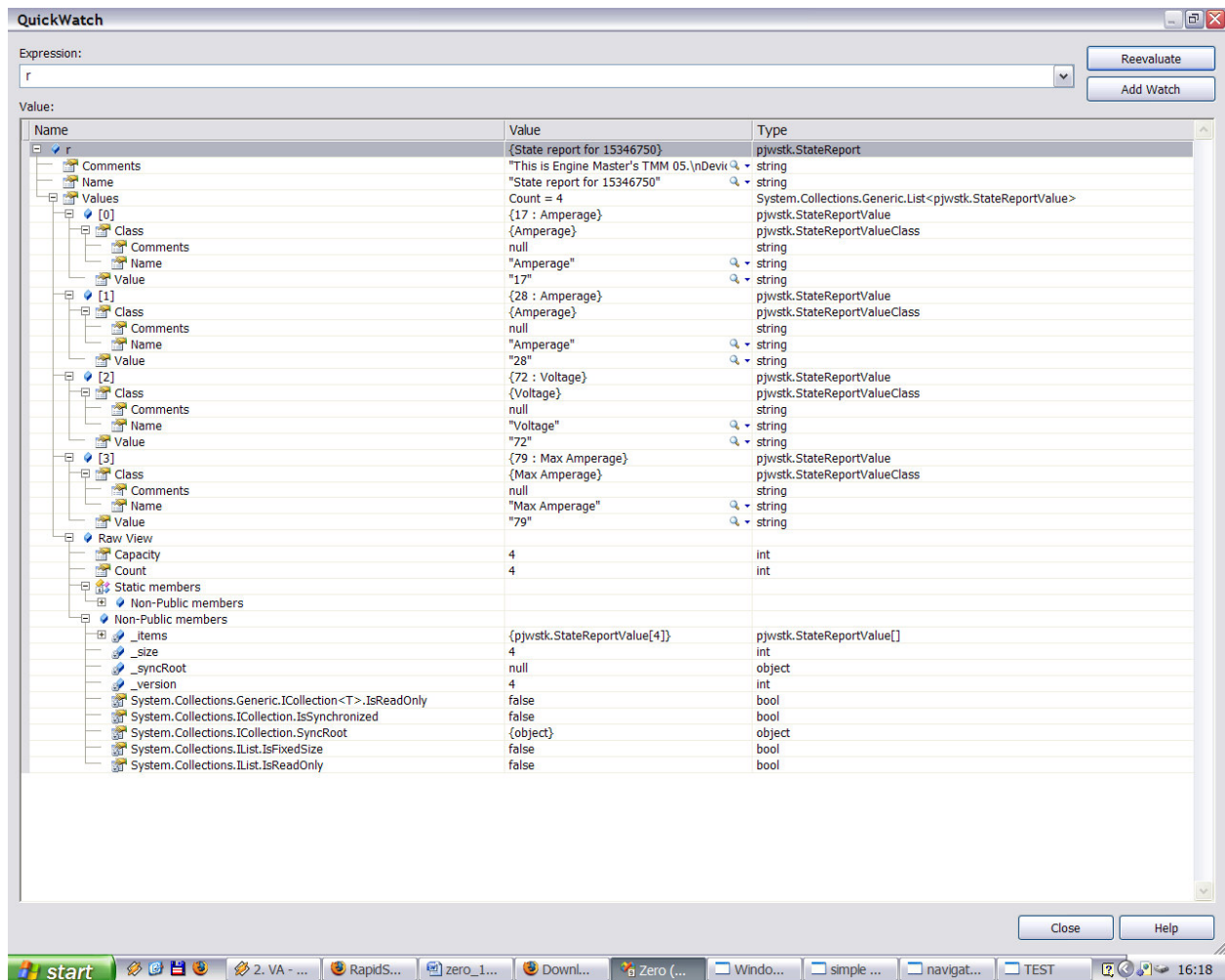
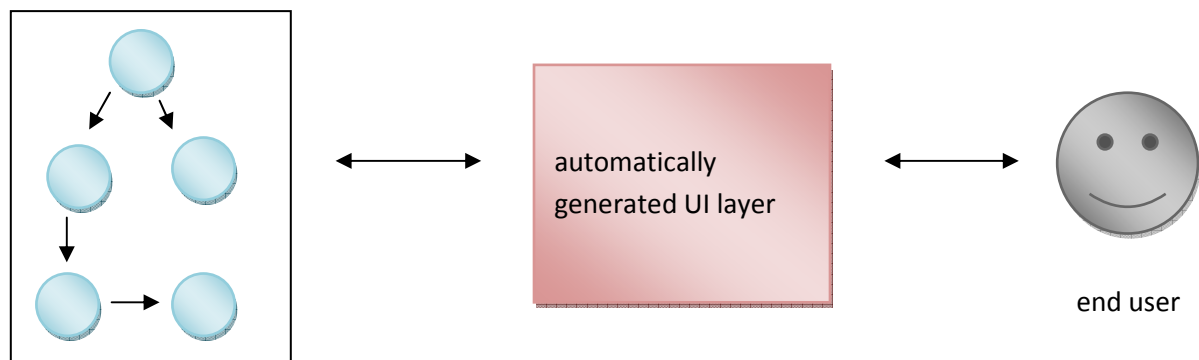


Figure 30. In-memory object graph inspection with Visual Studio's debugger



in memory domain objects graph

Figure 31. Automatically generated UI layer

Of course when auto generating UI we are using “.NET” Framework binary types as the base model. As this is ultra important we will update figure 6 to this situation – see figure 31.

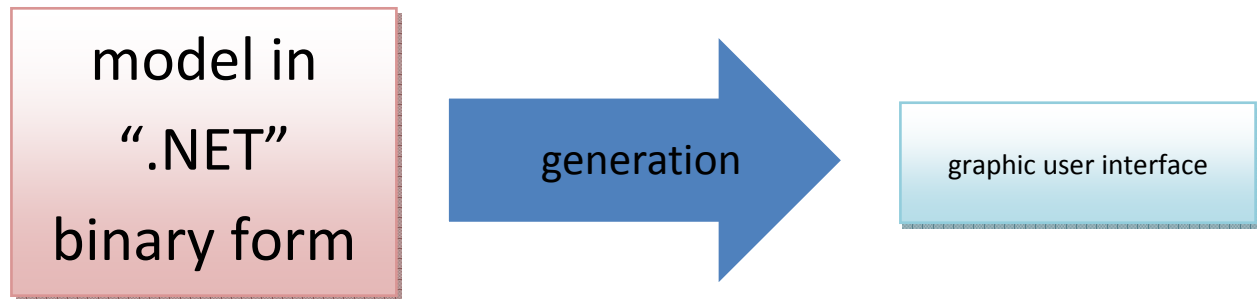


Figure 31. UI generation from “.NET” Framework binary types as the model

End user direct interaction with in-memory object graph via automatically generated UI layer can be used in situations where there are no resources (no time/ no money/ no programmers) for UI layer hand-coding. But maybe such auto generated UI can be competitive with hand coded one ? It could be - if only domain object model is well designed and when it contains UI specific meta attributes (such as “**Description**” field’s meta attribute with forces displaying appropriate tooltip).

5.1.3. Model as multi user interfaces base

Auto generating UI from model have a lot of pros. We have been talking about some of them, but until now we had not been speaking about possibility of creating different target media UIs from the same model. We called this feature “*multi target medium UI generation*”.

For example in “zero” we currently support generating web UI (asp.net one) and windows application UI (based on Windows Presentation Foundation) from the same model. UI specific meta attributes are shared among different target medium UIs if is only possible. For example both UI medium targets use same **Description** meta attribute for customizing tooltips.

Multi target medium UI generation is shown on figure 32.

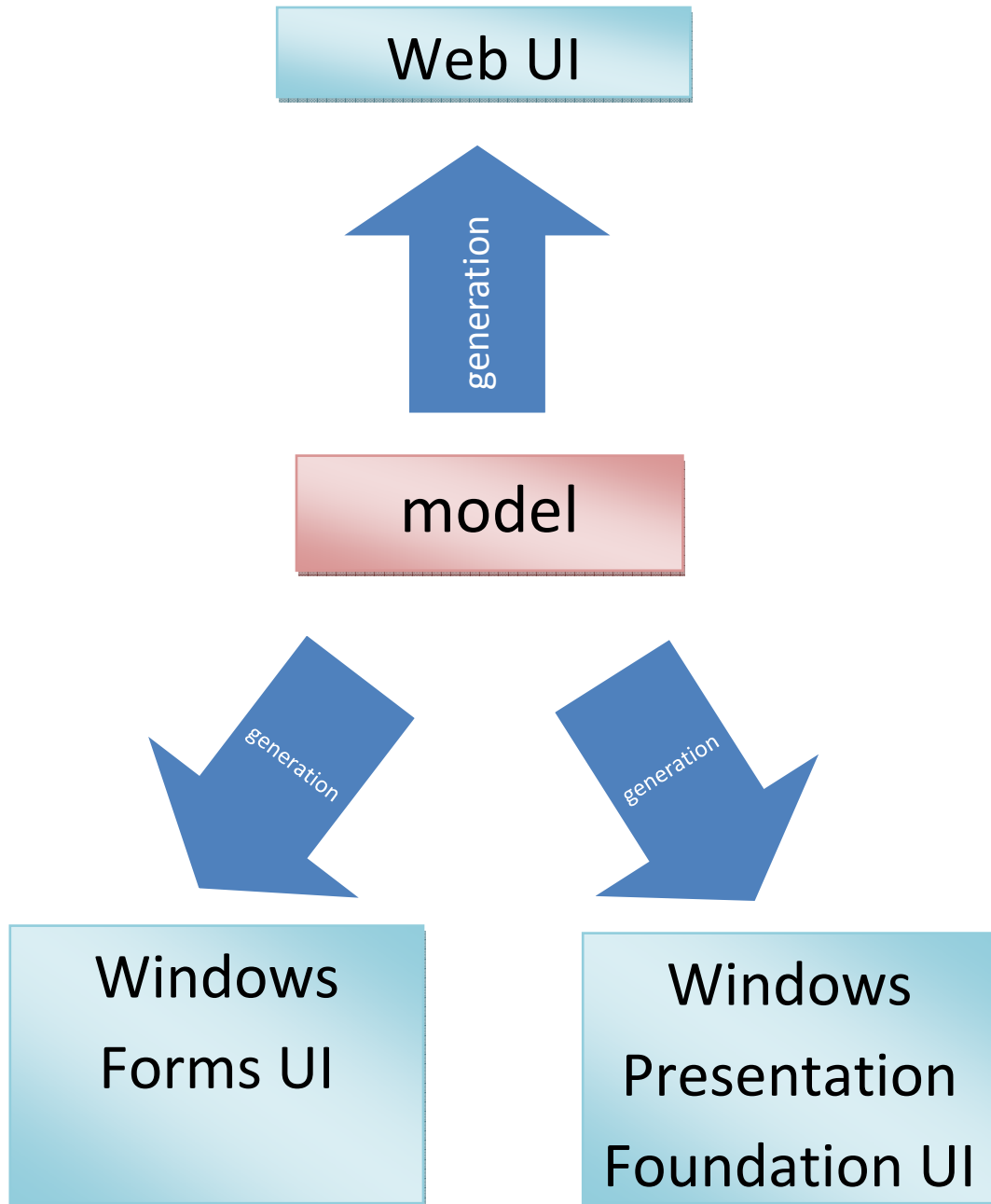


Figure 32. Multi target medium UI generation

5.2. Overall solution architecture overview

Now we will present overall “zero” solution architecture. In “zero” there are two basic processes: “**VisualizerItems** generation” and “session execution”.

“**VisualizerItem** generation” is a process where basic UI elements are generated. “Session execution” is process that allows user to interact with in-memory object graph. “Session execution” uses artifacts (**VisualizerItems**) produced by “**VisualizerItem** generation”. Both processes execute at run-time – typically “session execution” calls “**VisualizerItem** generation” when object of new type needs to be presented to the user.

Figure 33 shows overall “**VisualizerItem** generation” process. **VisualizerItems** are generated UI basic units. **VisualizerItems** are used for **Visualizer** instantiation. One **VisualizerItem** may instant many **Visualizers** as there is such a need. **Visualizers** are UI objects used for user interaction with particular object (this is called *Detail* view) or object collections (this is called *Bulk* view). Typically **Visualizer** is a UI framework control object.

VisualizerItem generation typically consists of following steps:

- Object needs to be presented to the user.
- Binary object’s type model is identified in “.NET” framework runtime.
- “zero’s” **TypeSchema** is generated by chosen or default **TypeSchemaGenerator**.
- **VisualizerItem** is generated by chosen or default **Generator**. **Generator** uses **TypeSchema** and optionally “Generator attributes” to generate **VisualizerItem**.

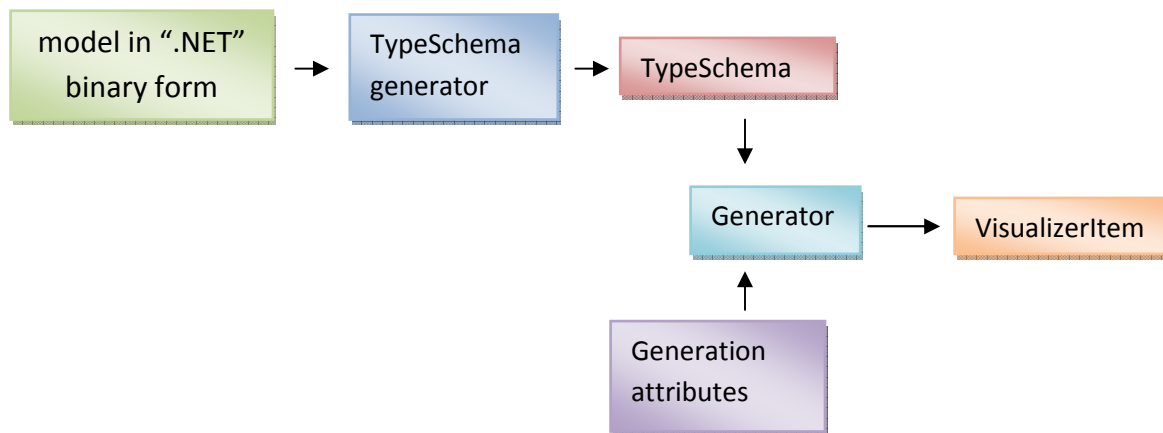


Figure 33. VisualizerItem generation process

Figure 34 shows example “zero” session. Session is basic unit used when serving user interaction with in-memory object graph.

Session may contain:

- **SessionObjectCollectionItems** – objects or object collections that are part of in-memory domain object graph that is root for this Session. Each **SessionObjectCollectionItem** is member of Session’s **ObjectCollection** field.
- **Visualizers** – allows user interaction with particular object or particular object collection. Each Visualizer has associated **SessionObjectCollectionItem** – this association is shown as green arrows.
- **Navigators** – presents **TypeSchemas** and relations between them in graphic way. Allows user to navigate in in-memory object graph schema. Navigators may be used in many types of navigation (for example intensional or extensional).
- **SessionItemsRelations** – describing relations between Session’s items – shown as red arrows. For example at figure 34 relation representing intensional navigation is show.

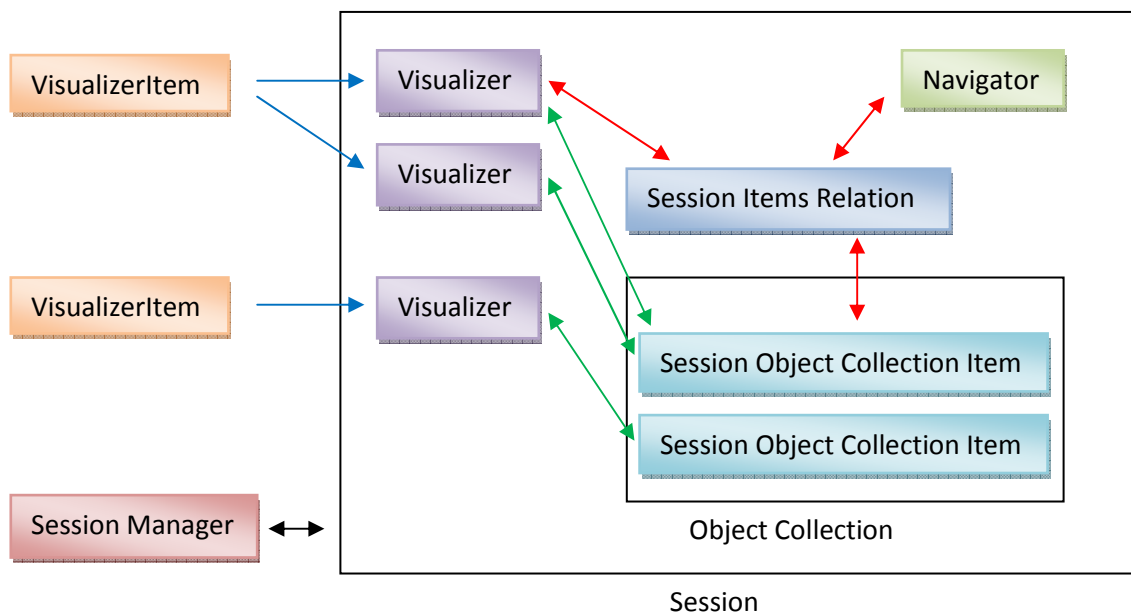


Figure 34. Session

As it was described earlier **Visualizers** are instantiated by appropriate **VisualizerItem** objects (shown as blue arrows). **Sessions** have associated **SessionManager** that handles tasks such as domain object’s fields update or domain object deletion.

5.3.The TypeSchema

TypeSchema semi UML diagram is shown below.

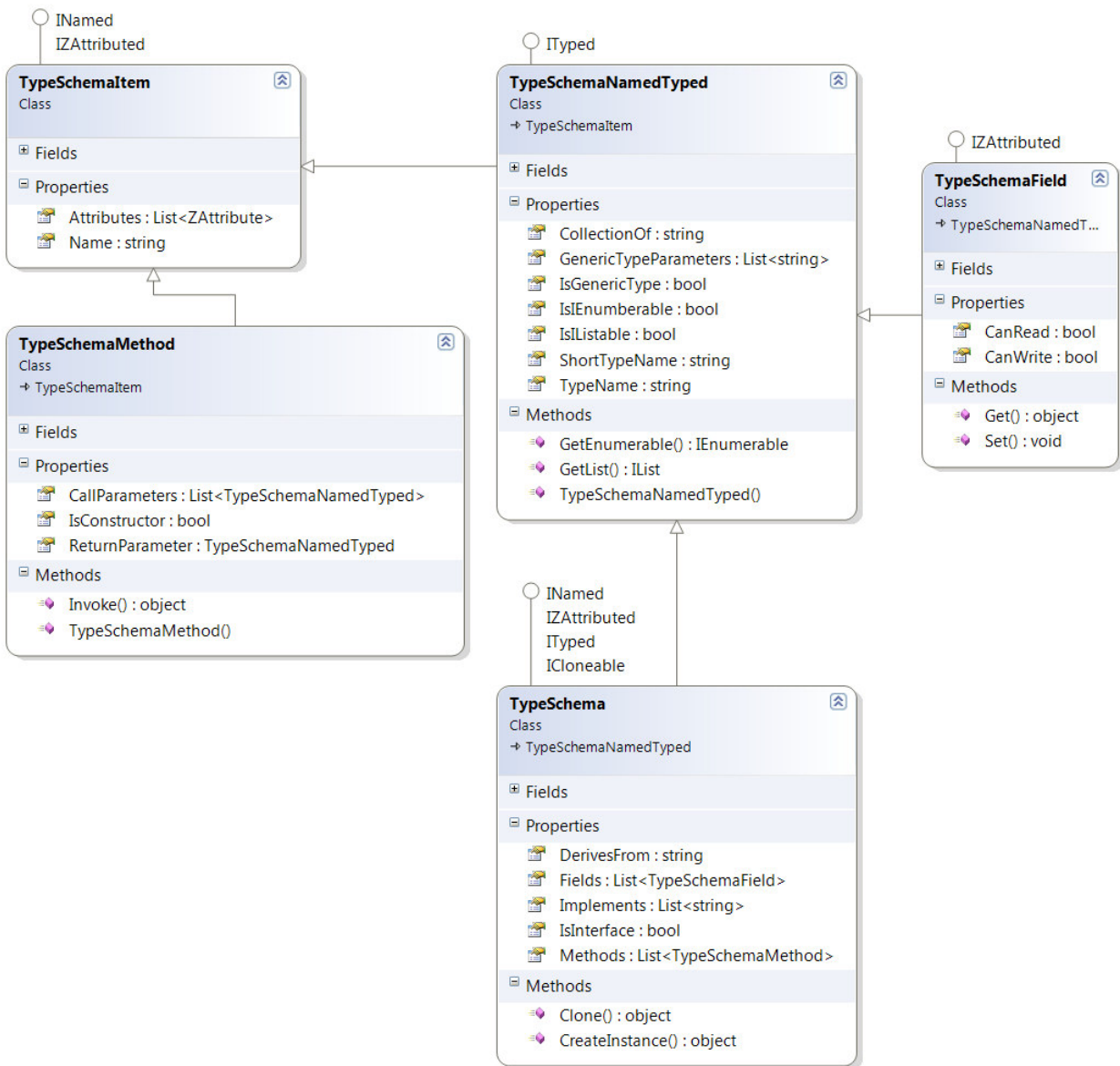


Figure 35. TypeSchema semi UML diagram

5.3.1. TypeSchema overview

TypeSchema is central conception in “zero” framework. Generally speaking a **TypeSchema** models particular entity class. We are not directly using “.NET” Framework binary type information to make “zero” independent of particular model form, thus allowing “zero” to be used for interaction with arbitrary object graphs.

“Zero” supports **TypeSchemaGenerators** plug-in system so **TypeSchemas** can be generated from any model form, of course with assumption that this *any* model can be represented in object oriented way.

This common to all “zero” framework type abstraction (**TypeSchema**) allows “zero” framework to be used for interaction with object graphs being stored in arbitrary store. For example one could prepare **TypeSchemaGenerator** that generates **TypeSchemas** for XML files or object database such as ODRA [15].

In “zero” **TypeSchema** is used not only for getting information about types. It is used for full interaction with type instances (objects). That’s why **TypeSchema** contains mechanisms for methods invocation, setting object’s field’s values etc.

Each **TypeSchema** contains information about:

- Name – **TypeSchema** name (string). Same binary type can have many **TypeSchemas**, for example each generated by different **TypeSchemaGenerator**.
- TypeName – underlying type name
- ShortTypeName – human readable underlying type name
- IsGeneric – information if this type is generic type. Generic types are parameterized by other types
- GenericTypeParameters – collection of **TypeSchema** names that are used to parameterize underlying type
- IsInterface – information if this type is interface and not a class
- DerivesFrom – name of ancestor type (string)
- Implements – collection of interface names that this type implements
- IsEnumerable – information if this type supports getting **IEnumerable** interface
- IsIListable - information if this type supports getting **IList** interface
- Fields – collection of **TypeSchemaFields**

- Methods – collection of **TypeSchemaMethods**
- Attributes – collection of metadata attributes

TypeSchema contains following method:

- CreateInstance – creates underlying type instance (object)

5.3.2. TypeSchema fields

TypeSchemaField represents **TypeSchema** field. **TypeSchemaField** is used for getting field's type and name information representation and for interaction with particular object's fields – getting and setting its values.

Each **TypeSchemaField** contains information about:

- Name – **TypeSchemaField** name (string)
- TypeName – underlying type name
- ShortTypeName – human readable underlying type name
- IsGeneric – information if this type is generic type. Generic types are parameterized by other types
- GenericTypeParameters – collection of **TypeSchema** names that are used to parameterize underlying type
- IsInterface – information if this type is interface and not a class
- IsEnumerable – information if this type supports getting IEnumerable interface
- IsIListable - information if this type supports getting IList interface
- CanRead – information if this field can be read
- CanWrite – information if this field can be written
- Attributes – collection of metadata attributes

TypeSchemaField contains following methods:

- Get – gets particular object's field's value
- Set – sets particular object's field's value

5.3.3. TypeSchema methods

TypeSchemaMethod represents **TypeSchema** method. **TypeSchemaMethod** is used for method's type and name information representation and for method invocation.

Each **TypeSchemaMethod** contains following information:

- Name – method's name
- CallParameters – collection of type information about call parameters
- ReturnParameter – type information about value returned
- IsConstructor – information if this method is underlying type's constructor

TypeSchemaMethod contains following method:

- Invoke – invoke underlying method on particular object

5.3.4. Metadata - TypeSchema attributes

As one can see all above items (**TypeSchema/TypeSchemaField/TypeSchemaMethod**) contains **Attributes** collection. **Attributes** collection stores metadata represented as **ZAttribute**.

Each **ZAttribute** consists of:

- Name – attribute name
- Values - Name and Value pairs containing detailed attribute values

Besides items mentioned above (**TypeSchema/TypeSchemaField/TypeSchemaMethod**) **Attributes** collection is available for much more fine grained **TypeSchema** details. Those are:

- Method's call parameters
- Methods's return parameter

Those metadata are used when generating UI. For example *Description* attribute tells UI generation system to generate graphic representation of Text *Description's* attribute value.

To better illustrate metadata usage and **TypeSchema** generation in general we will give simple example of “.NET” framework type (shown as C# source) and corresponding **TypeSchema** generated by default **ClrTypeSchemaGenerator**.

```
[Serializable]
public class DeviceStatus
{
    [Description(Text="This is device status name")]
    public string Name { get; set; }

    public override string ToString()
    {
        return Name;
    }
}
```

Default **ClrTypeSchemaGenerator** generates following TypeSchema (shown as serialized XML):

```
<?xml version="1.0" encoding="utf-8" ?>
<TypeSchema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema" xsi:type="ClrTypeSchema"
  IsGenericType="false" IsIEnumerable="false" IsIListable="false"
  IsInterface="false">
  <Attributes>
    <ZAttribute Name="System.SerializableAttribute">
      <Items />
    </ZAttribute>
  </Attributes>
  <Name>pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
  PublicKeyToken=null</Name>
  <TypeName>pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
  PublicKeyToken=null</TypeName>
  <ShortTypeName>DeviceStatus</ShortTypeName>
  <DerivesFrom>System.Object, mscorlib, Version=2.0.0.0, Culture=neutral,
  PublicKeyToken=b77a5c561934e089</DerivesFrom>
  <Fields>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="false"
    IsIEnumerable="false" IsIListable="false" CanRead="true" CanWrite="true"
    IsProperty="true">
      <Attributes>
```

```

        <ZAttribute Name="Zero.Description">
            <Items>
                <ZAttributeValue>
                    <Name>Text</Name>
                    <Value xsi:type="xsd:string">This is device
                        status name</Value>
                </ZAttributeValue>
            </Items>
        </ZAttribute>
    </Attributes>
    <Name>Name</Name>
    <TypeName>System.String, mscorlib, Version=2.0.0.0, Culture=neutral,
        PublicKeyToken=b77a5c561934e089</TypeName>
    <ShortTypeName>String</ShortTypeName>
</TypeSchemaField>
</Fields>
<Methods />
<ClassName>DeviceStatus</ClassName>
<Namespace>pjwstk</Namespace>
</TypeSchema>

```

In above **DeviceStatus** type definition one can see following field definition.

```

[Description(Text="This is device status name")]
public string Name { get; set; }

```

Name field has ***Description*** attribute. Attributes decorating programming language constructs are common idea in “.NET” framework. Based on above ***Description*** attribute Default **ClrTypeSchemaGenerator** produces following **ZAttribute** (shown as serialized XML):

```

<ZAttribute Name="Zero.Description">
    <Items>
        <ZAttributeValue>
            <Name>Text</Name>
            <Value xsi:type="xsd:string">This is device status
                name</Value>
        </ZAttributeValue>
    </Items>
</ZAttribute>

```

As one can see such **ZAttribute**’s representation in serialized XML form is quite “big”. In “zero’s” framework usage scenarios there is place for manual editing of **TypeSchema** in serialized XML form. Such “big” **ZAttribute**’s representation in serialized XML form is not suitable for this manual editing

usage scenario. That's why "zero" framework allows to use "short" **ZAttribute** representation in serialized XML form.

Above DeviceStatus **TypeSchema** represented with "short" **ZAttribute** coding look as follows:

```
<?xml version="1.0" encoding="utf-8" ?>
<TypeSchema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema" xsi:type="ClrTypeSchema"
  IsGenericType="false" IsIEnumerable="false" IsIListable="false"
  IsInterface="false" System.SerializableAttribute-ATTRIBUTE="">
  <Name>pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
  PublicKeyToken=null</Name>
  <TypeName>pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
  PublicKeyToken=null</TypeName>
  <ShortTypeName>DeviceStatus</ShortTypeName>
  <DerivesFrom>System.Object, mscorlib, Version=2.0.0.0, Culture=neutral,
  PublicKeyToken=b77a5c561934e089</DerivesFrom>
  <Fields>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="false"
    IsIEnumerable="false" IsIListable="false" CanRead="true" CanWrite="true"
    IsProperty="true" Zero.Description-Text="This is device status name">
      <Name>Name</Name>
      <TypeName>System.String, mscorlib, Version=2.0.0.0, Culture=neutral,
      PublicKeyToken=b77a5c561934e089</TypeName>
      <ShortTypeName>String</ShortTypeName>
    </TypeSchemaField>
  </Fields>
  <Methods />
  <ClassName>DeviceStatus</ClassName>
  <NameSpace>pjwstk</NameSpace>
</TypeSchema>
```

As it could be seen above, *Description* attribute is now represented as **TypeSchemaField**'s XML attribute:

Zero.Description-Text="This is device status name"

Above XML attribute's name consists of two parts "Zero.Description" and "Text". First part represents **ZAttribute** name. Second part represents **ZAttribute**'s Name/Value pair's Name. Attribute's value represents Name/Value pair's Value.

5.3.5. Working with TypeSchema

In running instance of “zero” framework list of generated **TypeSchemas** is stored in memory. Those **TypeSchemas** are identified by its Name. Because relations between:

- native form types (for example “.NET” type binary form)
- **TypeSchema** Names
- **TypeSchemas**

are not all 1-1, “zero” framework introduces several interfaces describing services serving translations between above items. Those interfaces are part of “zero” plug-in system that allows flexibility in those translations implementations. Those interfaces are:

- **TypeNameGenerator**
- **TypeFinder**
- **TypeSchemaGenerator**
- **TypeSchemaGetter**

Possible translations are shown on figure 36.

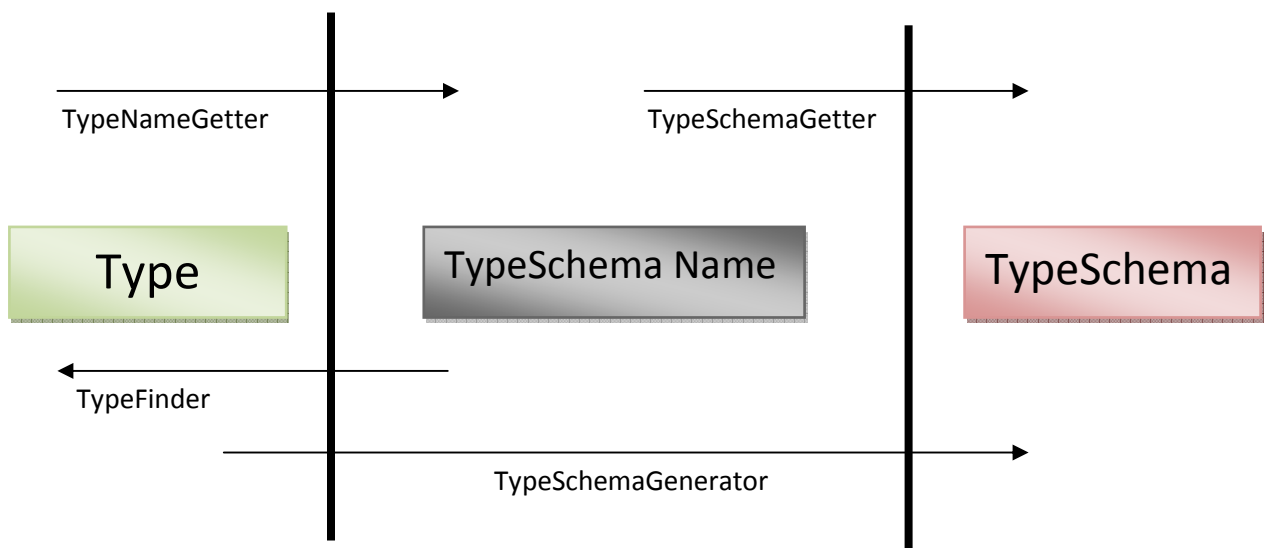


Figure 36. Type<>TypeSchemaName<>TypeSchema translations

5.3.6. ClrTypeSchema

Because basic “zero” framework usage scenario is based on “.NET” framework objects interaction we will describe some additions addressing “.NET” framework made to **TypeSchema**. “.NET” framework type aware **TypeSchema** is called **ClrTypeSchema**. “Clr” prefix comes from “Common Language Runtime” with is name of “.NET” framework type system.

We will present “.NET” framework based additions made to above **TypeSchema** items.

ClrTypeSchema represents “.NET” framework type aware **TypeSchema**. In addition to **TypeSchema**, **ClrTypeSchema** contains following information:

- **ClassName** – string representing types name
- **Namespace** – string representing types namespace
- **Type** – “.NET” specific type information

ClrTypeSchemaField represents “.NET” framework type aware **TypeSchemaField**. In addition to **TypeSchemaField**, **ClrTypeSchemaField** contains following information:

- **IsProperty** – information indicating if this field has underlying “.NET” property or field
- **PropertyInfo** – “.NET” specific property information. Is used when field has underling “.NET” property
- **FieldInfo** – “.NET” specific property information. Is used when field has underling “.NET” field

ClrTypeSchemaMethod represents “.NET” framework type aware **TypeSchemaMethod**. In addition to **TypeSchemaMethod**, **ClrTypeSchemaMethod** contains following information:

- **MethodInfo** – “.NET” specific method information

At this moment non of “zero” framework services uses information stored in **ClrTypeSchema** but some plug-ins always are able to do so.

5.4. Generator and VisualizerItem

Generator and **VisualizerItem** semi UML diagram is shown below.

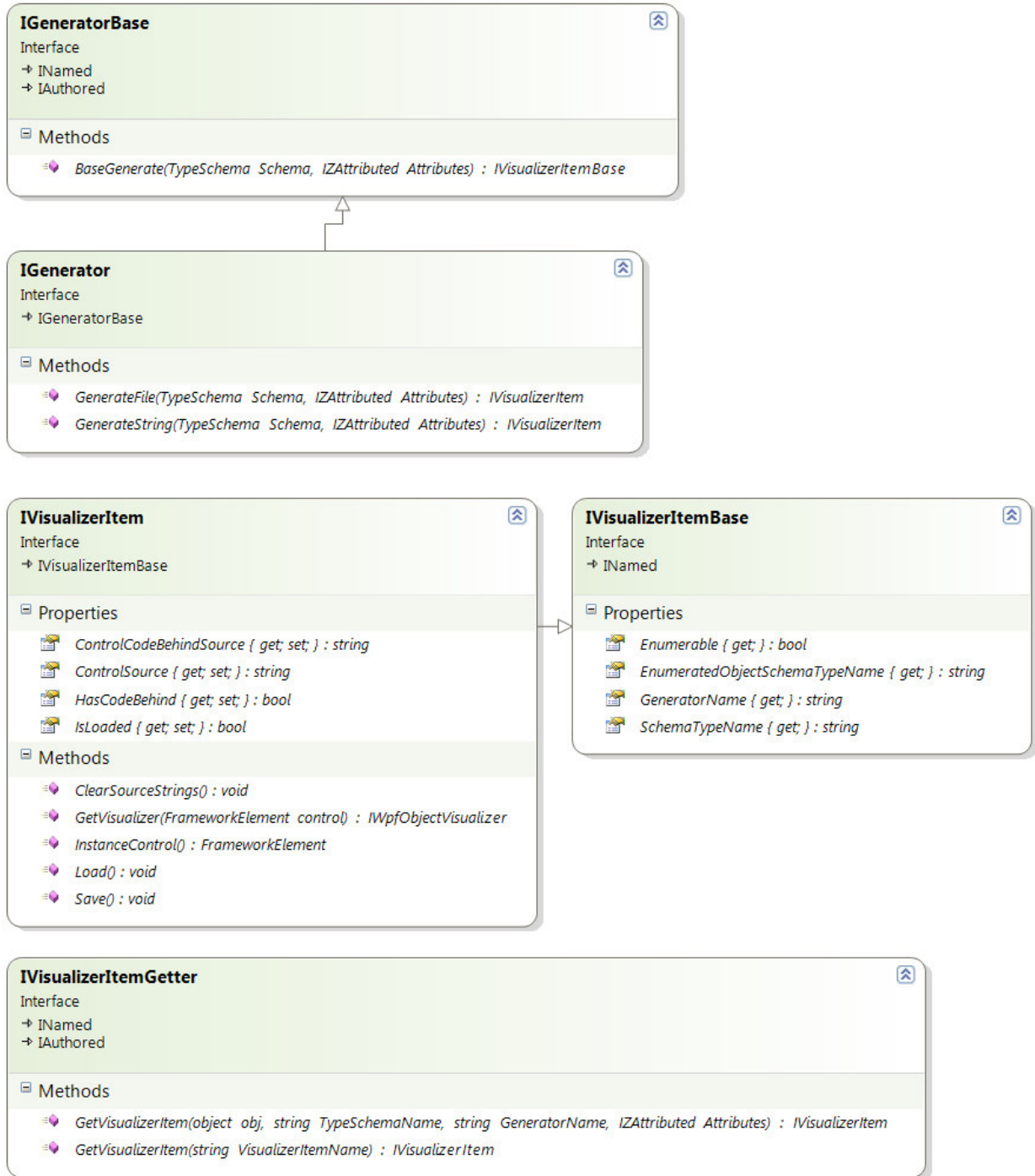


Figure 37. Generator and VisualizerItem semi UML diagram

Generator is center of “zero’s” UI generation infrastructure. **Generator** takes **TypeSchema** and generation attributes as an input and produces **VisualizerItem** as output. **VisualizerItem** is used later on to instance **Visualizers**. **Visualizers** are particular domain type dedicated objects serving domain objects UI representation.

In running instance of “zero” framework list of available **Generators** is stored in system. **Generators** are being indentified by its Name. New **Generators** can be developed and integrated with “zero” framework without need of “zero” framework recompilation.

Typical **Generator** implementations are **SimpleGenerator** and **SimpleEnumerableGenerator**. **SimpleGenerator** creates UI controls in *Detail* view. **SimpleEnumerableGenerator** creates UI controls in *Bulk* view.

Generation process is determined by:

- **Generator** implementation code
- **TypeSchema** and its metadata (**ZAttributes**)
- Generation attributes

Generation attributes are treated in same way as **TypeSchema** top level metadata. So if one needs to customize generation process it can be achieved by passing additional attributes to the generator as generation attributes or by preparing dedicated **TypeSchema** with those attributes as **TypeSchema** top level metadata.

Also **VisualizerItemGetters** serving plug-in system is supported. **VisualizerItemGetters** return **VisualizerItems**. **VisualizerItem** preparation by **VisualizerItemGetter** can be achived by two ways:

- Providing **VisualizerItemGetter**’s **GetVisualizerItem** method one of following parameters combinations:
 - *Object that needs to be presented to the user.*
GetVisualizerItem implementation will get proper **TypeSchema** and will use default normal (if passed object is single one) or enumerable (if passed object is collection) generator.
 - ***TypeSchema** name and **Generators** name.*
 - *Object that needs to be presented to the user and **Generators** name.*
GetVisualizerItem implementation will get proper **TypeSchema** and will use specified **Generator**.
 - *Object that needs to be presented to the user and **TypeSchema** name.*
GetVisualizerItem implementation will use specified **TypeSchema** and will use default normal (if passed object is single one) or enumerable (if passed object is collection) generator.
- Providing name of needed **VisualizerItem** to the **VisualizerItemGetter**’s **GetVisualizerItem** method

VisualizerItem name is three part string. Those parts are:

- **TypeSchema** name
- **Generator** name
- Generation attributes encoded in *PUKO* microlanguage

To better illustrate **VisualizeItem** names syntax we will give simple example of such name.

```
{@InheritanceWpfTab@InheritanceGroup@pjwstk.Employee, PowerPlant,  
Version=1.0.0.0, Culture=neutral,  
PublicKeyToken=null|Standard|{Zero.NoUpdateControl}}
```

We will discuss above name by its parts.

- **TypeSchema** name

```
@InheritanceWpfTab@InheritanceGroup@pjwstk.Employee, PowerPlant,  
Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
```

Above **TypeSchema** name is complex one. Gennerally it is based on **TypeSchema** named “pjwstk.Employee, PowerPlant, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null”. This **TypeSchema** is then processed by two **TypeSchemaTransforms**: “InheritanceWpfTab” and “InheritanceGroup”. Those example transforms are adding UI specific additional meta attributes based on inheritance types hierarchy to arbitrary **TypeSchema**.

- **Generator** name

```
Standard
```

This is name of **Generator** that should be used.

- Generation attributes encoded in *PUKO* microlanguage

```
{Zero.NoUpdateControl}
```

This is Zero.NoUpdateControl **ZAttribute** that will be passed to **Generator** as generation attribute.

5.5. The Visualizer

Visualizer semi UML diagram is shown below.

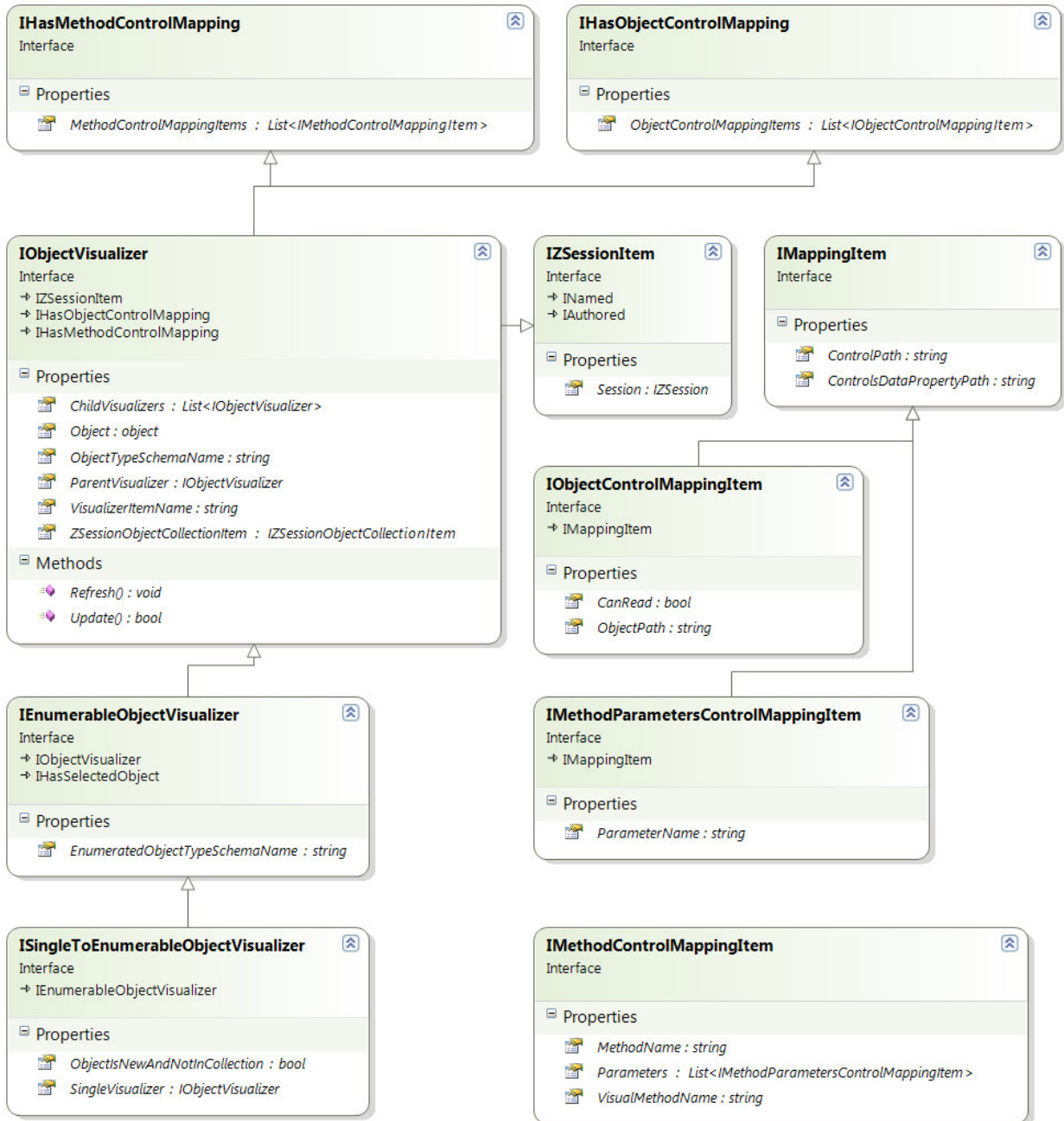


Figure 38. Visualizer semi UML diagram

Visualizer is particular domain type object serving domain objects UI representation.

Each **Visualizer** instance contains following informations:

- ChildVisualizers – collection of nested **Visualizers**
- Object – domain object that is being displayed by **Visualizer**
- ObjectTypeSchemaName – **TypeSchema** name of displayed object
- ParentVisualizer – **Visualizer** in which this **Visualizer** is nested
- VisualizerItemName – name of **VisualizerItem**
- ZSessionObjectCollectionItem – **ZsessionObjectCollectionItem** containing object that is displayed by **Visualizer**

Besides **ObjectVisualizers** that are typically used to represent single object in *Detail* view (see figure 1) there are **EnumerableObjectVisualizers** that are used to represent all object collections at once – it is called *Bulk* view (see figure 2).

EnumerableObjectVisualizer contains `EnumeratedObjectTypeSchemaName` with stores **TypeSchema** name of objects in collection.

If one needs to use single **ObjectVisualizer** for displaying collection of objects one can use **Visualizer** of **SingleToEnumerableObjectVisualizer** type. **SingleToEnumerableObjectVisualizer** encapsulates single **ObjectVisualizer** (in `SingleObjectVisualizer` property) acting for its consumers as **EnumerableObjectVisualizer**.

“Mapping” interfaces shown at figure 38 are used for domain object <> UI objects mapping for domain objects fields and methods.

5.6. The Session

Session semi UML diagram is shown below.

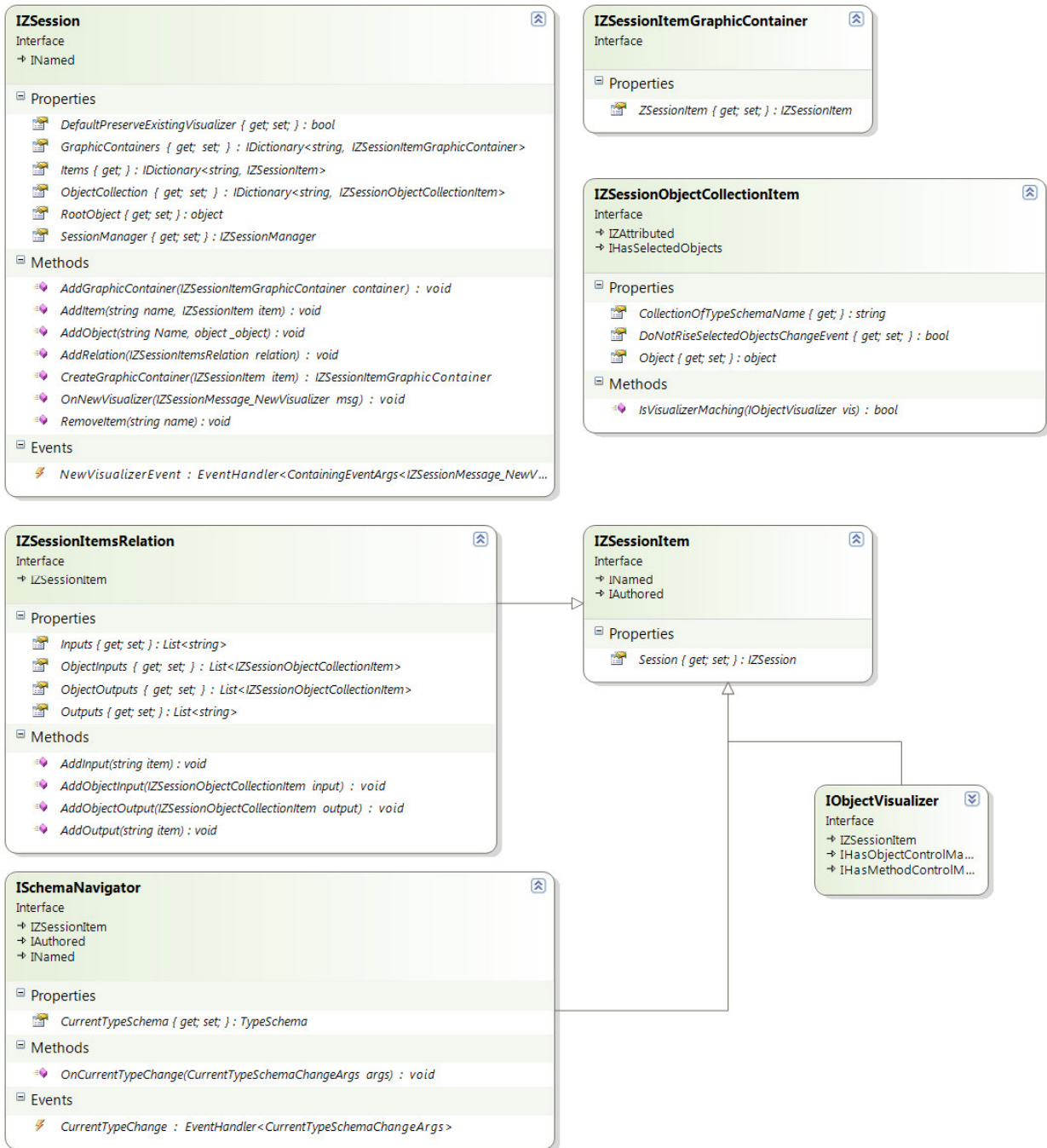


Figure 39. Session semi UML diagram

Session is central item in “zero” framework execution. Session is basic unit used when serving user interaction with in-memory object graph.

Session (see figure 34) may contain:

- **SessionObjectCollectionItems** – objects or object collections that are part of in-memory domain object graph that is root for this **Session**. Each **SessionObjectCollectionItem** is member of **Session**’s **ObjectCollection** field.
- **Visualizers** – allows user interaction with particular object or particular object collection. Each **Visualizer** has associated **SessionObjectCollectionItem**.
- **Navigators** – presents **TypeSchemas** and relations between them in graphic way. Allows user to navigate in in-memory object graph’s schema. Navigators may be used in many types of navigation (for example intensional or extensional).
- **SessionItemsRelations** – describing relations between Session’s items. For example at figure 34 relation representing intensional navigation is show.

Figure 40 shows computer running typical WPF **Session**. **Session** from figure 40 uses graph based **Navigator** – for untrained user it may seem complicated . Figure 41 shows WPF **Session** with simple list based **Navigator** (small window at bottom right) that does not look so complicated as graph based one does.

Web based **Session** in shown on figure 42.

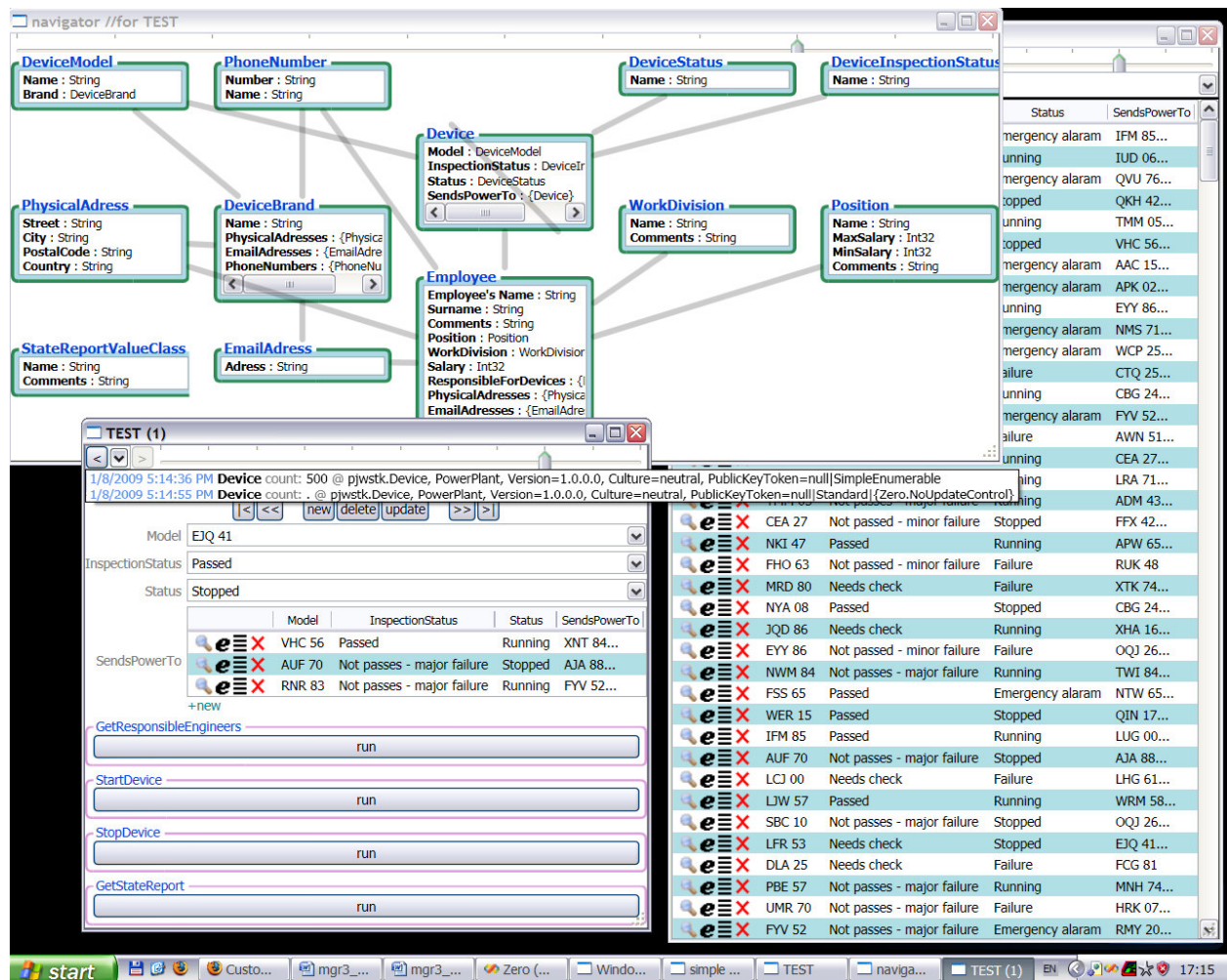


Figure 40. Example WPF session with graph based Navigator

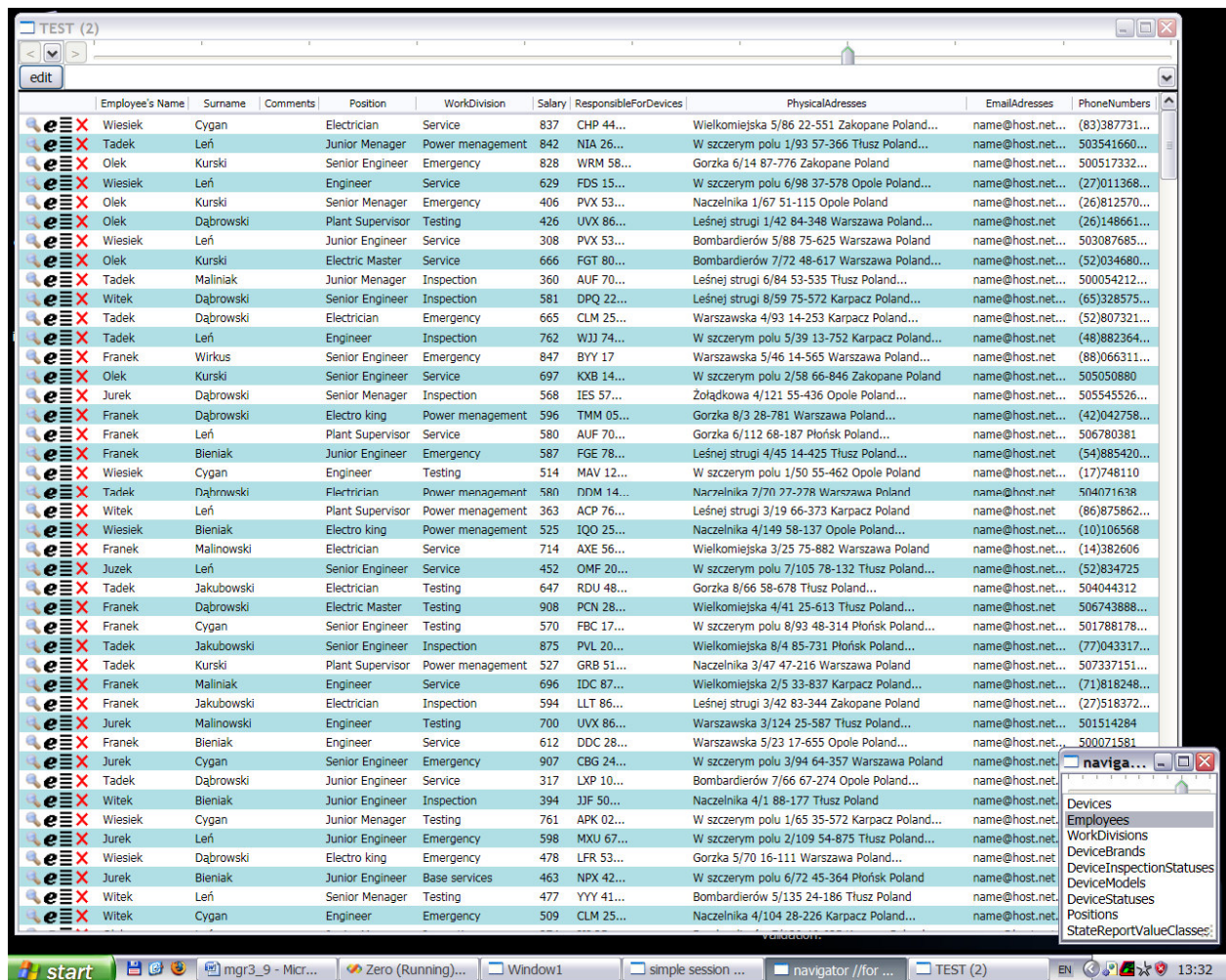


Figure 41. Example WPF session with graph based Navigator

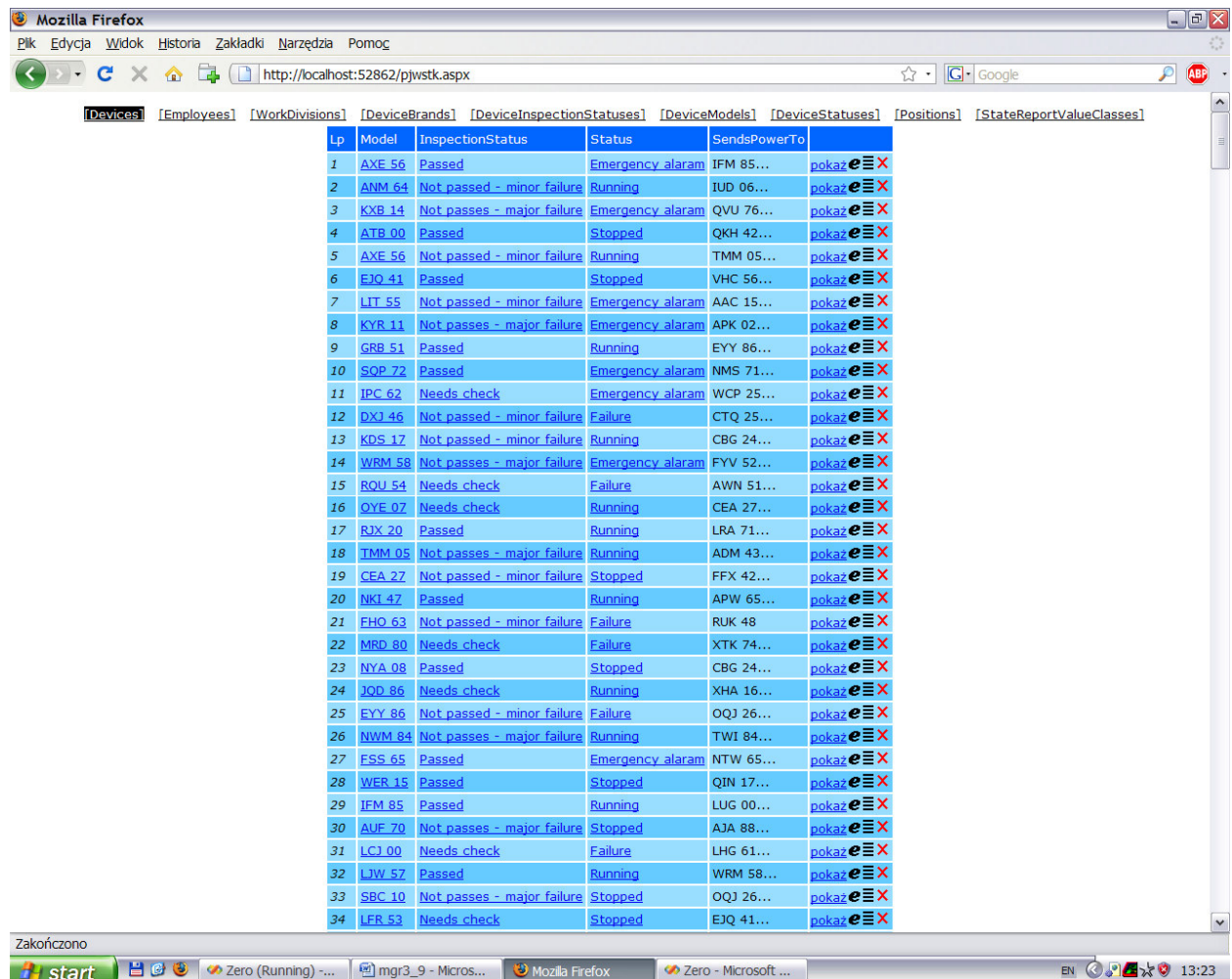


Figure 42. Example ASP.NET Session. Navigator is at page top.

5.6.1. ObjectCollection

ObjectCollectionItems represents object or set of objects that is part of whole domain object graph. Each session may contain many **ObjectCollectionItems**, each representing users individual data mining sub task. Additionally **ObjectCollectionItem** contains information about what objects from **ObjectCollectionItem**'s set are selected by user. This information is used by some types of navigation.

As user navigates thru domain object graph content of **ObjectCollectionItem** will change and will correspond to users activities. Those changes are not defined by **Session** but by particular **SessionItemsRelations** defining **ObjectCollectionItem** changes in context of user interaction with **SessionItems** such as **Navigators**.

5.6.2. Session Items

SessionItem is abstract type that represents all session items. Currently those are:

- **ObjectVisualizers**
- **SchemaNavigators**
- **SessionItemRelations**

5.6.3. Session Items Graphic Containers

“Zero” framework is aimed at graphic user interfaces. That’s why we introduce **SessionItemGraphicContainer**. Those serves as graphic containers for session items such as **Visualizers** or **SchemaNavigators**.

To better illustrate **SessionItemGraphicContainer** conception we will use window shown in figure 1. Its **SessionItemGraphicContainer** is show on figure 43 and its **Visualizer** is shown on figure 44.

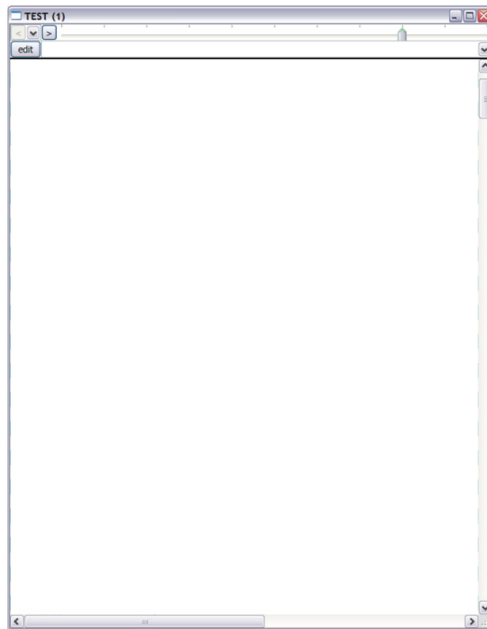


Figure 43. Figure 1’s SessionItemGraphicContainer

X	Olek	Kurski	Senior Manager	Emergency	406	PVX 53...
X	Olek	Dąbrowski	Plant Supervisor	Testing	426	UVX 86...
X	Wesiek	Leń	Junior Engineer	Service	308	PVX 53...
X	Olek	Kurski	Electric Master	Service	666	FGT 80...
X	Tadek	Maliniak	Junior Manager	Inspection	360	AUF 70...
X	Witek	Dąbrowski	Senior Engineer	Inspection	581	DPQ 22...
X	Tadek	Dąbrowski	Electrician	Emergency	665	CLM 25...
X	Tadek	Leń	Engineer	Inspection	762	WJJ 74...
X	Franeek	Wirkus	Senior Engineer	Emergency	847	BYJ 17...
X	Olek	Kurski	Senior Engineer	Service	697	KOB 14...
X	Jurek	Dąbrowski	Senior Manager	Inspection	568	IES 57...
X	Franeek	Dąbrowski	Electro king	Power management	596	TMM 05...
X	Franeek	Leń	Plant Supervisor	Service	580	AUF 70...
X	Franeek	Bieniak	Junior Engineer	Emergency	587	FGT 78...
X	Wesiek	Cygan	Engineer	Testing	514	MAV 12...
X	Tadek	Dąbrowski	Electrician	Power management	580	DDM 14...
X	Witek	Leń	Plant Supervisor	Power management	363	ACP 76...
X	Wesiek	Bieniak	Electro king	Power management	525	IQO 25...
X	Franeek	Malinowski	Electrician	Service	714	AXE 56...
X	Juzek	Leń	Senior Engineer	Service	452	OMF 20...
X	Tadek	Jakubowski	Electrician	Testing	647	RDU 48...
X	Franeek	Dąbrowski	Electric Master	Testing	908	PCN 28...
X	Franeek	Cygan	Senior Engineer	Testing	570	FBC 17...
X	Tadek	Jakubowski	Senior Engineer	Inspection	875	PVL 20...
X	Tadek	Kurski	Plant Supervisor	Power management	527	GRB 51...
X	Franeek	Maliniak	Engineer	Service	696	IDC 87...
X	Franeek	Jakubowski	Electrician	Inspection	594	LLT 86...
X	Jurek	Malinowski	Engineer	Testing	700	UVX 86...
X	Franeek	Bieniak	Engineer	Service	612	DOC 28...
X	Jurek	Cygan	Senior Engineer	Emergency	907	CBG 24...
X	Tadek	Dąbrowski	Junior Engineer	Service	317	LQO 10...
X	Witek	Bieniak	Junior Engineer	Inspection	394	JIF 50...
X	Wesiek	Cygan	Junior Manager	Testing	761	APK 02...
X	Jurek	Leń	Junior Engineer	Emergency	598	MDJ 67...
X	Wesiek	Dąbrowski	Electro king	Emergency	478	LFR 53...
X	Jurek	Bieniak	Junior Engineer	Base services	463	NPX 42...
X	Witek	Leń	Senior Manager	Testing	477	YYY 41...
X	Witek	Cygan	Engineer	Emergency	509	CLM 25...

Figure 44. Figure 1's Visualizer

5.6.4. Navigators

SchemaNavigators represents **TypeSchemas** and relations between them in graphic way. Typically **SchemaNavigator** renders as UML like diagram. Rendering and interaction details may vary depending on particular **SchemaNavigator**.

Figure 45 shows PowerPlant's graph based SchemaNavigator's view.

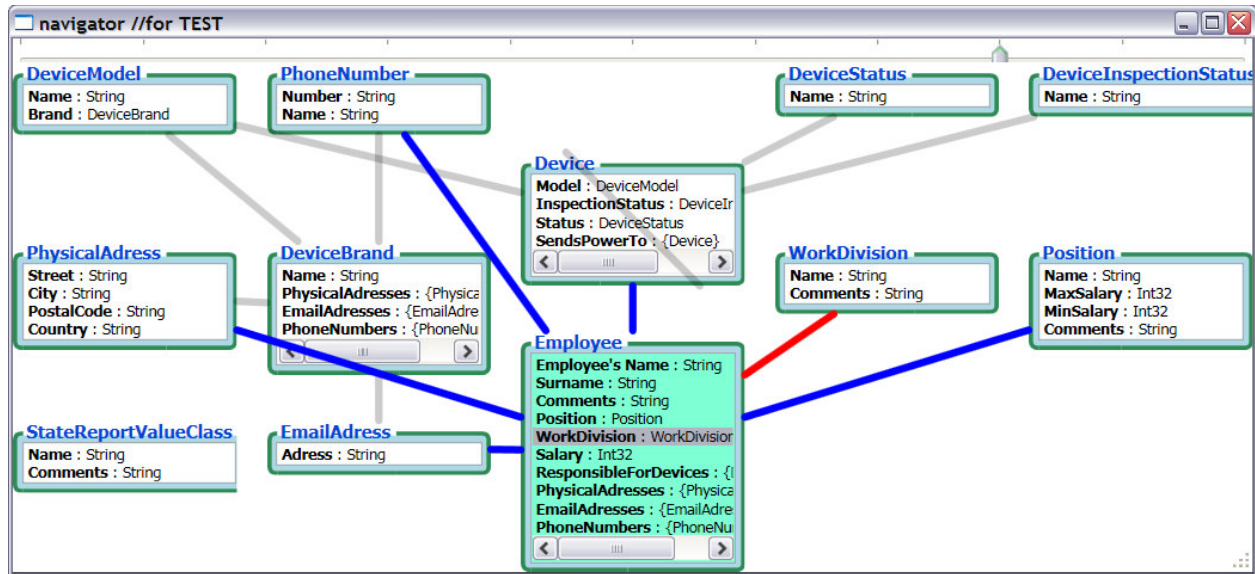


Figure 45. PowerPlant's SchemaNavigator's view

5.6.5. Session Item's Relations

SessionItemRelations represents relations between **SessionItems**. Nature of those relations is not defined but in current “zero” framework version **SessionItemRelations** are used only for master-detail relation and for domain object graph navigation purposes. Currently there are two basic navigation relations: intensional and extensional. For complete intensional and extensional description see [5].

5.6.5.1. Master-detail relation

Master-detail relation changes “detail” side **Visualizer** Object when “master” side **Visualizer** SelectedObject is changed by user.

5.6.5.2. Intensional navigation

In intensional navigation one operates on all objects of particular type. For example when navigating in PowerPlant's sample from Employee to PhysicalAddress, all PhysicalAddresses available in domain object graph will be shown.

5.6.5.3. *Extensional navigation*

In extensional navigation one operates on particular objects instances. For example (see figure 45) when navigating in PowerPlant's sample from Employee to PhysicalAddress, corresponding **SessionObjectCollectionItem** changes its content from collection of selected Employee objects to collection of PhysicalAddress objects that are in relation with those selected Employee objects.

5.7. The Session Manager

SessionManager semi UML diagram is shown on figure 46.

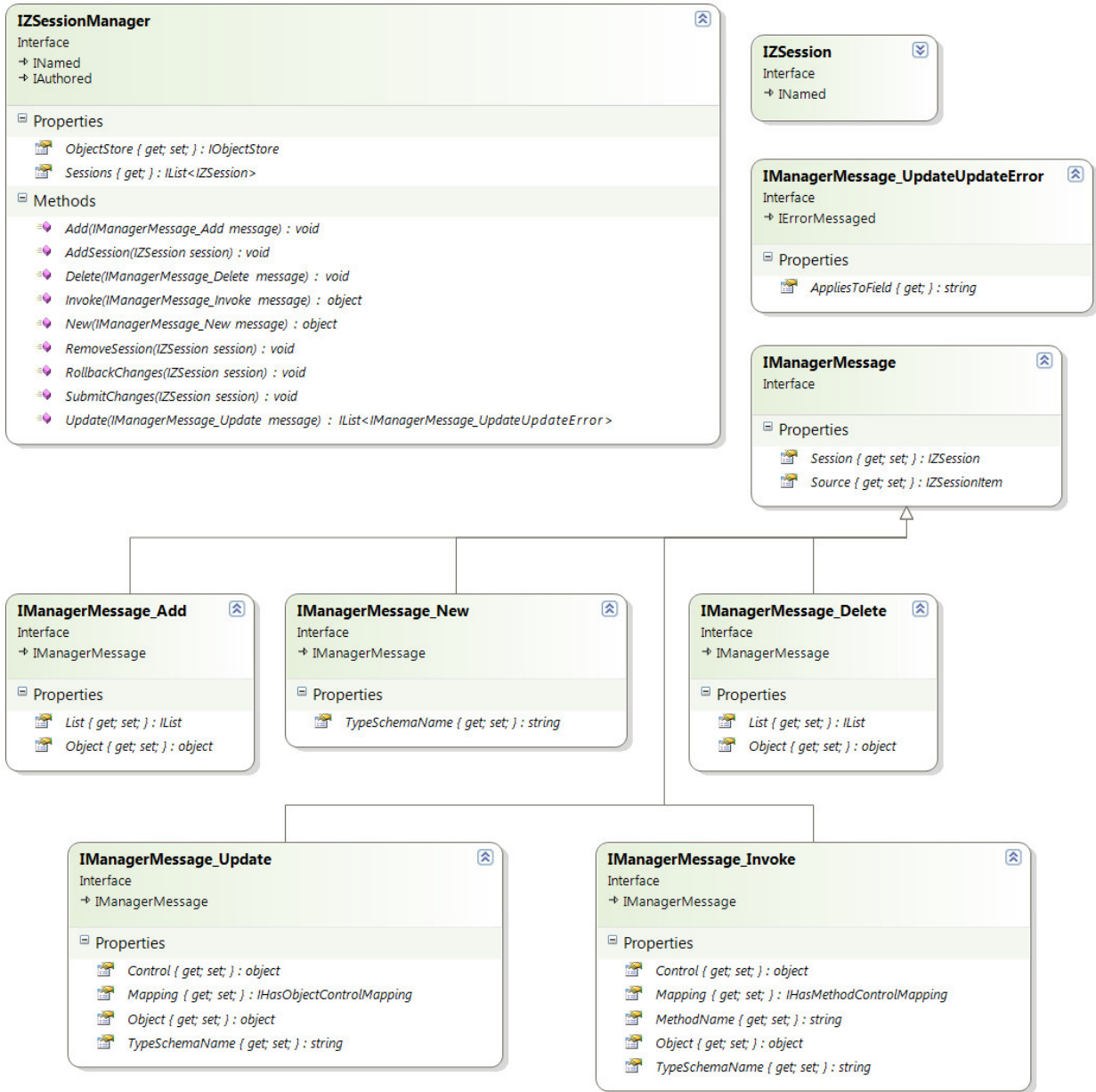


Figure 46. SessionManager semi UML diagram

SessionManager acts as an interface between domain object graph (accessed by **SessionManager** by **TypeSchema** methods) and **Session**. **SessionManager** handles tasks such as object graph store and object graph interaction.

Object graph interactions are realized by sending **ManagerMessages** objects. Those encapsulates all data needed to perform particular action. Those actions are:

- Add – adds object to existing collection
- New – creates new domain type instance
- Delete – deletes object
- Update – updates object's field's values
- Invoke – invoke particular method with particular parameters and returns method invocation result or an error message (**ManagerMessage_UpdateUpdateError**)

Store is realized as pluggable **ObjectStore** providers. Any particular store method is not defined. ObjectStore semi UML diagram is shown on figure 47.

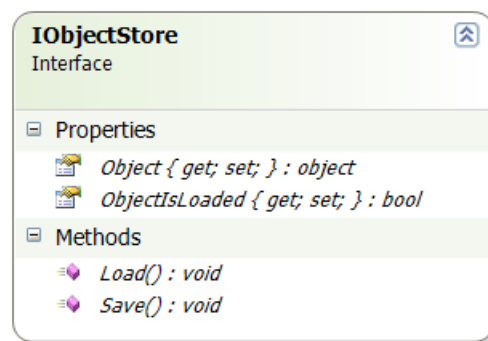


Figure 47. SessionManager semi UML diagram

Currently “zero” framework have **ObjectStore** providers for following store models:

- **LinqToSqlObjectStore**
Linq To Sql based store
- **MemoryNoStore**
In memory object graph without any store support
- **SimpleBinaryNetSerializerObjectStore**
Binary “.NET” serializer store

- **SimpleSoapNetSerializerObjectStore**

Soap (XML) “.NET” serializer store

6. Solution implementation

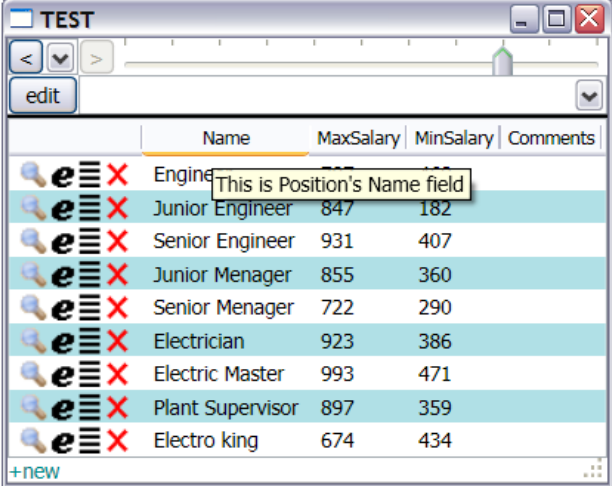
In this chapter we will present some implementation information. In general we will present some **ZAttributes** usage scenarios divided in three parts: common attributes (attributes that are common to all target UIs), ASP.NET specific attributes and WPF specific attributes.

For each attribute we will give sample code and its visual result.

6.1. Common attributes

6.1.1. Description

```
public class Position
{
    [Description(Text = "This is Position's Name field")]
    public string Name {get; set;}
    public int MaxSalary {get; set;}
    public int MinSalary {get; set;}
    public string Comments {get; set;}
}
```



	Name	MaxSalary	MinSalary	Comments
	Engineer	847	182	
	Junior Engineer	847	182	
	Senior Engineer	931	407	
	Junior Manager	855	360	
	Senior Manager	722	290	
	Electrician	923	386	
	Electric Master	993	471	
	Plant Supervisor	897	359	
	Electro king	674	434	

Figure 48. Description attribute visual result in *Bulk* view

TEST

edit

new delete update

Name Senior Menager

MaxSalary 722

MinSalary 290

Comments

This is Position's Name field

Figure 49. Description attribute visual result in *Detail* view

Lp	Name	MaxSalary	MinSalary	Comments	
1	Engineer	787	468		pokaż  
2	Junior Engineer	847	182		pokaż  
3	Senior Engineer	931	407		pokaż  
4	Junior Menager	855	360		pokaż  
5	Senior Menager	722	290		pokaż  
6	Electrician	923	386		pokaż  
7	Electric Master	993	471		pokaż  
8	Plant Supervisor	897	359		pokaż  
9	Electro king	674	434		pokaż  

dodaj nowy

This is Position's Name field

Figure 50. Description attribute visual result in *Bulk* view @ ASP.NET

|<< < Zapisz zmiany Usuń ten obiekt Stwórz nowy obiekt > >>|

Name Engineer

Max This is Position's Name field

MinSalary 468

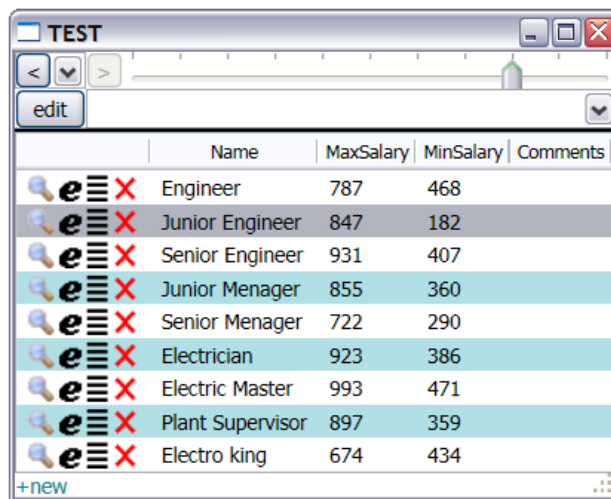
Comments

Figure 51. Description attribute visual result in *Detail* view @ ASP.NET

6.1.2. Ignore

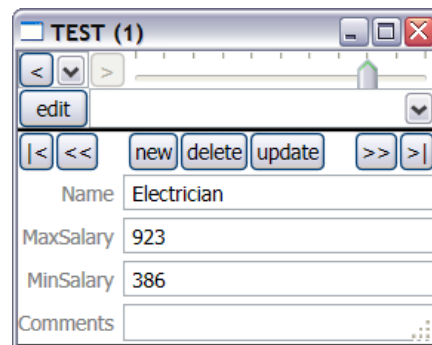
```
public class Position
{
    [Ignore]
    public int IgnoreMe { get; set; }

    public string Name {get; set;}
    public int MaxSalary {get; set;}
    public int MinSalary {get; set;}
    public string Comments {get; set;}
}
```



	Name	MaxSalary	MinSalary	Comments
 	Engineer	787	468	
 	Junior Engineer	847	182	
 	Senior Engineer	931	407	
 	Junior Menager	855	360	
 	Senior Menager	722	290	
 	Electrician	923	386	
 	Electric Master	993	471	
 	Plant Supervisor	897	359	
 	Electro king	674	434	

Figure 52. Ignore attribute visual result in *Bulk* view



TEST (1)	
edit	
< << new delete update >> >	
Name	Electrician
MaxSalary	923
MinSalary	386
Comments	

Figure 53. Ignore attribute visual result in *Detail* view

Lp	Name	MaxSalary	MinSalary	Comments	
1	Engineer	787	468		pokaż  
2	Junior Engineer	847	182		pokaż  
3	Senior Engineer	931	407		pokaż  
4	Junior Menager	855	360		pokaż  
5	Senior Menager	722	290		pokaż  
6	Electrician	923	386		pokaż  
7	Electric Master	993	471		pokaż  
8	Plant Supervisor	897	359		pokaż  
9	Electro king	674	434		pokaż  

[dodaj nowy](#)

Figure 54. Ignore attribute visual result in *Bulk* view @ ASP.NET

[|<<](#)
[<](#)
[Zapisz zmiany](#)
[Usuń ten obiekt](#)
[Stwórz nowy obiekt](#)
[>](#)
[>>|](#)

Name	Engineer
MaxSalary	787
MinSalary	468
Comments	

Figure 55. Ignore attribute visual result in *Detail* view @ ASP.NET

6.1.3. MethodsFirst

```
[MethodsFirst]
public class Employee: IContactable
```


TEST (1)

edit

|<<<

new

delete

update

>>>

SendMessage

message

run

IncreaseSalaryBy10Percent

run

SetSalaryToPositionMaxSalary

run

SetSalaryToPositionMinSalary

run

SetSalaryToWorkDivisionAvarage

run

Employee's Name

Jurek

Surname

Bieniak

Comments

Position

Junior Engineer

WorkDivision

Base services

Salary

463

ResponsibleForDevices

	Model	InspectionStatus	Status	SendsPowerTo
	NPX 42	Needs check	Running	FCG 81...
	LYR 71	Not passes - major failure	Running	YOJ 86...

+new

PhysicalAdresses

	Street	City	PostalCode	Country
	W szczerym polu 6/72	Płońsk	45-364	Poland

+new

EmailAdresses

	Adress
	name@host.net

+new

PhoneNumbers

	Number	Name
	(78)671865	
	(43)821438	

+new

Figure 56. MethodsFirst attribute visual result (compare with figure 1)

6.1.4. ObjectSourceCollection

```
public class DeviceModel
{
    public String Name { get; set; }

    [ObjectSourceCollection(Name="DeviceBrands")]
    public DeviceBrand Brand { get; set; }
}
```

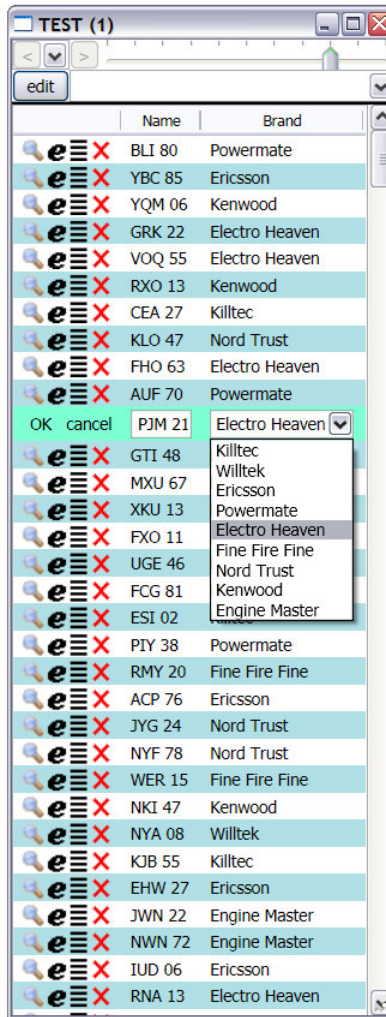


Figure 57. ObjectSourceCollection attribute visual result in *Bulk* view – drop down options are taken from DeviceBrands object

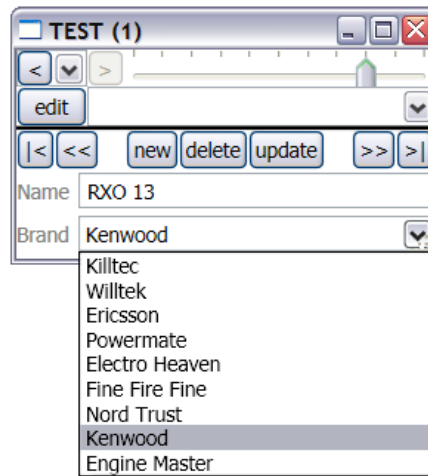


Figure 58. ObjectSourceCollection attribute visual result in *Detail* view – drop down options are taken from DeviceBrands object

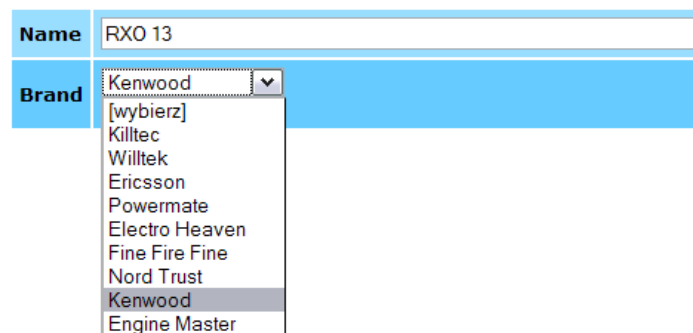


Figure 59. ObjectSourceCollection attribute visual result in *Detail* view @ ASP.NET – drop down options are taken from DeviceBrands object

6.1.5. TextMultiLine

```
public class StateReportValueClass
{
    public string Name {get;set;}

    [TextMultiLine(Lines="20")]

```

```

    public string Comments {get; set;}
}

```

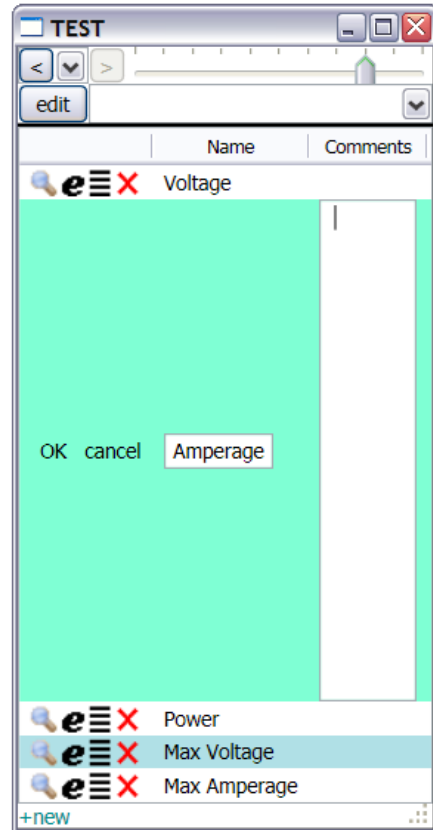


Figure 60. TextMultiLine attribute visual result in *Bulk* view – 20 lines for comments

TEST (1)

Navigation: < > << >> new delete update

Name: Power

Comments: [Large text area]

Figure 61. TextMultiLine attribute visual result in *Detail* view – 20 lines for comments

Lp	Name	Comments	
1	Voltage		pokaż e X
2	Amperage		pokaż e X
3	Power		Zapisz Anuluj
4	Max Voltage		pokaż e X
5	Max Amperage		pokaż e X

dodaj nowy

Figure 62. TextMultiLine attribute visual result in *Bulk* view @ ASP.NET – 20 lines for comments

|<<
<
Zapisz zmiany
Usuń ten obiekt
Stwórz nowy obiekt
>
>>|

Name	Power
Comments	

Figure 63. TextMultiLine attribute visual result in *Detail* view @ ASP.NET – 20 lines for comments

6.1.6. ValidateMethod

```

[ValidateMethod(Name="pjwtstkt.StateReport.Validate")]
public class StateReport
{
    [Zero.Ignore]
    public static List<IManagerMessage_UpdateUpdateError>
Validate(StateReport r)
    {
        List<IManagerMessage_UpdateUpdateError> list = new
List<IManagerMessage_UpdateUpdateError>();

        if(r.Comments.Length<10)
            list.Add(new
SimpleManagerMessage_UpdateUpdateError("Comment too short"));

        return list;
    }

    public string Name {get; set;}

    [TextMultiLine()]

```

```

    public string Comments {get; set;}
    public List<StateReportValue> Values {get; set;}
}

```

ValidateMethod attribute specifies method used for verification if updated object is proper one.

6.1.7. VisualGroupName

```

public class Employee:IContactable
{
    [VisualName (Name="Employee's Name")]
    public string Name {get; set;}
    public string Surname {get; set;}

    [TextMultiLine]
    public string Comments { get; set; }

    [VisualGroupName (Name = "Position and Work Division")]
    [ObjectSourceCollection (Name = "Positions")]
    public Position Position {get; set;}

    [VisualGroupName (Name = "Position and Work Division")]
    [ObjectSourceCollection (Name = "WorkDivisions")]
    public WorkDivision WorkDivision {get;set;}

    [VisualGroupName (Name="Salary")]
    public int Salary {get; set;}

    public List<Device> ResponsibleForDevices {get;set;}

    [VisualGroupName (Name = "Contact database")]
    public List<PhysicalAddress> PhysicalAdresses { get; set; }

    [VisualGroupName (Name = "Contact database")]
    public List<EmailAdress> EmailAdresses { get; set; }

    [VisualGroupName (Name = "Contact database")]
    public List<PhoneNumber> PhoneNumbers { get; set; }

    public void SendMessage(string message) {}

    [VisualGroupName (Name = "Salary")]
    public void IncreaseSalaryBy10Percent () {}
    [VisualGroupName (Name = "Salary")]
    public void SetSalaryToPositionMaxSalary () {}
    [VisualGroupName (Name = "Salary")]
    public void SetSalaryToPositionMinSalary () {}
    [VisualGroupName (Name = "Salary")]
    public void SetSalaryToWorkDivisionAvarage () {}
}

```

TEST (1)

edit

<<

<

>

>>

new

delete

update

Employee's Name

Franek

Surname

Dąbrowski

Comments

ResponsibleForDevices

	Model	InspectionStatus	Status	SendsPowerTo
	TMM 05	Not passes - major failure	Running	ADM 43...
	YLN 72	Not passed - minor failure	Emergency alarm	OUQ 40...
	YBC 85	Passed	Running	NIQ 62...
	KLO 47	Needs check	Emergency alarm	DKB 15...
	PQR 31	Not passed - minor failure	Emergency alarm	CKQ 20...
	PQJ 03	Not passes - major failure	Failure	OQJ 26...
	PVL 20	Not passes - major failure	Stopped	PCN 28...
	RWF 60	Needs check	Failure	YLN 72...
	FYV 52	Not passed - minor failure	Failure	BLI 80...
	IJI 25	Not passed - minor failure	Stopped	LUG 00...
	BHM 60	Passed	Failure	CHX 03...
	JQD 86	Needs check	Running	XHA 16...
	DPQ 22	Needs check	Emergency alarm	TMC 31...
	IPE 47	Not passes - major failure	Emergency alarm	OEK 35...
	NBH 37	Not passed - minor failure	Stopped	JJF 50...
	AUF 70	Not passes - major failure	Stopped	AJA 88...
	YQM 06	Not passes - major failure	Running	LUG 00...

+new

SendMessage

message

run

Salary

Salary 596

IncreaseSalaryBy10Percent

run

SetSalaryToPositionMaxSalary

run

SetSalaryToPositionMinSalary

run

SetSalaryToWorkDivisionAvarage

run

Position and Work Division

Position Electro king

WorkDivision Power management

Contact database

PhysicalAddresses

	Street	City	PostalCode	Country
	Gorzka 8/3	Warszawa	28-781	Poland
	Bombardierów 2/101	Warszawa	53-216	Poland

+new

EmailAddresses

	Adress
	name@host.net
	name@host.net
	name@host.net
	name@host.net

+new

PhoneNumbers

	Number	Name
	(42)042758	
	508071224	
	502585552	

+new

Figure 64. VisualGroupName attribute visual result in *Detail* view

|<< < Zapisz zmiany Usuń ten obiekt Stwórz nowy obiekt > >>|

SendMessage	message wykonaj
Employee's Name	Wiesiek
Surname	Cygan
Comments	
ResponsibleForDevices	• CHP 44 ✖ • TWI 84 ✖ • NIQ 62 ✖ • XOF 17 ✖ • JFI 44 ✖ • FLU 44 ✖ • RNA 13 ✖ • UUP 44 ✖ • HCA 33 ✖ • JHL 05 ✖ dodaj nowy

Salary	
IncreaseSalaryBy10Percent	wykonaj
SetSalaryToPositionMaxSalary	wykonaj
SetSalaryToPositionMinSalary	wykonaj
SetSalaryToWorkDivisionAvarage	wykonaj
Salary	837

Position and Work Division	
Position	Electrician ▼ nawiguj
WorkDivision	Service ▼ nawiguj

Contact database	
PhysicalAddresses	• Wielkomiejaska 5/86 22-551 Zakopane Poland ✖ • Naczelnika 8/35 48-581 Zakopane Poland ✖ • raz ✖ dodaj nowy
EmailAddresses	• name@host.net ✖ • name@host.net ✖ • name@host.net ✖ dodaj nowy
PhoneNumbers	• (83)387731 ✖ • 505831781 ✖ dodaj nowy

Figure 65. VisualGroupName attribute visual result in *Detail* view @ ASP.NET

6.1.8. VisualName

```
public class WorkDivision
{
    [VisualName(Name="Work devision name")]
    public string Name {get;set;}
    public string Comments {get; set;}
}
```

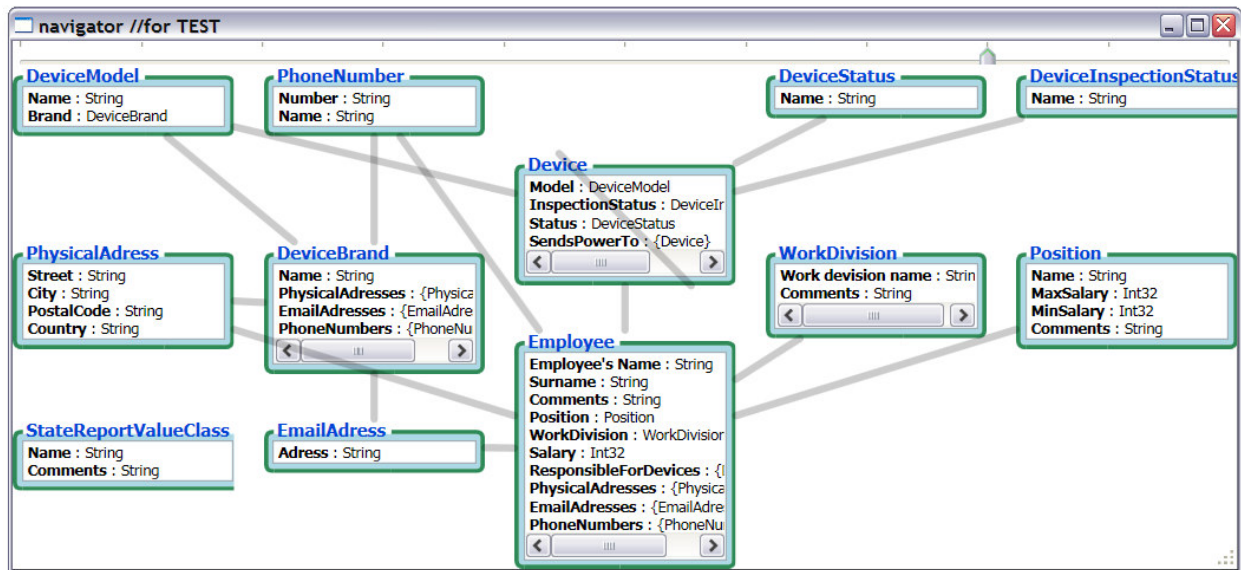


Figure 66. VisualName attribute visual result in Navigator

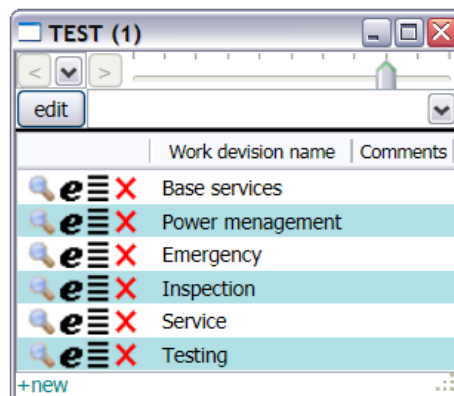


Figure 67. VisualName attribute visual result in Bulk view

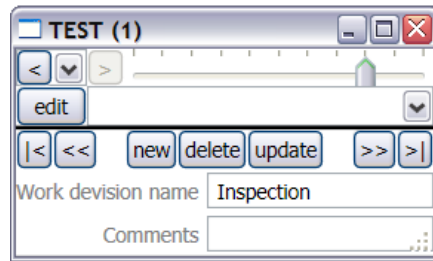


Figure 68. VisualName attribute visual result in *Detail* view

Lp	Work devision name	Comments	
1	Base services		pokaż  
2	Power menagement		pokaż  
3	Emergency		pokaż  
4	Inspection		pokaż  
5	Service		pokaż  
6	Testing		pokaż  

[dodaj nowy](#)

Figure 69. VisualName attribute visual result in *Bulk* view @ ASP.NET

[<<](#)
[<](#)
[Zapisz zmiany](#)
[Usuń ten obiekt](#)
[Stwórz nowy obiekt](#)
[>](#)
[>>](#>>)

Work devision name	Base services
Comments	

Figure 70. VisualName attribute visual result in *Detail* view @ ASP.NET

6.2. ASP.NET specific attributes

6.2.1. UseControl

Zero.UIWeb.UseControl attribute forces using particular control for fields Web UI visual representation.

6.2.2. UseGenerator

Zero.UIWeb.UseGenerator attribute forces using particular generator when targeting Web UI.

6.2.3. UseVisualizerItem

Zero.UIWeb.UseVisualizerItem attribute forces using particular VisualizerItem name when targeting Web UI.

6.3. WPF specific attributes

6.3.1. BulkColors

```
[Zero.UIWpf.BulkColors(AlternatingBackground="White",NormalBackground="Gray",EditBackground="Coral",SelectedBackground="Pink")]  
public class Employee:IContactable
```

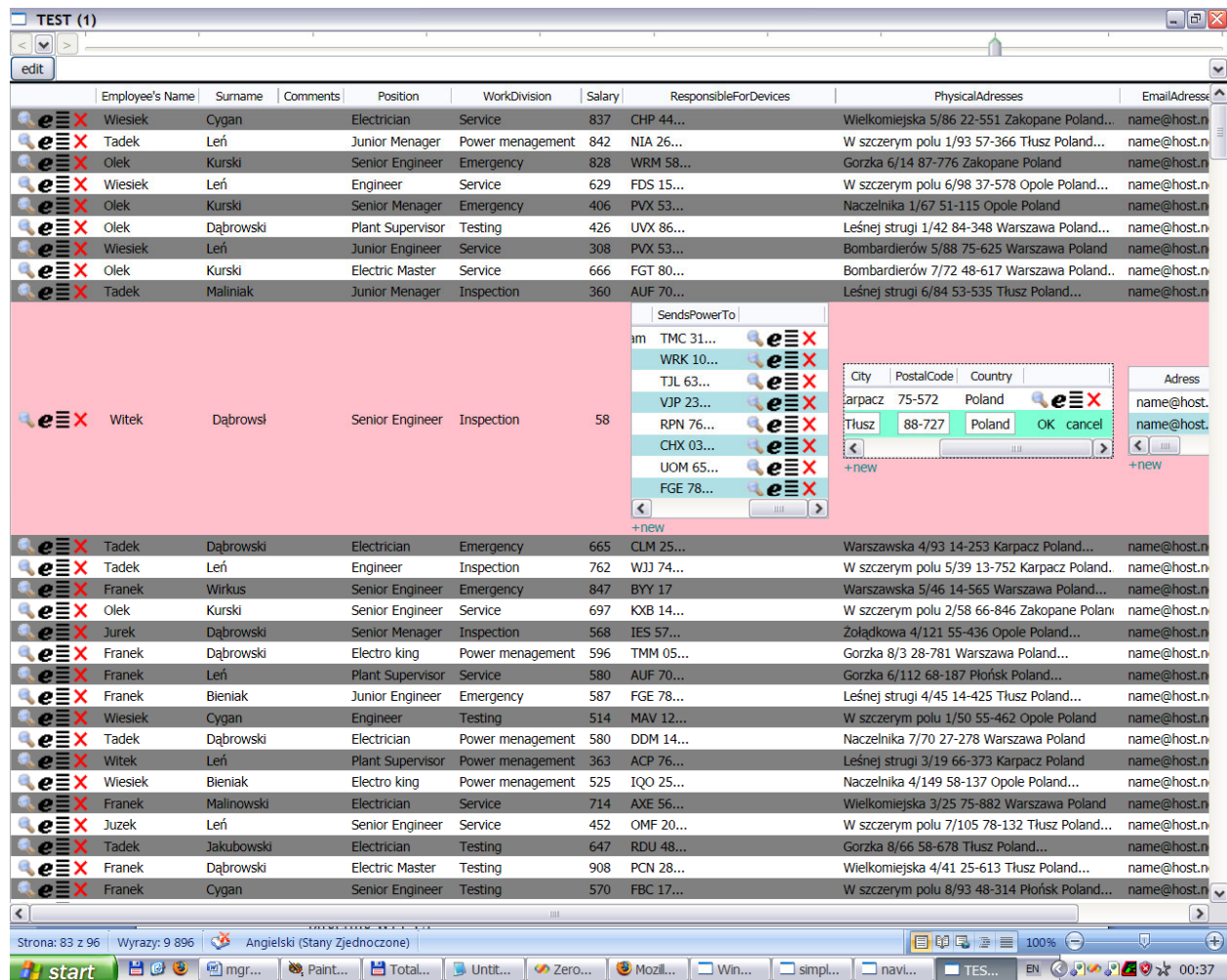


Figure 71. BulkColors attribute visual result

6.3.2. ControlParameter

```
public class DeviceStatus
{
    [ControlParameter("Text", "THIS FIELD HAS BEEN HACKED
:0)", "Detail", true)]
    [ControlParameter("Background", "Red")]
    [ControlParameter("Foreground", "White")]
    public string Name { get; set; }
}
```

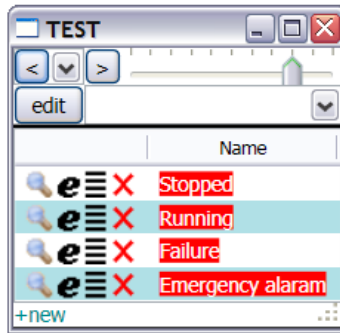


Figure 72. ControlParameter attribute visual result in *Bulk* view



Figure 73. ControlParameter attribute visual result in *Detail* view

6.3.3. GeneratorAttributes

```
public class DeviceBrand: IContactable
{
    public String Name { get; set; }

    [GeneratorAttributes (CopyAll=true)]
    [BulkControlItems (Details = false, Edit = false, Target =
    "Detail")]
    [BulkControlItems (ControlItemsOnRight = true)]
    [BulkColors (AlternatingBackground="LightSeaGreen", NormalBackgr
    ound="LightSteelBlue")]
    public List<PhysicalAdress> PhysicalAdresses { get; set; }

    public List<EmailAddress> EmailAdresses { get; set; }
    public List<PhoneNumber> PhoneNumbers { get; set; }
}
```

TEST (1)

<

>

edit

	Name	PhysicalAdresses	EmailAdresses	PhoneNumbers																							
	Killtec	Leśnej strugi 4/106 32-812 Zakopane Poland...	name@host.net...	(17)338463...																							
	Willtek	Wielkomiejska 2/80 26-184 Tłusz Poland...	name@host.net...	(20)245328																							
	Ericsson	Wielkomiejska 4/35 47-845 Karpacz Poland	name@host.net	(47)046551																							
	Powermate	Wielkomiejska 3/86 43-658 Warszawa Poland	name@host.net...	(87)568258...																							
	Electro Heav	<table> <tr> <th>Street</th> <th>City</th> <th>PostalCode</th> <th>Country</th> </tr> <tr> <td>Bombardierów 2/115</td> <td>Opole</td> <td>76-168</td> <td>Poland</td> </tr> <tr> <td>Naczelnika 8/74</td> <td>Opole</td> <td>12-648</td> <td>Poland</td> </tr> <tr> <td>Warszawska 7/45</td> <td>Karpacz</td> <td>74-823</td> <td>Poland</td> </tr> </table>	Street	City	PostalCode	Country	Bombardierów 2/115	Opole	76-168	Poland	Naczelnika 8/74	Opole	12-648	Poland	Warszawska 7/45	Karpacz	74-823	Poland	<table> <tr><th>Adress</th></tr> <tr><td>name@host.net</td></tr> <tr><td>name@host.net</td></tr> <tr><td>name@host.net</td></tr> <tr><td>name@host.net</td></tr> </table>	Adress	name@host.net	name@host.net	name@host.net	name@host.net	<table> <tr><th>Number</th></tr> <tr><td>503613784</td></tr> </table>	Number	503613784
Street	City	PostalCode	Country																								
Bombardierów 2/115	Opole	76-168	Poland																								
Naczelnika 8/74	Opole	12-648	Poland																								
Warszawska 7/45	Karpacz	74-823	Poland																								
Adress																											
name@host.net																											
name@host.net																											
name@host.net																											
name@host.net																											
Number																											
503613784																											
		+new																									
		+new																									
	Fine Fire Fine	Gorzka 6/97 74-174 Zakopane Poland...	name@host.net	(16)361425...																							
	Nord Trust	Żołądkowa 3/84 64-518 Płońsk Poland	name@host.net	(14)684788...																							
	Kenwood	Wielkomiejska 7/63 73-367 Płońsk Poland	name@host.net...	507080302...																							
	Engine Master	W szczerym polu 6/60 38-185 Zakopane Poland...	name@host.net	(61)632251																							
+new																											

Figure 74. GeneratorAttributes attribute visual result in *Bulk* view

TEST (1)																				
edit																				
< << new delete update >> >																				
Name Electro Heaven																				
PhysicalAdresses	<table> <tr> <th>Street</th><th>City</th><th>PostalCode</th><th>Country</th></tr> <tr> <td>Bombardierów 2/115</td><td>Opole</td><td>76-168</td><td>Poland</td></tr> <tr> <td>Naczelnika 8/74</td><td>Opole</td><td>12-648</td><td>Poland</td></tr> <tr> <td>Warszawska 7/45</td><td>Karpacz</td><td>74-823</td><td>Poland</td></tr> </table>				Street	City	PostalCode	Country	Bombardierów 2/115	Opole	76-168	Poland	Naczelnika 8/74	Opole	12-648	Poland	Warszawska 7/45	Karpacz	74-823	Poland
Street	City	PostalCode	Country																	
Bombardierów 2/115	Opole	76-168	Poland																	
Naczelnika 8/74	Opole	12-648	Poland																	
Warszawska 7/45	Karpacz	74-823	Poland																	
+new																				
<table> <tr><th>Adress</th></tr> <tr><td>name@host.net</td></tr> <tr><td>name@host.net</td></tr> <tr><td>name@host.net</td></tr> <tr><td>name@host.net</td></tr> </table>				Adress	name@host.net	name@host.net	name@host.net	name@host.net												
Adress																				
name@host.net																				
name@host.net																				
name@host.net																				
name@host.net																				
+new																				
PhoneNumbers	<table> <tr> <th>Number</th><th>Name</th></tr> <tr> <td>503613784</td><td></td></tr> </table>				Number	Name	503613784													
Number	Name																			
503613784																				
+new																				

Figure 75. GeneratorAttributes attribute visual result in *Detail* view

6.3.4. LabelParameter

```
public class DeviceStatus
{
    [LabelParameter("Background", "Red")]
    [LabelParameter("Foreground", "White", true)]
    public string Name { get; set; }
}
```

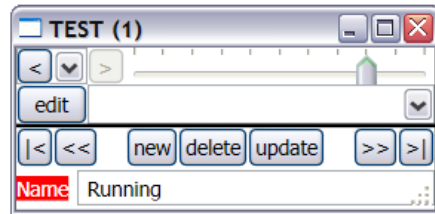


Figure 76. LabelParameter attribute visual result

```
public void SendMessage(
    [LabelParameter("Background", "Red")]
    [LabelParameter("Foreground", "White", true)]
    [ControlParameter("Background", "Green")]
    [ControlParameter("Foreground", "White")]
    string message) {
}
```


TEST (1)

edit

<

<<

new

delete

update

>>

>

Work

Employee's Name

Witek

Surname

Dąbrowski

Comments

ResponsibleForDevices

	Model	InspectionStatus	Status	SendsPowerTo
	DPQ 22	Needs check	Emergency alarm	TMC 31...
	PJM 21	Not passed - minor failure	Failure	WRK 10...
	GII 57	Passed	Failure	TJL 63...
	PIY 38	Not passed - minor failure	Failure	VJP 23...
	ORH 56	Passed	Running	RPN 76...
	BHM 60	Passed	Failure	CHX 03...
	QKH 42	Not passed - minor failure	Failure	UOM 65...
	QFQ 04	Not passed - minor failure	Running	FGE 78...

+new

SendMessage

message

SOME TEXT THAT WILL BE SENT

run

Contact database

PhysicalAddresses

	Street	City	PostalCode	Country
	Leśnej strugi 8/59	Karpacz	75-572	Poland
	Żołądkowa 8/102	Tłuszcz	88-727	Poland

+new

EmailAddresses

	Adress
	name@host.net
	name@host.net

+new

PhoneNumbers

	Number	Name
	(65)328575	
	(37)355822	

+new

Figure 77. LabelParameter attribute used on method parameter visual result

6.3.5. MethodControlParameter

```
[MethodControlParameter("Background", "LightSkyBlue")]  
public class Device
```

The screenshot shows a software application window titled 'TEST'. It features a toolbar with navigation and action buttons (new, delete, update, etc.). Below the toolbar, there are three dropdown menus for 'Model' (SQP 72), 'InspectionStatus' (Passed), and 'Status' (Emergency alaram). A table displays a list of devices with columns for Model, InspectionStatus, Status, and SendsPowerTo. The table contains 10 rows of data, each with a magnifying glass icon and a red 'X' in the first column. Below the table, there is a '+new' button and four buttons labeled 'run' corresponding to the actions: GetResponsibleEngineers, StartDevice, StopDevice, and GetStateReport.

	Model	InspectionStatus	Status	SendsPowerTo
 	NMS 71	Needs check	Emergency alaram	KQN 64...
 	IQO 25	Needs check	Failure	YQM 06...
 	FLU 44	Not passed - minor failure	Stopped	PWP 47...
 	EYY 86	Passed	Running	RMY 20...
 	QVR 72	Not passed - minor failure	Running	CBG 24...
 	RUK 48	Needs check	Running	FCG 81...
 	UTC 68	Not passes - major failure	Running	RWQ 23...
 	SEN 14	Passed	Stopped	TXC 52...
 	NYA 08	Passed	Stopped	CBG 24...
 	APW 65	Not passes - major failure	Stopped	XQC 47...

+new

GetResponsibleEngineers
run

StartDevice
run

StopDevice
run

GetStateReport
run

Figure 78. MethodControlParameter attribute visual result

```
[MethodControlParameter("Background", "LightSteelBlue")]
public IList<Employee> GetResponsibleEngineers() {}
```

TEST

edit

Model: MRD 80

InspectionStatus: Needs check

Status: Failure

	Model	InspectionStatus	Status	SendsPowerTo
  	XTK 74	Not passed - minor failure	Failure	NWM 84...
  	IUD 06	Passed	Emergency alarm	RHH 55...
  	TYF 14	Not passed - minor failure	Failure	AWL 40...
  	AQD 67	Not passed - minor failure	Failure	YYY 41...
  	MYF 25	Passed	Running	DVT 12...
  	DXJ 46	Not passed - minor failure	Failure	CTQ 25...
  	CEA 27	Not passed - minor failure	Stopped	FFX 42...
  	XLL 73	Not passes - major failure	Emergency alarm	TMM 05...
  	PVL 20	Passed	Emergency alarm	YQM 06
  	SKD 54	Not passed - minor failure	Emergency alarm	XOF 17...
  	YXE 85	Not passed - minor failure	Failure	XVS 86...
  	CBG 24	Passed	Emergency alarm	DVT 12...
  	VAT 04	Not passed - minor failure	Running	IVY 40...

+new

GetResponsibleEngineers

run

StartDevice

run

StopDevice

run

GetStateReport

run

Figure 79. MethodControlParameter attribute visual result

6.3.6. MethodUseControl

```
[MethodUseControl(Name = "Expander")]
public class Device
{
    [ObjectSourceCollection(Name = "DeviceModels")]
    public DeviceModel Model { get; set; }

    [ObjectSourceCollection(Name = "DeviceInspectionStatuses")]
    public DeviceInspectionStatus InspectionStatus { get; set; }

    [ObjectSourceCollection(Name = "DeviceStatuses")]
    public DeviceStatus Status { get; set; }

    public List<Device> SendsPowerTo { get; set; }

    [ControlParameter("Background", "LightSteelBlue")]
    public IList<Employee> GetResponsibleEngineers() {}

    [MethodUseControl(Name="GroupBox")]
    public void StartDevice() {}

    [ControlParameter("IsExpanded", "True")]
    public void StopDevice() {}

    public StateReport GetStateReport() {}
}
```

TEST

edit

<

>

<<

>>

new

delete

update

Model

RJX 20

InspectionStatus

Passed

Status

Running

	Model	InspectionStatus	Status	SendsPowerTo
	LRA 71	Passed	Failure	WJJ 74...
	FID 31	Needs check	Running	ANM 64...
	EWQ 86	Not passed - minor failure	Running	DCN 44...
	NBG 72	Not passes - major failure	Emergency alarm	DAJ 40...
	KQN 64	Not passed - minor failure	Running	FFQ 54...
	MXU 67	Needs check	Stopped	WQI 06...
	PXA 58	Passed	Emergency alarm	SKC 01...
	CBG 24	Not passes - major failure	Running	GII 57...
	DKA 32	Not passed - minor failure	Failure	RWF 60...
	NWM 84	Not passed - minor failure	Stopped	PVX 53...
	LEL 04	Passed	Stopped	XHA 16...
	QDS 71	Not passed - minor failure	Running	PWP 47...
	EOE 50	Not passes - major failure	Stopped	FAT 14...
	WRM 58	Passed	Stopped	IDC 87...
	PVL 20	Not passes - major failure	Stopped	PCN 28...
	LYR 71	Passed	Running	QDS 71...
	YLN 72	Not passed - minor failure	Emergency alarm	OUQ 40...
	YXE 85	Not passed - minor failure	Failure	XVS 86...
	YOJ 86	Not passes - major failure	Running	KLR 54...

+new

GetResponsibleEngineers

StartDevice

run

StopDevice

run

GetStateReport

Figure 80.. MethodUseControl attribute visual result

6.3.7. TabName

```
public class Employee: IContactable
{
    [VisualName(Name="Employee's Name")]
    public string Name {get; set;}
    public string Surname {get; set;}

    [TextMultiLine]
    public string Comments { get; set; }

    [TabName(Name="Work")]
    [VisualGroupName(Name = "Position and Work Division")]
    [ObjectSourceCollection(Name = "Positions")]
    public Position Position {get; set;}

    [TabName(Name = "Work")]
    [VisualGroupName(Name = "Position and Work Division")]
    [ObjectSourceCollection(Name = "WorkDivisions")]
    public WorkDivision WorkDivision {get;set;}

    [TabName(Name = "Work")]
    [VisualGroupName(Name="Salary")]
    public int Salary {get; set;}

    public List<Device> ResponsibleForDevices {get;set;}

    [VisualGroupName(Name = "Contact database")]
    public List<PhysicalAdress> PhysicalAdresses { get; set; }

    [VisualGroupName(Name = "Contact database")]
    public List<EmailAddress> EmailAdresses { get; set; }

    [VisualGroupName(Name = "Contact database")]
    public List<PhoneNumber> PhoneNumbers { get; set; }

    public void SendMessage(string message) {}

    [TabName(Name = "Work")]
    [VisualGroupName(Name = "Salary")]
    public void IncreaseSalaryBy10Percent() {}

    [TabName(Name = "Work")]
    [VisualGroupName(Name = "Salary")]
    public void SetSalaryToPositionMaxSalary() {}

    [TabName(Name = "Work")]
    [VisualGroupName(Name = "Salary")]
    public void SetSalaryToPositionMinSalary() {}
}
```

```

[TabName (Name = "Work")]
[VisualGroupName (Name = "Salary")]
public void SetSalaryToWorkDivisionAvarage() {}
}

```

TEST (2)

edit

Work

Employee's Name: Tadek

Surname: Dąbrowski

Comments:

ResponsibleForDevices:

	Model	InspectionStatus	Status	SendsPowerTo
  	CLM 25	Not passed - minor failure	Stopped	TWI 84...
  	PWP 47	Not passes - major failure	Emergency alaram	HRK 07...
  	YLN 72	Not passed - minor failure	Emergency alaram	OUQ 40...
  	ORH 56	Needs check	Emergency alaram	UTC 68
  	LXP 10	Needs check	Running	LCJ 00...

+new

SendMessage

message:

run

Contact database

PhysicalAdresses:

	Street	City	PostalCode	Country
  	Warszawska 4/93	Karpacz	14-253	Poland
  	Naczelnika 4/105	Zakopane	65-333	Poland
  	Naczelnika 6/62	Warszawa	21-224	Poland
  	W szczerym polu 7/120	Płońsk	72-433	Poland

+new

EmailAdresses:

	Adress
  	name@host.net
  	name@host.net
  	name@host.net

+new

PhoneNumbers:

	Number	Name
  	(52)807321	
  	507115136	
  	504805870	

+new

Figure 81. TabName attribute visual result – first tab

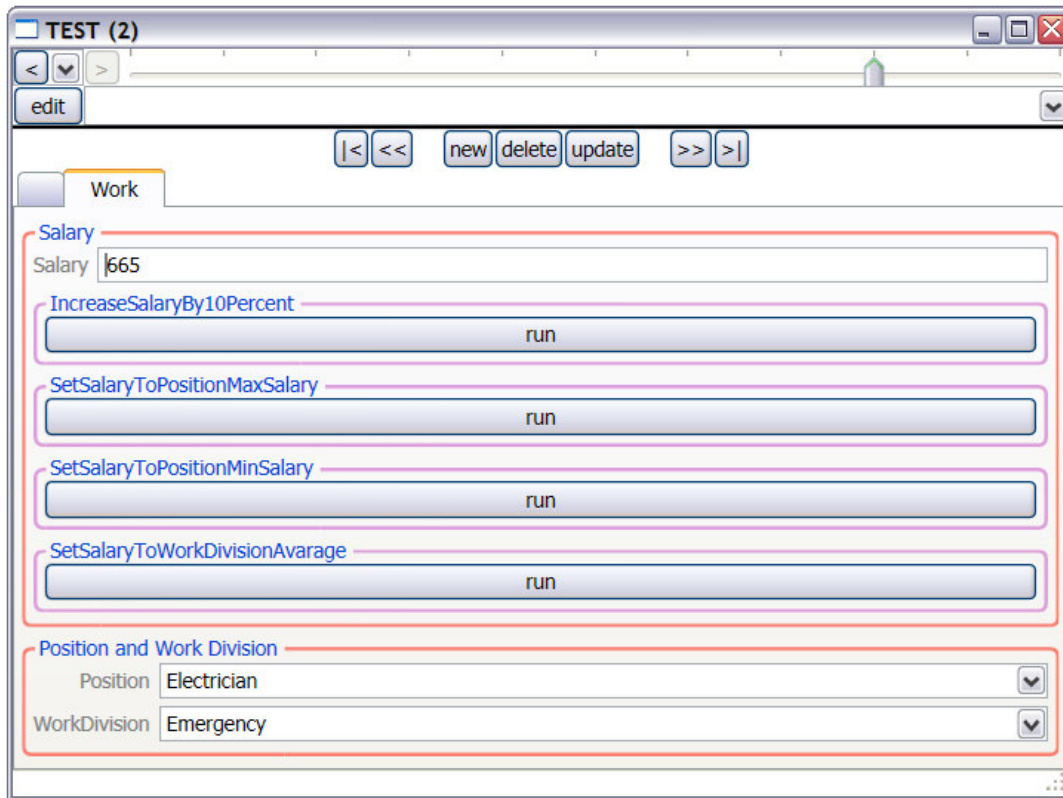


Figure 81. TabName attribute visual result – “Work” tab

6.3.8. TabParameter

```
[TabParameter("Salary","Background","LightSteelBlue")]
public class Position
{
    public string Name {get; set;}

    [TabName(Name = "Salary")]
    public int MaxSalary {get; set;}

    [TabName(Name = "Salary")]
    public int MinSalary {get; set;}
    public string Comments {get; set;}
}
```

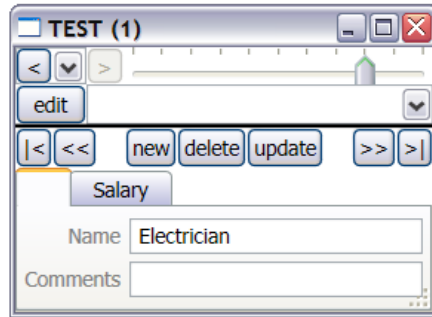



Figure 82. TabParameter attribute visual result (no results to this tab)

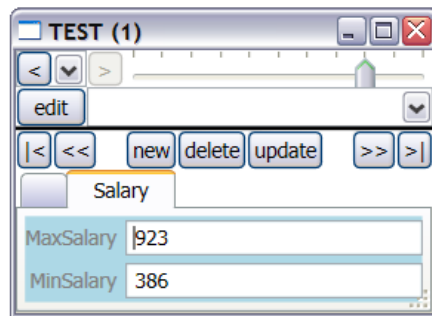


Figure 83. TabParameter attribute visual result

6.3.9. UseControl

Zero.UIWpf.UseControl attribute forces using particular control for fields WPF UI visual representation.

```
public class Position
{
    public string Name {get; set;}

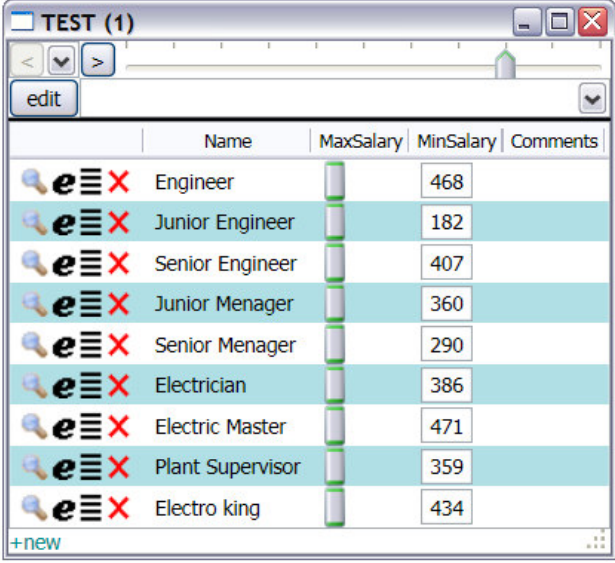
    [UseControl(Name="Slider", ControlsDataPropertyPath="Value")]
    public int MaxSalary {get; set;}

    [UseControl(Name = "Slider", ControlsDataPropertyPath =
"Value")]
    [UseControl(Name = "TextBox", ControlsDataPropertyPath="Text",
Target="Bulk")]
}
```

```

public int MinSalary {get; set;}
public string Comments {get; set;}
}

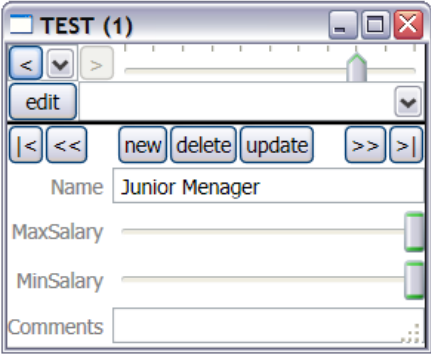
```



	Name	MaxSalary	MinSalary	Comments
 	Engineer		468	
 	Junior Engineer		182	
 	Senior Engineer		407	
 	Junior Menager		360	
 	Senior Menager		290	
 	Electrician		386	
 	Electric Master		471	
 	Plant Supervisor		359	
 	Electro king		434	

+new

Figure 84. UseControl attribute visual result in *Bulk* view



TEST (1)

< >

edit

|< << new delete update >> >|

Name Junior Menager

MaxSalary 

MinSalary 

Comments

Figure 85. UseControl attribute visual result in *Detail* view

6.3.10. UseGenerator

Zero.UIWpf.UseGenerator attribute forces using particular generator when targeting WPF UI.

```
[UseGenerator(Name="Simple")]
public class Position
{
    public string Name {get; set;}

    [TabName(Name = "Salary")]
    public int MaxSalary {get; set;}

    [TabName(Name = "Salary")]
    public int MinSalary {get; set;}
    public string Comments {get; set;}
}
```

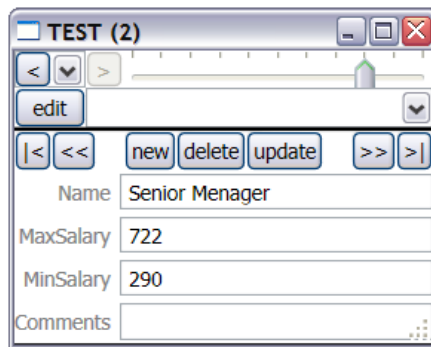


Figure 86. UseGenerator attribute visual result. “Simple” generator does not support “TabName” attribute - no tabs are created.

6.3.11. UseVisualizerItem

Zero.UIWpf.UseVisualizerItem attribute forces using particular VisualizerItem name when targeting WPF UI.

```
public class StateReport
{
    public string Name {get; set;}
}
```

```

[TextMultiLine()]
public string Comments {get; set;}

[UseVisualizerItem(Name =
"System.Collections.Generic.List`1[[pjwstk.StateReportValue,
PowerPlant, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null]],
mscorlib, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089|SimpleEnumerable|{Zero.UIWpf.BulkColor
s{AlternatingBackground='LightSeaGreen',NormalBackground='LightSteelBl
ue'},Zero.BulkControlItem{ControlItemOnRight='True'}}")]
public List<StateReportValue> Values {get; set;}
}

```

TEST

edit

update

Name: State report for 60112688

Comments: This is Fine Fire Fine's DXJ 46. Device is Failure and its inspection status is Not passed - minor failure device sends power to 16 devices. Device has 10 responsible engineers.

Class	Value			
Max Voltage	86		e	X
Max Voltage	19		e	X
Power	57		e	X
Max Amperage	59		e	X
Amperage	51		e	X
Amperage	55		e	X
Max Voltage	86		e	X
Voltage	41		e	X
Amperage	55		e	X

+new

Figure 87. UseVisualizerItem attribute visual result

```

public class Device
{
    [UseVisualizerItem]
    public DeviceModel Model {get; set;}

    [ObjectSourceCollection(Name = "DeviceInspectionStatuses")]
    public DeviceInspectionStatus InspectionStatus { get; set; }

    [ObjectSourceCollection(Name = "DeviceStatuses")]

```

```

public DeviceStatus Status { get; set; }

public List<Device> SendsPowerTo { get; set; }

...
}

```

TEST (1)

edit

|< << new delete update >> >|

Name: GRB 51

Model:

Brand: Ericsson

InspectionStatus: Passed

Status: Running

	Model	InspectionStatus	Status	SendsPowerTo
	EYY 86	Not passed - minor failure	Failure	OQJ 26...
	FID 31	Needs check	Running	ANM 64...
	FGE 78	Needs check	Failure	NSP 17...
	DKB 15	Not passed - minor failure	Running	ANM 64...
	EYY 86	Passed	Running	RMV 20...
	TWI 84	Needs check	Failure	PQJ 03...
	WDO 62	Not passes - major failure	Failure	HRB 71...
	WRM 58	Not passes - major failure	Emergency alarm	FYV 52...
	KOS 77	Needs check	Emergency alarm	DKA 32...
	ANM 64	Not passed - minor failure	Running	IUD 06...
	MRD 80	Needs check	Failure	XTK 74...
	RUA 85	Passed	Emergency alarm	SKD 54...
	ANM 64	Not passed - minor failure	Running	IUD 06...
	XVS 86	Needs check	Running	BFO 25...
	DRS 43	Not passed - minor failure	Stopped	FLU 44...
	AJA 88	Not passed - minor failure	Stopped	CPF 13
	RNR 83	Not passes - major failure	Running	FYV 52...

+new

GetResponsibleEngineers

run

StartDevice

run

StopDevice

run

GetStateReport

run

Figure 88. UseVisualizerItem attribute visual result

6.3.12. VerticalMargin

```
[VerticalMargin(Size="10")]
public class Device
{
    [ObjectSourceCollection(Name="DeviceModels")]
    public DeviceModel Model {get;set;}

    [ObjectSourceCollection(Name = "DeviceInspectionStatuses")]
    public DeviceInspectionStatus InspectionStatus { get; set; }

    [ObjectSourceCollection(Name = "DeviceStatuses")]
    public DeviceStatus Status { get; set; }

    public List<Device> SendsPowerTo { get; set; }

    public IList<Employee> GetResponsibleEngineers() {}

    public void StartDevice() {}

    public void StopDevice() {}

    public StateReport GetStateReport() {}
}
```

TEST

edit

<

>

new

delete

update

>>

>|

Model

SQP 72

InspectionStatus

Passed

Status

Emergency alaram

SendsPowerTo

	Model	InspectionStatus	Status	SendsPowerTo
	NMS 71	Needs check	Emergency alaram	KQN 64...
	IQO 25	Needs check	Failure	YQM 06...
	FLU 44	Not passed - minor failure	Stopped	PWP 47...
	EYY 86	Passed	Running	RMY 20...
	QVR 72	Not passed - minor failure	Running	CBG 24...
	RUK 48	Needs check	Running	FCG 81...
	UTC 68	Not passes - major failure	Running	RWQ 23...
	SEN 14	Passed	Stopped	TXC 52...
	NYA 08	Passed	Stopped	CBG 24...
	APW 65	Not passes - major failure	Stopped	XQC 47...

+new

GetResponsibleEngineers

run

StartDevice

run

StopDevice

run

GetStateReport

run

Figure 89. VerticalMargin attribute visual result

6.3.13. VisualGroupParameter

```
[VisualGroupParameter("Salary", "Background", "LightPink")]
public class Position
```

```

{
    [VisualGroupName(Name="Main")]
    public string Name {get; set;}

    [VisualGroupName(Name = "Salary")]
    public int MaxSalary {get; set;}

    [VisualGroupName(Name = "Salary")]
    public int MinSalary {get; set;}

    [VisualGroupName(Name = "Main")]
    public string Comments {get; set;}
}

```

Figure 90. VisualGroupParameter attribute visual result

6.3.14. VisualGroupUseControl

```

[VisualGroupUseControl(VisualGroupName="Salary",Name="Expander")]
public class Position
{
    [VisualGroupName(Name="Main")]
    public string Name {get; set;}

    [VisualGroupName(Name = "Salary")]
    public int MaxSalary {get; set;}

    [VisualGroupName(Name = "Salary")]
    public int MinSalary {get; set;}
}

```



```

[VisualGroupName(Name = "Main")]
public string Comments {get; set;}
}

```

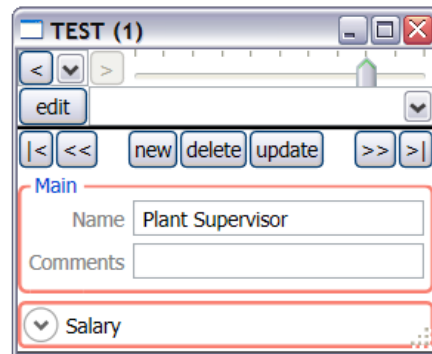


Figure 91. VisualGroupUseControl attribute visual result

6.3.15. VisualizerParameter

```

[VisualizerParameter("BorderBrush", "Red")]
[VisualizerParameter("BorderThickness", "50")]
[VisualizerParameter("BorderThickness", "5", "Bulk")]
[VisualizerParameter("Background", "Green")]

public class Position
{
    public string Name {get; set;}
    public int MaxSalary {get; set;}
    public int MinSalary {get; set;}
    public string Comments {get; set;}
}

```

	Name	MaxSalary	MinSalary	Comments
	Engineer	787	468	
	Junior Engineer	847	182	
	Senior Engineer	931	407	
	Junior Menager	855	360	
	Senior Menager	722	290	
	Electrician	923	386	
	Electric Master	993	471	
	Plant Supervisor	897	359	
	Electro king	674	434	

new

Figure 92. VisualizerParameter attribute visual result in *Bulk* view

Name	Senior Menager
MaxSalary	722
MinSalary	290
Comments	

Figure 93. VisualizerParameter attribute visual result in *Detail* view

7. Solution applications, limitations and future work

7.1. Applications

“Zero” framework was designed and programmed with commercial, line-of-business usage in mind. We had successfully developed few “zero” based real world commercial applications.

Basic “zero” usage scenario is described as time/money/programmers resources limited – generated UI is thought as not being “best” solution, while hand coded one is. But as practice shows, in many cases “zero” generated UI is even better than traditionally hand coded one. As such “zero” framework could be successfully used in most line-of-business applications.

7.2. Limitations

Currently “zero” framework does not support any object graph or tree view.

As being Windows Presentation Foundation based, “zero” requires modern cpu & gpu. Not all computers currently present in business environment meet those standards.

7.3. Future work

As “zero” framework seems to have sound commercial usage scenarios it probably will be developed further. New TypeSchema generators, VisualizerItems generators and Visualizers can be developed to better suit all line-of-business application needs.

As WindowsForms UI framework does not require modern cpu & gpu, WindowsForms support could be implemented thus expanding supported hardware base.

Summary

“Zero” conceptual framework in form of target UI medium independent generation framework was presented. Two target UI generations infrastructures were implemented: ASP.NET based for WWW sessions and Windows Presentation Foundation based for windows native client sessions.

Bibliography

- [1]. Microsoft Access. <http://microsoft.com/access>
- [2]. Naked Objects. <http://www.nakedobjects.org>
- [3]. Dynamic Data. <http://www.asp.net/dynamicdata/>
- [4]. XAML Power Toys. <http://karlshifflett.wordpress.com/xaml-power-toys/>
- [5]. M.Trzaska. *Usability of Visual Information Retrieval Metaphors for Object-Oriented Databases*
- [6]. ASP.NET. <http://www.asp.net>
- [7]. Windows Presentation Foundation.
<http://www.microsoft.com/net/WindowsPresentationFoundation.aspx>
- [8]. C# 3.0. <http://msdn.microsoft.com/en-us/vcsharp>
- [9]. Visual Studio 2008. <http://msdn.microsoft.com/en-us/vstudio>
- [10]. Model-driven architecture at Wikipedia. http://en.wikipedia.org/wiki/Model-driven_architecture
- [11]. Domain specific language at Wikipedia. http://en.wikipedia.org/wiki/Domain_Specific_Language
- [12]. Richard Pawson. *Naked Objects*
- [13]. Model at Wikipedia. [http://en.wikipedia.org/wiki/Model_\(abstract\)](http://en.wikipedia.org/wiki/Model_(abstract))
- [14]. Microsoft .NET <http://www.microsoft.com/NET/>
- [15]. ODRA database http://www.sqpl.pl/various/ODRA/ODRA_manual.html

Appendix A. PowerPlant model

In this paper we use PowerPlant model in all examples. Below we present PowerPlant C# source with is used as model.

```
[Serializable]
public class PowerPlant
{
    [Ignore]
    static public PowerPlant _PowerPlant { get; set; }

    [OnDeserializedAttribute()]
    private void AfterDeserialization(StreamingContext context){
        _PowerPlant = this;
    }

    public List<Device> Devices { get; set; }

    public List<Employee> Employees { get; set; }

    public List<WorkDivision> WorkDivisions { get; set; }

    public List<pjwstk.DeviceBrand> DeviceBrands { get; set; }

    public List<pjwstk.DeviceInspectionStatus> DeviceInspectionStatuses { get; set; }

    public List<pjwstk.DeviceModel> DeviceModels { get; set; }

    public List<pjwstk.DeviceStatus> DeviceStatuses { get; set; }

    public List<pjwstk.Position> Positions { get; set; }

    public List<pjwstk.StateReportValueClass> StateReportValueClasses { get; set; }
}

[Serializable]
public class PhysicalAddress
{
    public override string ToString()
    {
        return Street + " " + PostalCode + " " + City + " " + Country;
    }

    public PhysicalAddress(String street, String city, String postalcode, String country)
    {
        Street = street;
        City = city;
        PostalCode = postalcode;
        Country = country;
    }

    public PhysicalAddress()
    {
    }

    public String Street { get; set; }

    public String City { get; set; }

    public String PostalCode { get; set; }

    public String Country { get; set; }
}

[Serializable]
public class EmailAddress
```

```

{
    public override string ToString()
    {
        return Address;
    }

    public String Address { get; set; }

    public EmailAddress() { }

    public EmailAddress(String address)
    {
        Address = address;
    }
}

[Serializable]
public class PhoneNumber : IComparable
{
    public override string ToString()
    {
        return Number;
    }

    public String Number { get; set; }

    public String Name { get; set; }

    #region IComparable Members

    public int CompareTo(object obj)
    {
        return Number.CompareTo(((PhoneNumber)obj).Number);
    }

    #endregion
}

public interface IContactable
{
    List<PhysicalAddress> PhysicalAddresses { get; set; }

    List<EmailAddress> EmailAddresses { get; set; }
    List<PhoneNumber> PhoneNumbers { get; set; }
}

[Serializable]
public class DeviceBrand: IContactable
{
    public String Name { get; set; }
    public List<PhysicalAddress> PhysicalAddresses { get; set; }
    public List<EmailAddress> EmailAddresses { get; set; }
    public List<PhoneNumber> PhoneNumbers { get; set; }

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class DeviceModel
{
    public String Name { get; set; }

    [ObjectSourceCollection(Name="DeviceBrands")]
    public DeviceBrand Brand { get; set; }

    public override string ToString()
    {
        return Name;
    }
}

```

```

    }
}

[Serializable]
public class Position
{
    public string Name {get; set;}
    public int MaxSalary {get; set;}
    public int MinSalary {get; set;}
    public string Comments {get; set;}

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class WorkDivision
{
    public string Name {get;set;}
    public string Comments {get; set;}

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class Employee:IContactable
{
    [VisualName(Name="Employee's Name")]
    [Description(Text="This is employees name")]
    public string Name {get; set;}
    public string Surname {get; set;}

    [TextMultiline]
    public string Comments { get; set; }

    [ObjectSourceCollection(Name = "Positions")]
    public Position Position {get; set;}

    [ObjectSourceCollection(Name = "WorkDivisions")]
    public WorkDivision WorkDivision {get;set;}

    public int Salary {get; set;}

    public List<Device> ResponsibleForDevices {get;set;}

    public List<PhysicalAddress> PhysicalAdresses { get; set; }
    public List<EmailAddress> EmailAdresses { get; set; }
    public List<PhoneNumber> PhoneNumbers { get; set; }

    public void SendMessage(string message){
    }

    public void IncreaseSalaryBy10Percent (){
        Salary += Salary / 10;
    }

    public void SetSalaryToPositionMaxSalary(){
        Salary = Position.MaxSalary;
    }

    public void SetSalaryToPositionMinSalary(){
        Salary = Position.MinSalary;
    }

    public void SetSalaryToWorkDivisionAvarage(){

```



```

        Salary = (int)PowerPlant._PowerPlant.Employees.Where(e => e.WorkDivision ==
this.WorkDivision).Average(e => e.Salary);
    }

}

[Serializable]
public class DeviceInspectionStatus
{
    public string Name { get; set; }

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class DeviceStatus
{
    public string Name { get; set; }

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class StateReportValueClass
{
    public string Name {get;set;}
    public string Comments {get; set;}

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class StateReportValue
{
    [ObjectSourceCollection(Name = "StateReportValueClasses")]
    public StateReportValueClass Class {get; set;}
    public string Value {get; set;}

    public override string ToString()
    {
        return Value+" : "+Class.Name;
    }
}

[Serializable]
public class StateReport
{
    public string Name {get; set;}

    [TextMultiLine()]
    public string Comments {get; set;}
    public List<StateReportValue> Values {get; set;}

    public override string ToString()
    {
        return Name;
    }
}

[Serializable]
public class Device
{

```

```

[ObjectSourceCollection(Name = "DeviceModels")]
public DeviceModel Model {get;set;}

[ObjectSourceCollection(Name = "DeviceInspectionStatuses")]
public DeviceInspectionStatus InspectionStatus { get; set; }

[ObjectSourceCollection(Name = "DeviceStatuses")]
public DeviceStatus Status { get; set; }

public List<Device> SendsPowerTo { get; set; }

public override string ToString()
{
    return Model.Name;
}

public IList<Employee> GetResponsibleEngineers()
{
    return PowerPlant._PowerPlant.Employees.Where(e => e.ResponsibleForDevices.Any(d => d
== this)).ToList();
}

public void StartDevice()
{
    Status = PowerPlant._PowerPlant.DeviceStatuses[1];
}

public void StopDevice()
{
    Status = PowerPlant._PowerPlant.DeviceStatuses[0];
}

public StateReport GetStateReport()
{
    StateReport r = new StateReport();

    r.Name = "State report for " + this.GetHashCode();

    r.Comments = "This is " + this.Model.Brand.Name + "'s " + this.Model.Name +
"\nDevice is " + this.Status.Name + " and its inspection status is " +
this.InspectionStatus.Name + "\ndevice sends power to " + this.SendsPowerTo.Count + " devices.
Device has " + PowerPlant._PowerPlant.Employees.Where(e => e.ResponsibleForDevices.Any(d => d ==
this)).Count() + " responsible engineers.";

    r.Values = new List<StateReportValue>();

    Random rnd = new Random();

    int i = rnd.Next(10);

    for (int a = 0; a < i; a++)
    {
        StateReportValue v = new StateReportValue();

        v.Class = PowerPlant._PowerPlant.StateReportValueClasses[rnd.Next(0,
PowerPlant._PowerPlant.StateReportValueClasses.Count)];

        v.Value = rnd.Next(100).ToString();

        r.Values.Add(v);
    }

    return r;
}
}

```

Above C# class is show on *semi UML like* diagram below.

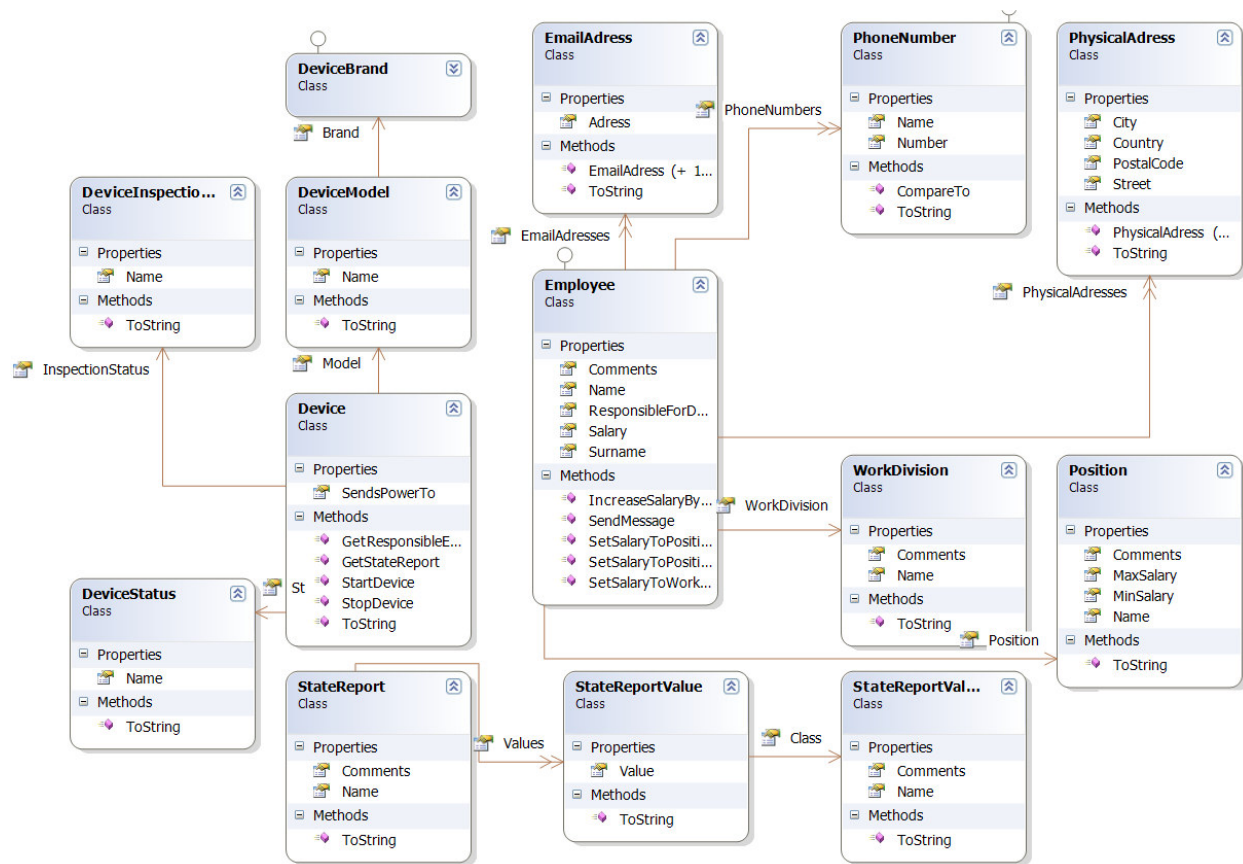


Figure 94. PowerPlant model semi UML diagram

Appendix B. PowerPlant.Employee TypeSchema

```
<?xml version="1.0" encoding="utf-8"?>
<TypeSchema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xsi:type="ClrTypeSchema" IsGenericType="false"
IsIEnumerable="false" IsIListable="false" IsInterface="false">
  <Attributes>
    <ZAttribute Name="System.SerializableAttribute">
      <Items />
    </ZAttribute>
  </Attributes>
  <Name>pjwstk.PowerPlant, PowerPlant, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null</Name>
  <TypeName>pjwstk.PowerPlant, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null</TypeName>
  <ShortTypeName>PowerPlant</ShortTypeName>
  <DerivesFrom>System.Object, mscorlib, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</DerivesFrom>
  <Fields>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="false" IsIEnumerable="false"
IsIListable="false" CanRead="true" CanWrite="true" IsProperty="true">
      <Attributes>
        <ZAttribute Name="Zero.Ignore">
          <Items />
        </ZAttribute>
      </Attributes>
      <Name>_PowerPlant</Name>
      <TypeName>pjwstk.PowerPlant, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null</TypeName>
      <ShortTypeName>PowerPlant</ShortTypeName>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
IsIListable="true" CollectionOf="pjwstk.Device, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
      <Name>Devices</Name>
      <TypeName>System.Collections.Generic.List`1[[pjwstk.Device, PowerPlant, Version=1.0.0.0,
Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</TypeName>
      <ShortTypeName>List`1</ShortTypeName>
      <GenericTypeParameters>
        <string>pjwstk.Device, PowerPlant, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null</string>
      </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
IsIListable="true" CollectionOf="pjwstk.Employee, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
      <Name>Employees</Name>
      <TypeName>System.Collections.Generic.List`1[[pjwstk.Employee, PowerPlant, Version=1.0.0.0,
Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</TypeName>
      <ShortTypeName>List`1</ShortTypeName>
      <GenericTypeParameters>
        <string>pjwstk.Employee, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null</string>
      </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
IsIListable="true" CollectionOf="pjwstk.WorkDivision, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
      <Name>WorkDivisions</Name>
      <TypeName>System.Collections.Generic.List`1[[pjwstk.WorkDivision, PowerPlant, Version=1.0.0.0,
Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</TypeName>
      <ShortTypeName>List`1</ShortTypeName>
      <GenericTypeParameters>
        <string>pjwstk.WorkDivision, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null</string>
      </GenericTypeParameters>
    </TypeSchemaField>
  </Fields>
</TypeSchema>
```

```

        </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
    IsIListable="true" CollectionOf="pjwstk.DeviceBrand, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
        <Name>DeviceBrands</Name>
        <TypeName>System.Collections.Generic.List`1[[pjwstk.DeviceBrand, PowerPlant, Version=1.0.0.0,
    Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
    PublicKeyToken=b77a5c561934e089</TypeName>
        <ShortTypeName>List`1</ShortTypeName>
        <GenericTypeParameters>
            <string>pjwstk.DeviceBrand, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null</string>
        </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
    IsIListable="true" CollectionOf="pjwstk.DeviceInspectionStatus, PowerPlant, Version=1.0.0.0,
    Culture=neutral, PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
        <Name>DeviceInspectionStatuses</Name>
        <TypeName>System.Collections.Generic.List`1[[pjwstk.DeviceInspectionStatus, PowerPlant,
    Version=1.0.0.0, Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
    PublicKeyToken=b77a5c561934e089</TypeName>
        <ShortTypeName>List`1</ShortTypeName>
        <GenericTypeParameters>
            <string>pjwstk.DeviceInspectionStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null</string>
        </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
    IsIListable="true" CollectionOf="pjwstk.DeviceModel, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
        <Name>DeviceModels</Name>
        <TypeName>System.Collections.Generic.List`1[[pjwstk.DeviceModel, PowerPlant, Version=1.0.0.0,
    Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
    PublicKeyToken=b77a5c561934e089</TypeName>
        <ShortTypeName>List`1</ShortTypeName>
        <GenericTypeParameters>
            <string>pjwstk.DeviceModel, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null</string>
        </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
    IsIListable="true" CollectionOf="pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
        <Name>DeviceStatuses</Name>
        <TypeName>System.Collections.Generic.List`1[[pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0,
    Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
    PublicKeyToken=b77a5c561934e089</TypeName>
        <ShortTypeName>List`1</ShortTypeName>
        <GenericTypeParameters>
            <string>pjwstk.DeviceStatus, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null</string>
        </GenericTypeParameters>
    </TypeSchemaField>
    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
    IsIListable="true" CollectionOf="pjwstk.Position, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
        <Name>Positions</Name>
        <TypeName>System.Collections.Generic.List`1[[pjwstk.Position, PowerPlant, Version=1.0.0.0,
    Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
    PublicKeyToken=b77a5c561934e089</TypeName>
        <ShortTypeName>List`1</ShortTypeName>
        <GenericTypeParameters>
            <string>pjwstk.Position, PowerPlant, Version=1.0.0.0, Culture=neutral,
    PublicKeyToken=null</string>
        </GenericTypeParameters>
    </TypeSchemaField>

```

```

    <TypeSchemaField xsi:type="ClrTypeSchemaField" IsGenericType="true" IsIEnumerable="true"
IsIListable="true" CollectionOf="pjwtst.StateReportValueClass, PowerPlant, Version=1.0.0.0,
Culture=neutral, PublicKeyToken=null" CanRead="true" CanWrite="true" IsProperty="true">
    <Name>StateReportValueClasses</Name>
    <TypeName>System.Collections.Generic.List`1[[pjwtst.StateReportValueClass, PowerPlant,
Version=1.0.0.0, Culture=neutral, PublicKeyToken=null]], mscorlib, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</TypeName>
    <ShortTypeName>List`1</ShortTypeName>
    <GenericTypeParameters>
    <string>pjwtst.StateReportValueClass, PowerPlant, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null</string>
    </GenericTypeParameters>
    </TypeSchemaField>
</Fields>
<Methods />
<ClassName>PowerPlant</ClassName>
<Namespace>pjwtst</Namespace>
</TypeSchema>

```